VULKAN

Explicit GPU programming from first principles

UNIT 2

Compute Shaders

The execution model, shader memory, parallel patterns, performance and a ray tracer

2.1 The compute execution model

2.2 Memory in shaders

2.3 Parallel patterns

2.4 Images and image processing

2.5 Measuring and tuning

2.6 Dispatch at scale

2.7 A ray tracer in a compute shader

First edition, October 2026

Contents

About this unit 2
Before you start 2
What this unit needs 2
2.1 The compute execution model 3
Core ideas 3
Reading 9
Labs 9
Problem set 11
2.2 Memory in shaders 12
Core ideas 12
Reading 19
Labs 19
Problem set 21
2.3 Parallel patterns 22
Core ideas 22
Reading 33
Labs 34
Problem set 35
2.4 Images and image processing 36
Core ideas 36
Reading 43
Labs 43
Problem set 45
2.5 Measuring and tuning 46
Core ideas 46
Reading 54
Labs 54
Problem set 55
2.6 Dispatch at scale 57
Core ideas 57
Reading 63
Labs 64
Problem set 65
2.7 A ray tracer in a compute shader 66
Core ideas 66
Reading 78
Labs 78
Problem set 79
Sign-off 81

Vulkan, First edition, October 2026. © 2026 O'Neill Group LLC. Vulkan and the Vulkan logo are registered trademarks of the Khronos Group Inc. OpenGL and the oval logo are trademarks or registered trademarks of Hewlett Packard Enterprise in the United States and other countries, used by permission by Khronos. All other trademarks belong to their owners. This course is not endorsed by the Khronos Group.

UNIT 2

About this unit

Unit 2 is about the GPU as a parallel computer. It explains how a dispatch becomes workgroups and subgroups on the hardware, how shaders use memory, and why access patterns decide their speed. It builds the classic parallel patterns (reduction, scan, compaction, histograms and sorting) and makes them fast. It processes images through storage images and samplers, measures kernels against the hardware's limits, handles problems too large for one dispatch, and ends with a ray tracer written entirely in compute shaders. From this unit on, programs use the `vkf` helper library from Chapter 1.6.

Chapter	You build	Hours
2.1	A map of invocations to workgroups and subgroups; a sweep of local sizes	5
2.2	Three transposes, a privatised histogram, two races observed and fixed	6
2.3	Reductions at the bandwidth limit, scans, compaction and a stable radix sort	7
2.4	A tiled separable blur, a sampled downscale, divergence measured in a fractal	6
2.5	A bandwidth benchmark, a measured roofline, and autotuning by specialisation	6
2.6	An N-body simulation with indirect dispatch and buffer device addresses	6
2.7	A path tracer with a bounding volume hierarchy, checked and measured	8

Before you start

Finish Unit 1, and run `vkf_selftest`. Read the Handbook's part on how GPUs work if you have not.

What this unit needs

The same machine and tools as Unit 1. A discrete GPU makes the measurements of Chapters 2.2 and 2.5 more instructive, but every lab runs on integrated graphics.

CHAPTER 2.1

The compute execution model

TIME	5 hours
YOU BUILD	A program that shows which workgroup and subgroup runs each invocation of a dispatch and checks every identity between Vulkan's built-in indices; and a sweep of workgroup sizes for a memory-bound and an arithmetic-bound kernel.
NEEDS	Unit 1.

WHY IT MATTERS

Unit 1 dispatched SAXPY in workgroups of 256 invocations without explaining the choice. This chapter explains how a dispatch is divided into workgroups and subgroups, what each level guarantees and what it does not, and how the division reaches the GPU's compute units. Everything else in Unit 2 builds on it: shared memory and barriers belong to workgroups (Chapter 2.2), the fastest parallel algorithms work one subgroup at a time (Chapters 2.3 and 2.5), and the sizes you choose decide how much of the hardware a kernel can use.

SKIP TEST

1. A dispatch of 4 × 2 workgroups has a local size of 8 × 8. How many invocations run, and what is `gl_GlobalInvocationID` for the invocation with `gl_WorkGroupID` (1, 1) and `gl_LocalInvocationID` (3, 5)?
2. What may the invocations of one workgroup do together that the invocations of different workgroups may not?
3. A workgroup of 36 invocations runs on a GPU whose subgroups have 32. How many subgroups does it occupy, and how many lanes do nothing?
4. Why are workgroup sizes usually multiples of 32 or 64?

If all four are easy, skim the core ideas and do Labs 2.1.2 and 2.1.3.

Core ideas

Invocations, workgroups and dispatches

A compute shader is run by *invocations*, one instance of `main` each. Invocations are grouped into *workgroups* whose size, the *local size*, the shader declares in up to three dimensions. A dispatch runs a grid of workgroups whose dimensions `vkCmdDispatch` gives (Figure 2.1.1). Each invocation finds its place through built-in variables:

- `gl_NumWorkGroups` and `gl_WorkGroupSize`: the grid's dimensions in workgroups, and the local size.
- `gl_WorkGroupID`: which workgroup the invocation belongs to.
- `gl_LocalInvocationID`: its position within the workgroup, and `gl_LocalInvocationIndex`, the same position as a single number, `z·X·Y + y·X + x` for a local size of X × Y × Z.
- `gl_GlobalInvocationID`: its position in the whole grid, equal to `gl_WorkGroupID * gl_WorkGroupSize + gl_LocalInvocationID`.

The first program of this unit records every built-in for every invocation of a small dispatch. Its shader writes one record per invocation, at the position its global index gives:

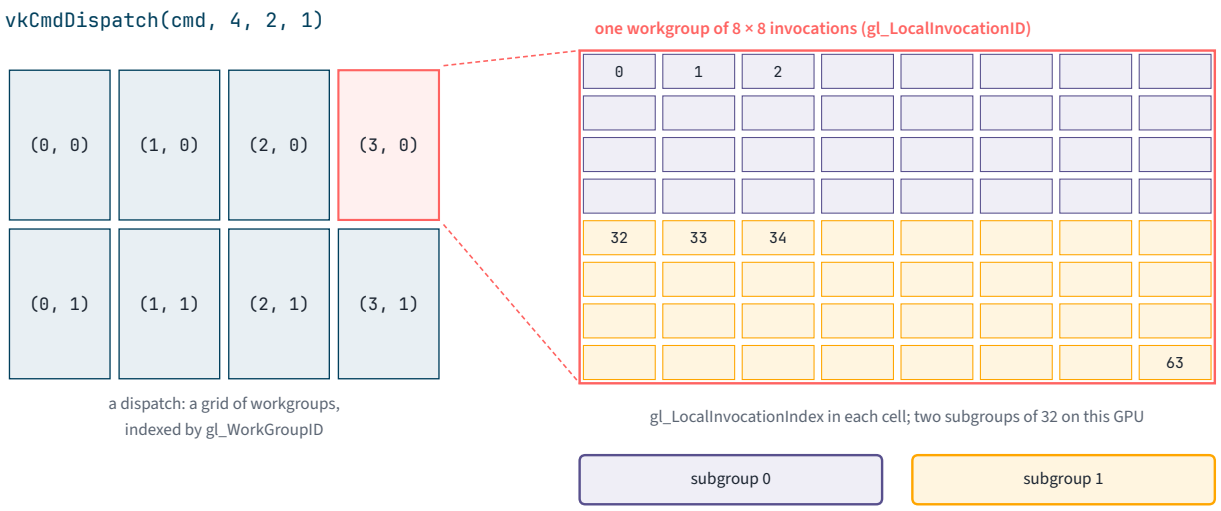


Figure 2.1.1. A dispatch of 4 × 2 workgroups, each of 8 × 8 invocations. On the M2 Pro each workgroup is divided into two subgroups of 32 invocations.

A record of everything an invocation knows about its place — code/u2/indexing/indexing.comp

```
layout(local_size_x_id = 0, local_size_y_id = 1) in;

struct Record {
    uvec2 globalId;
    uvec2 localId;
    uvec2 groupId;
    uint localIndex;
    uint subgroupId;
    uint subgroupInvocation;
    uint subgroupSize;
};

layout(std430, set = 0, binding = 0) writeonly buffer Records {
    Record records[];
};
```

Each invocation writes its own record — code/u2/indexing/indexing.comp

```
void main() {
    uvec2 grid = gl_NumWorkGroups.xy * gl_WorkGroupSize.xy;
    uvec2 id = gl_GlobalInvocationID.xy;
    records[id.y * grid.x + id.x] = Record(id, gl_LocalInvocationID.xy, gl_WorkGroupID.xy,
                                           gl_LocalInvocationIndex, gl_SubgroupID,
                                           gl_SubgroupInvocationID, gl_SubgroupSize);
}
```

The C++ side mirrors the record exactly, as Chapter 1.5 taught, and checks its size against the `std430` layout:

The host's view of a record — code/u2/indexing/main.cpp

```
struct Record {
    uint32_t globalId[2];
    uint32_t localId[2];
    uint32_t groupId[2];
    uint32_t localIndex;
    uint32_t subgroupId;
    uint32_t subgroupInvocation;
    uint32_t subgroupSize;
};
static_assert(sizeof(Record) == 40, "Record must match the std430 layout in indexing.comp");
```

The program fills the buffer with `0xFFFFFFFF` before the dispatch, so that it can detect the record of an invocation that never ran. It then checks every identity for every invocation, and draws the grid twice: once by workgroup, once by subgroup.

Checking the identities on the CPU — code/u2/indexing/main.cpp

```
uint32_t written = 0, globalOk = 0, indexOk = 0, linear = 0;
std::set<uint32_t> subgroupSizes;
std::vector<std::set<std::pair<uint32_t, uint32_t>>> lanes(groupCount);
for (uint32_t y = 0; y < grid.y; ++y) {
    for (uint32_t x = 0; x < grid.x; ++x) {
        const Record& r = result[y * grid.x + x];
        if (r.localIndex == 0xFFFFFFFF) continue;
        ++written;
        const bool global = r.globalId[0] == x && r.globalId[1] == y &&
                            x == r.groupId[0] * local.x + r.localId[0] &&
                            y == r.groupId[1] * local.y + r.localId[1];
        if (!global) continue;
        ++globalOk;
        if (r.localIndex == r.localId[1] * local.x + r.localId[0]) ++indexOk;
        if (r.subgroupInvocation >= r.subgroupSize) continue;
        subgroupSizes.insert(r.subgroupSize);
        lanes[r.groupId[1] * groups.x + r.groupId[0]].insert(
            {r.subgroupId, r.subgroupInvocation});
        if (r.localIndex == r.subgroupId * r.subgroupSize + r.subgroupInvocation) {
            ++linear;
        }
    }
}
const auto uniqueLanes = static_cast<uint32_t>(
    std::count_if(lanes.begin(), lanes.end(),
                  [&](const auto& s) { return s.size() == local.x * local.y; }));
```

A dispatch of 4 × 2 workgroups of 8 × 8 on the M2 Pro: the device's compute limits, which workgroup and which subgroup ran each position, and the checks

```
$ ./build/bin/u2_indexing
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
maxComputeWorkGroupCount       1073741824 x 1073741824 x 1073741824
maxComputeWorkGroupSize        1024 x 1024 x 1024
maxComputeWorkGroupInvocations 1024
maxComputeSharedMemorySize     32768 bytes
subgroupSize                   32 (minSubgroupSize 4, maxSubgroupSize 32)
subgroupSupportedOperations    basic vote arithmetic ballot shuffle shuffle_relative
    clustered quad

dispatch: 4 x 2 workgroups of 8 x 8 invocations, a 32 x 16 grid
gl_SubgroupSize 32, 2 subgroup(s) per workgroup

gl_WorkGroupID as y * 4 + x         gl_SubgroupID
00000000111111112222222233333333    00000000000000000000000000000000
00000000111111112222222233333333    00000000000000000000000000000000
00000000111111112222222233333333    00000000000000000000000000000000
00000000111111112222222233333333    00000000000000000000000000000000
00000000111111112222222233333333    11111111111111111111111111111111
00000000111111112222222233333333    11111111111111111111111111111111
00000000111111112222222233333333    11111111111111111111111111111111
00000000111111112222222233333333    11111111111111111111111111111111
44444444555555556666666677777777    00000000000000000000000000000000
44444444555555556666666677777777    00000000000000000000000000000000
44444444555555556666666677777777    00000000000000000000000000000000
44444444555555556666666677777777    00000000000000000000000000000000
44444444555555556666666677777777    11111111111111111111111111111111
44444444555555556666666677777777    11111111111111111111111111111111
44444444555555556666666677777777    11111111111111111111111111111111
44444444555555556666666677777777    11111111111111111111111111111111

invocations that wrote their record                                  512/512
GlobalInvocationID = WorkGroupID * WorkGroupSize + LocalInvocationID  512/512
```

```
LocalInvocationIndex = LocalInvocationID.y * 8 + LocalInvocationID.x        512/512
(SubgroupID, SubgroupInvocationID) unique within each workgroup               8/8
LocalInvocationIndex = SubgroupID * SubgroupSize + SubgroupInvocationID     512/512
  (the last line is the implementation's choice: Vulkan does not define it)
PASS indexing: 512 invocations, every identity holds
```

What a workgroup guarantees

The invocations of one workgroup run on the same compute unit, at the same time. They can therefore share variables declared `shared` and wait for each other with `barrier()`, which invocations of different workgroups cannot do. Chapter 2.2 covers both.

Different workgroups get no such guarantees. They run in no defined order: all at once, a few at a time, or one after another, depending on the GPU, the size of the dispatch and whatever else is running. They can communicate only through memory, with atomic operations. A workgroup must never wait for another to do something, because Vulkan does not guarantee that the other will run while the first is waiting. A kernel that needs every workgroup to finish a step before any starts the next must end and let a second dispatch, after a barrier, do the next step. Chapter 2.3's multi-pass algorithms are built this way.

> **KEY IDEA**
> A workgroup is the largest group of invocations that can cooperate. Design an algorithm so that each workgroup does an independent piece of work, and combine the pieces in a later dispatch.

Each level has limits that a program must query, which the program above prints first. The specification guarantees at least 128 invocations per workgroup (`maxComputeWorkGroupInvocations`), local sizes of at least 128 × 128 × 64 (`maxComputeWorkGroupSize`, with the total still bounded by the first limit), 65,535 workgroups in each dimension of a dispatch (`maxComputeWorkGroupCount`) and 16 KiB of shared memory per workgroup (`maxComputeSharedMemorySize`). Most desktop GPUs allow 1024 invocations and 32 to 64 KiB.

Refusing sizes the device cannot run — code/u2/indexing/main.cpp

```
void checkLimits(const VkPhysicalDeviceLimits& limits, Extent groups, Extent local) {
    if (local.x == 0 || local.y == 0 || local.x > limits.maxComputeWorkGroupSize[0] ||
        local.y > limits.maxComputeWorkGroupSize[1] ||
        local.x * local.y > limits.maxComputeWorkGroupInvocations) {
        throw std::runtime_error(std::format(
            "a local size of {} x {} exceeds this device's limits", local.x, local.y));
    }
    if (groups.x == 0 || groups.y == 0 || groups.x > limits.maxComputeWorkGroupCount[0] ||
        groups.y > limits.maxComputeWorkGroupCount[1]) {
        throw std::runtime_error(
            std::format("{} x {} workgroups exceed this device's limits", groups.x, groups.y));
    }
}
```

Subgroups

Within a workgroup, the hardware runs invocations in *subgroups* that execute in lockstep, as Part 3 of the Handbook described. Vulkan exposes them through `VkPhysicalDeviceVulkan11Properties::subgroupSize` and, in shaders that enable `GL_KHR_shader_subgroup_basic`, through `gl_SubgroupSize`, `gl_NumSubgroups`, `gl_SubgroupID` and `gl_SubgroupInvocationID`. The subgroup size is 32 on NVIDIA and Apple GPUs; 64 on AMD's older GCN GPUs, and 32 or 64 on its RDNA GPUs, depending on the driver and the shader; and 8, 16 or 32 on Intel's, where the compiler chooses per shader. The M2 Pro reports a size of 32 and a minimum of 4.

How a workgroup's invocations are divided among subgroups is the implementation's choice. On the M2 Pro the division is linear: invocations 0 to 31 form subgroup 0, 32 to 63 subgroup 1. The program checks that this holds but does not require it, because another GPU may divide differently. Code that needs a particular arrangement must check for it, or request what Vulkan can guarantee: full subgroups, as Chapter 2.3 does, and on some devices a fixed subgroup size, as Chapter 2.5 describes.

A workgroup whose size is not a multiple of the subgroup size wastes lanes. With a local size of 6 × 6, each workgroup has 36 invocations and occupies two subgroups of 32, the second with only four active lanes:

36 invocations need two subgroups: 28 of the 64 lanes do nothing

```
$ ./build/bin/u2_indexing --groups 4,2 --local 6,6
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
maxComputeWorkGroupCount       1073741824 x 1073741824 x 1073741824
maxComputeWorkGroupSize        1024 x 1024 x 1024
maxComputeWorkGroupInvocations 1024
maxComputeSharedMemorySize     32768 bytes
subgroupSize                   32 (minSubgroupSize 4, maxSubgroupSize 32)
subgroupSupportedOperations    basic vote arithmetic ballot shuffle shuffle_relative
    clustered quad

dispatch: 4 x 2 workgroups of 6 x 6 invocations, a 24 x 12 grid
gl_SubgroupSize 32, 2 subgroup(s) per workgroup

gl_WorkGroupID as y * 4 + x    gl_SubgroupID
00000011111222222333333    00000000000000000000000000
00000011111222222333333    00000000000000000000000000
00000011111222222333333    00000000000000000000000000
00000011111222222333333    00000000000000000000000000
00000011111222222333333    00000000000000000000000000
00000011111222222333333    00111100111100111001111
44444455555666666777777    00000000000000000000000000
44444455555666666777777    00000000000000000000000000
44444455555666666777777    00000000000000000000000000
44444455555666666777777    00000000000000000000000000
44444455555666666777777    00000000000000000000000000
44444455555666666777777    00111100111100111001111

invocations that wrote their record                                    288/288
GlobalInvocationID = WorkGroupID * WorkGroupSize + LocalInvocationID   288/288
LocalInvocationIndex = LocalInvocationID.y * 6 + LocalInvocationID.x   288/288
(SubgroupID, SubgroupInvocationID) unique within each workgroup          8/8
LocalInvocationIndex = SubgroupID * SubgroupSize + SubgroupInvocationID 288/288
  (the last line is the implementation's choice: Vulkan does not define it)
PASS indexing: 288 invocations, every identity holds
```

PITFALL

Never assume a subgroup size, the mapping of invocations to subgroups, or the order in which workgroups run. Each differs between GPUs, and a kernel that relies on what one GPU happens to do will fail on another, often silently. Query the size, use the subgroup built-ins instead of computing them, and let separate dispatches order work that must be ordered.

How a dispatch reaches the hardware

When a dispatch starts, the GPU's command processor hands workgroups to compute units. Each compute unit takes as many workgroups as its resources allow: registers, which every invocation needs, shared memory, which every workgroup needs, and a fixed number of slots for workgroups and subgroups. A workgroup stays on its compute unit until all its invocations finish, and then the unit takes another.

It follows that a dispatch must be large enough to keep every compute unit busy, with several workgroups each, or part of the GPU idles; a large GPU needs tens of thousands of invocations in flight. Very small workgroups can also leave a compute unit underused even when there are many of them, because it runs out of workgroup slots before it runs out of lanes.

Choosing the local size

The rules of thumb follow from the hardware. Use a multiple of the subgroup size; 64 is a multiple of every common size. For one-dimensional work, 64 to 256 invocations suit most kernels; for images, 8×8 or 16×16. Go larger only for a reason, such as a tile of shared memory that a bigger workgroup can reuse. Then measure, because the best size depends on the kernel and the GPU.

Specialisation constants make the local size a parameter of the pipeline instead of the source, as Chapter 1.4 showed. The second program builds a pipeline for every power of two from 32 to the device's limit, for two kernels: SAXPY, which moves twelve bytes for every two operations, and a chain of 512 fused multiply-

adds per element, which does far more arithmetic than memory traffic. It times each with `u2::timeGpu`, which records many runs with timestamps and returns the median; Chapter 2.5 explains how.

One pipeline per local size, timed and checked — code/u2/localsize/main.cpp

```
vkf::Specialization spec;
spec.set(0, local);
auto saxpyPipeline = u2::loadPipeline(ctx, layout, "saxpy.comp", spec);
auto chainPipeline = u2::loadPipeline(ctx, layout, "fma_chain.comp", spec);
const uint32_t groups =
    std::min(vkf::groupCount(n, local), limits.maxComputeWorkGroupCount[0]);

const vkf::Stats saxpyMs = u2::timeGpu(ctx, runs, [&](VkCommandBuffer cmd) {
    record(cmd, saxpyPipeline, &saxpy, sizeof(saxpy), groups);
});
const std::vector<float> saxpyOut = vkf::download<float>(ctx, zBuffer, n);
const vkf::Stats chainMs = u2::timeGpu(ctx, runs, [&](VkCommandBuffer cmd) {
    record(cmd, chainPipeline, &fmaChain, sizeof(fmaChain), groups);
});
const std::vector<float> chainOut = vkf::download<float>(ctx, zBuffer, n);
```

Both kernels use a *grid-stride loop*: each invocation handles element `i`, then `i` plus the number of invocations in the dispatch, and so on. The loop lets a dispatch with a capped number of workgroups cover any number of elements, and makes the kernel correct for any local size.

SAXPY with a grid-stride loop and a specialised local size — code/u2/localsize/saxpy.comp

```
layout(local_size_x_id = 0) in;

layout(push_constant) uniform Push {
    float a;
    uint n;
} push;

layout(std430, set = 0, binding = 0) readonly buffer X { float x[]; };
layout(std430, set = 0, binding = 1) readonly buffer Y { float y[]; };
layout(std430, set = 0, binding = 2) writeonly buffer Z { float z[]; };

void main() {
    uint stride = gl_NumWorkGroups.x * gl_WorkGroupSize.x;
    for (uint i = gl_GlobalInvocationID.x; i < push.n; i += stride) {
        z[i] = push.a * x[i] + y[i];
    }
}
```

The sweep on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_localsize
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
SAXPY over 16777216 floats (192 MiB moved per run); an FMA chain of 512 per float
median of 10 runs after a 25 ms warm-up

local size   SAXPY ms     GB/s   FMA chain ms    GFLOP/s   results
        32      1.068    188.4          2.746     6256.9   ok
        64      1.063    189.3          2.671     6430.9   ok
       128      1.060    189.9          2.693     6379.8   ok
       256      1.059    190.2          2.694     6378.1   ok
       512      1.062    189.6          2.697     6370.8   ok
      1024      1.063    189.3          2.698     6367.1   ok
PASS localsize: 6 of 6 local sizes verified
```

On the M2 Pro the choice hardly matters. SAXPY reaches about 190 GB/s, close to the 200 GB/s that Apple quotes for the M2 Pro's memory, and is about 1% slower with workgroups of 32. The arithmetic-bound kernel runs at about 6.4 trillion operations per second whatever the size, because each invocation has enough independent work to keep the arithmetic units busy. Discrete GPUs are often more sensitive, and Lab 2.1.3 measures yours.

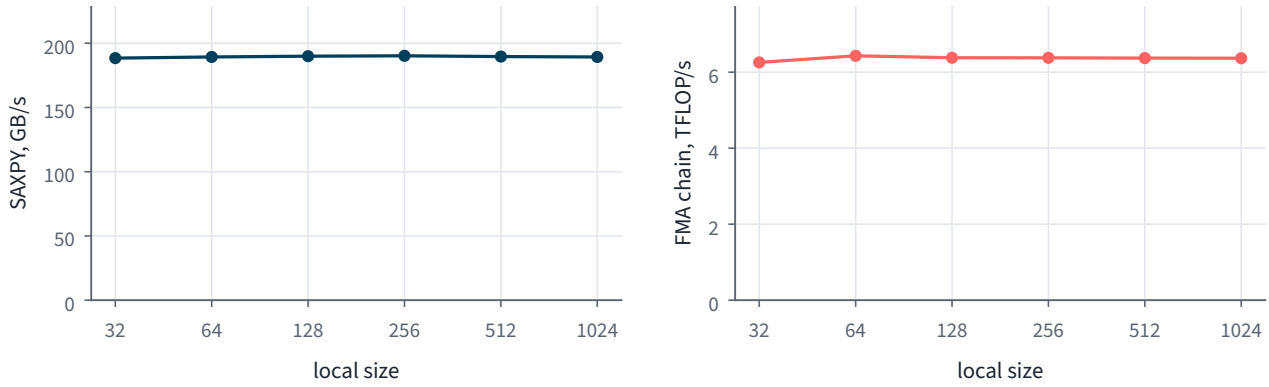


Figure 2.1.2. The sweep plotted. SAXPY is limited by memory bandwidth and reaches about 190 GB/s at every local size; the chain of multiply-adds is limited by arithmetic and changes little with the local size.

COMING FROM OPENGL

OpenGL 4.3 compute shaders use the same built-in variables and the same `layout(local_size_x = 64) in;` declaration, and `glDispatchCompute` takes the same three counts. The local size is fixed in the source; varying it needs the `ARB_compute_variable_group_size` extension, where Vulkan uses specialisation constants. Subgroup operations reached OpenGL only through the `GL_KHR_shader_subgroup` extension, which macOS does not have.

Reading

- The Vulkan specification, chapter “Dispatching Commands”, and the descriptions of the compute built-ins in “Built-In Variables”.
- The Vulkan Guide, “Compute Shaders” and “Subgroups”.
- Hwu, Kirk and El Hajj, *Programming Massively Parallel Processors*, chapters 3 and 4, on the CUDA equivalents: grids, blocks and warps.

Labs

LAB 2.1.1 Map a dispatch — 1 hour

Goal. Predict how invocations are placed, then check.

Code. `code/u2/indexing`

1. Before running anything, draw the workgroup and subgroup pictures you expect for `--groups 3,2 --local 16,4` on your GPU.
2. Run `build/bin/u2_indexing --groups 3,2 --local 16,4` and compare.
3. Run it with `--local 6,6`, and with a local size of your choice that leaves more than half of the lanes idle.

DONE WHEN

- [ ] Your predictions match the output, or you can explain each difference.
- [ ] For each local size you have the number of subgroups per workgroup and the fraction of lanes in use.

EVIDENCE FOR THE NOTEBOOK

- The predictions, the outputs and the fractions.

LAB 2.1.2 Three dimensions — 1.5 hours

Goal. Extend the program to three-dimensional dispatches.

Code. `code/u2/indexing`

1. Add `local_size_z_id = 2` to the shader and a third component to each `uvec2` in the record, and update the C++ mirror and its `static_assert`.
2. Accept three numbers in `--groups` and `--local`, and check the three-dimensional identities, including `gl_LocalInvocationIndex = z·X·Y + y·X + x`.
3. Run with `--groups 2,2,2 --local 4,4,4`.

DONE WHEN

- [ ] Every identity holds in three dimensions, with validation on.
- [ ] Your record's size and offsets are checked at compile time and match `std430`.

EVIDENCE FOR THE NOTEBOOK

- The changed record and the output.

LAB 2.1.3 Sweep the local size on your GPU — 1 hour

Goal. Find out how sensitive your GPU is to the local size.

Code. `code/u2/localsize`

1. Run `VKF_VALIDATION=0 build/bin/u2_localsize` three times and plot both throughputs against the local size.
2. Look up your GPU's memory bandwidth in its specification and compare it with SAXPY's best.
3. Run again with `--n 65536` and explain how the results change.

DONE WHEN

- [ ] You have both plots from three runs, with the spread.
- [ ] You can state what fraction of your GPU's quoted bandwidth SAXPY reaches, and why the small problem behaves differently.

EVIDENCE FOR THE NOTEBOOK

- The plots and two paragraphs of explanation.

LAB 2.1.4 Exceed the limits — 45 minutes

Goal. See what the limits protect against.

Code. `code/u2/indexing`

1. Ask for a local size larger than your device's `maxComputeWorkGroupInvocations` and record the program's refusal.
2. Remove the call to `checkLimits`, run again with validation on, and record which call fails and what the validation layer says.

DONE WHEN

- [ ] You have recorded both outcomes and the limit each concerns.
- [ ] You can say what the specification promises about a program that exceeds the limit with validation off.

EVIDENCE FOR THE NOTEBOOK

- The messages, and one sentence on the guarantee.

Problem set

1. A 75 × 40 image is processed with 16 × 16 workgroups. How many workgroups does the dispatch need, and how many invocations have no pixel? What must the shader do about them?
2. Explain why a workgroup must not wait for a flag that another workgroup of the same dispatch sets. Describe a GPU and a dispatch on which such a kernel never finishes.
3. A GPU has 40 compute units, each able to hold 32 subgroups of 32 invocations. How many invocations can be resident at once? What fraction of that does a dispatch of 8192 invocations use, and why does that matter for a memory-bound kernel?
4. From the M2 Pro's results, choose a local size for SAXPY there. What would you measure before using the same size on a discrete GPU?
5. Write `gl_LocalInvocationIndex` for a local size of X × Y × Z, and its inverse: `gl_LocalInvocationID` from the index.

COMPLETION CHECKLIST

- [ ] I can relate every compute built-in to the others, and compute dispatch sizes for any problem.
- [ ] I can state what workgroups and subgroups guarantee and what they do not.
- [ ] I can explain how workgroups are distributed to compute units, and what that implies for dispatch and workgroup sizes.
- [ ] I can choose a local size, make it a specialisation constant, and measure the choice.
- [ ] Lab 2.1.1–2.1.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.2

Memory in shaders

TIME	6 hours
YOU BUILD	A matrix transpose three ways, measured against a copy; a 256-bin histogram with global atomics and with per-workgroup bins in shared memory, on uniform and on contended input; and two data races, observed and then fixed.
NEEDS	Chapter 2.1.

WHY IT MATTERS

The speed of most kernels depends more on how they access memory than on their arithmetic. The same work can run at a fraction of its speed with a different access pattern, and a pattern that looks harmless gives wrong answers when two invocations access one address without synchronising. This chapter covers the kinds of memory a shader can use, the access patterns the hardware rewards, shared memory and the barriers that coordinate it, atomic operations, and data races. Every algorithm in the rest of the unit depends on them.

SKIP TEST

1. Which kinds of memory can a compute shader read and write, and which of them can another workgroup see?
2. Why is a naive matrix transpose slower than a copy on most GPUs, and how does a tile in shared memory help?
3. Thirty-two invocations read `tile[i][2]` from `shared float tile[32][32]`, one value of `i` each. Why might that be slow, and what does declaring `tile[32][33]` change?
4. A thousand invocations each execute `counter += 1` on the same buffer element. What can the result be, and what are two correct ways to count?
5. When is `barrier()` required, and where may it not be called?

If all five are easy, skim the core ideas and do Labs 2.2.1 and 2.2.3.

Core ideas

Where shader data lives

A compute shader can use five kinds of memory, summarised in Table 2.2.1. They differ in size, in speed, in who can see them, and in how long they last.

Kind	Declared as	Size, visibility, lifetime
Storage buffer	`buffer` block with `std430`	Up to `maxStorageBufferRange`; read and written by every invocation of every dispatch; lasts as long as the buffer.
Uniform buffer	`uniform` block, `std140` by default	Up to `maxUniformBufferRange`, at least 16 KiB; read-only; often read through a faster, cached path.
Push constants	`push_constant` block	Up to `maxPushConstantsSize`, at least 128 bytes; read-only; set per dispatch in the command buffer.
Shared memory	`shared` variables	Up to `maxComputeSharedMemorySize`, at least 16 KiB; read and written by one workgroup; lasts while the workgroup runs.
Private	ordinary local variables	Registers, private to one invocation; large arrays may spill to slower memory.

Table 2.2.1. The memory a compute shader can use

Qualifiers on buffer blocks tell the compiler how a buffer is used. `readonly` and `writeonly` promise that the shader only reads or only writes the buffer, and let the compiler and the driver choose faster paths. `restrict` promises that no other binding refers to the same memory, so the compiler may keep values in registers and reorder accesses; all of this unit's kernels use it. `coherent` makes writes visible to other invocations of the same dispatch without waiting for the dispatch to end; it is needed only when invocations communicate through a buffer while they run, as in single-pass algorithms such as the decoupled look-back scan that Chapter 2.3 mentions.

Access patterns: the transpose

Part 3 of the Handbook explained *coalescing*: when the invocations of a subgroup access consecutive addresses, the hardware serves them with a few wide memory transactions; when they access scattered addresses, each costs a transaction of its own. A matrix transpose shows the difference clearly. Reading a row is coalesced, and writing it out as a column is not (Figure 2.2.1).

The naive kernel assigns a 32 × 32 tile to each workgroup of 32 × 8 invocations, each handling four rows of the tile. Consecutive invocations read consecutive elements of a row and write elements a whole row of the destination apart:

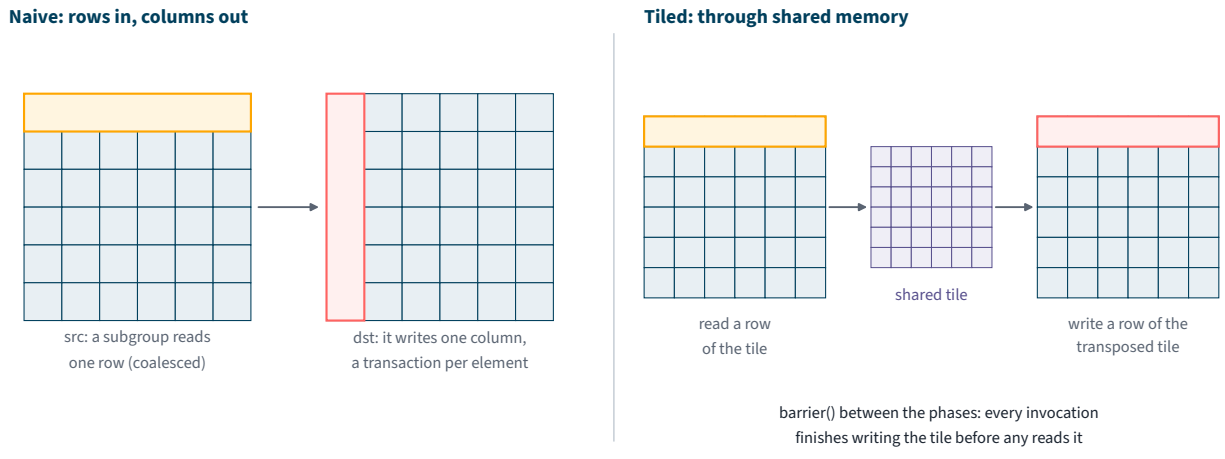


Figure 2.2.1. Two ways to transpose. The naive kernel reads rows and writes columns. The tiled kernel reads a 32 × 32 tile in rows, stores it in shared memory, and writes the transposed tile in rows too.

The naive transpose: coalesced reads, strided writes — code/u2/transpose/naive.comp

```
void main() {
    uvec2 origin = gl_WorkGroupID.xy * TILE;
    uint x = origin.x + gl_LocalInvocationID.x;
    for (uint row = gl_LocalInvocationID.y; row < TILE; row += gl_WorkGroupSize.y) {
        uint y = origin.y + row;
        dst[x * push.n + y] = src[y * push.n + x];
    }
}
```

The tiled kernel stages the tile through *shared memory*, a small on-chip memory that the invocations of a workgroup share and that is about as fast as the L1 cache. It reads the tile in rows, waits until the whole workgroup has done so, and writes the transposed tile in rows, so that both the reads and the writes of global memory are coalesced:

The tiled transpose, with optional padding chosen by a specialisation constant — code/u2/transpose/tiled.comp

```
layout(constant_id = 0) const uint PAD = 0;
shared float tile[TILE][TILE + PAD];

void main() {
    uvec2 origin = gl_WorkGroupID.xy * TILE;
    uvec2 id = gl_LocalInvocationID.xy;
    for (uint row = id.y; row < TILE; row += gl_WorkGroupSize.y) {
        tile[row][id.x] = src[(origin.y + row) * push.n + origin.x + id.x];
    }
    barrier();
    for (uint row = id.y; row < TILE; row += gl_WorkGroupSize.y) {
        dst[(origin.x + row) * push.n + origin.y + id.x] = tile[id.x][row];
    }
}
```

The program runs four kernels, all specialisations of three shaders, and checks every result bit for bit: a plain copy, which is the bandwidth any transpose can at best match, the naive transpose, the tiled transpose, and the tiled transpose with padded rows.

The four variants — code/u2/transpose/main.cpp

```
struct Variant {
    const char* label;
    const char* shader;
    uint32_t pad;
    bool transposes;
};

const Variant variants[] = {
    {"copy (the bandwidth reference)", "copy.comp", 0, false},
    {"naive transpose", "naive.comp", 0, true},
    {"tiled in shared memory", "tiled.comp", 0, true},
    {"tiled, rows padded to 33 floats", "tiled.comp", 1, true},
};
```

A 4096 × 4096 transpose on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_transpose
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
4096 x 4096 floats (64 MiB); median of 10 runs after a 25 ms warm-up

kernel                                ms     GB/s   result
copy (the bandwidth reference)       0.804    167.0   exact
naive transpose                      0.810    165.7   exact
tiled in shared memory               0.788    170.4   exact
tiled, rows padded to 33 floats      0.787    170.5   exact
PASS transpose: 4 of 4 results exact
```

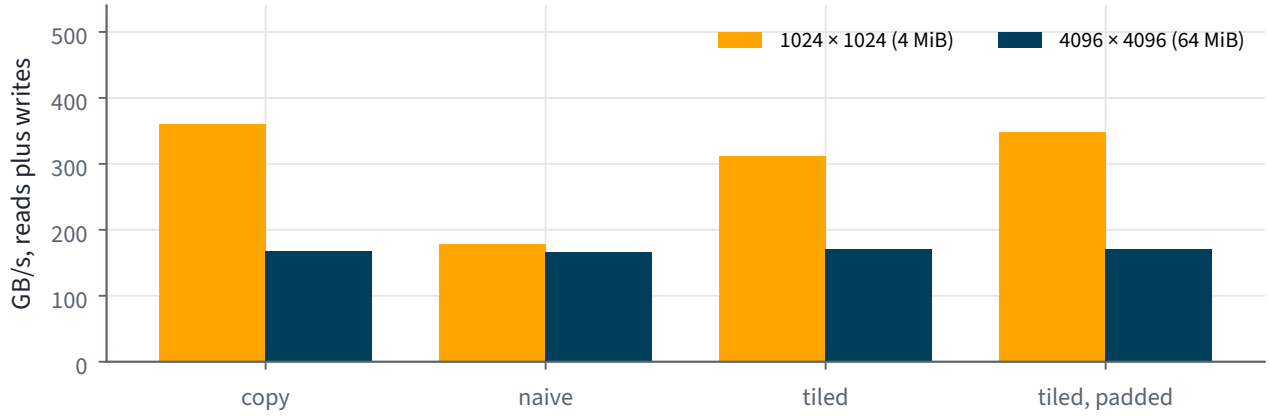


Figure 2.2.2. The transposes on the M2 Pro. With a small matrix, tiling recovers most of what the naive kernel loses; with a large one, every kernel is limited by memory bandwidth.

The same with a 1024 × 1024 matrix, which fits in the M2 Pro's caches

```
$ VKF_VALIDATION=0 ./build/bin/u2_transpose --n 1024 --runs 50
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
1024 x 1024 floats (4 MiB); median of 50 runs after a 25 ms warm-up

kernel                                ms     GB/s   result
copy (the bandwidth reference)     0.023    360.8   exact
naive transpose                    0.047    177.5   exact
tiled in shared memory             0.027    311.2   exact
tiled, rows padded to 33 floats    0.024    348.6   exact
PASS transpose: 4 of 4 results exact
```

The results differ from the textbook's. With the 64 MiB matrix, all three transposes run within about 2% of the copy. The M2 Pro's caches gather the scattered writes of each tile before they reach memory, so the naive kernel hardly suffers, and memory bandwidth limits all of them. With the 4 MiB matrix, the data stays in the GPU's caches, the copy reaches about 360 GB/s, and the naive kernel falls to less than half of it, while the tiled kernels recover most of the difference. On many discrete GPUs, the naive kernel is slow at every size, because their caches do not absorb strided writes as well. Measure on each GPU, because an optimisation that matters a great deal on one device may matter little on another.

Shared memory banks

Shared memory is divided into *banks*, typically 32 of them, each four bytes wide, with consecutive words in consecutive banks. A subgroup's accesses to different banks proceed together; accesses to different addresses in the same bank are serialised, a *bank conflict*. In a 32 × 32 tile of floats every row is 32 words long, so a whole column lies in a single bank, and the tiled transpose's column reads conflict 32 ways. Padding each row by one float shifts each row by one bank and spreads a column over all of them (Figure 2.2.3).

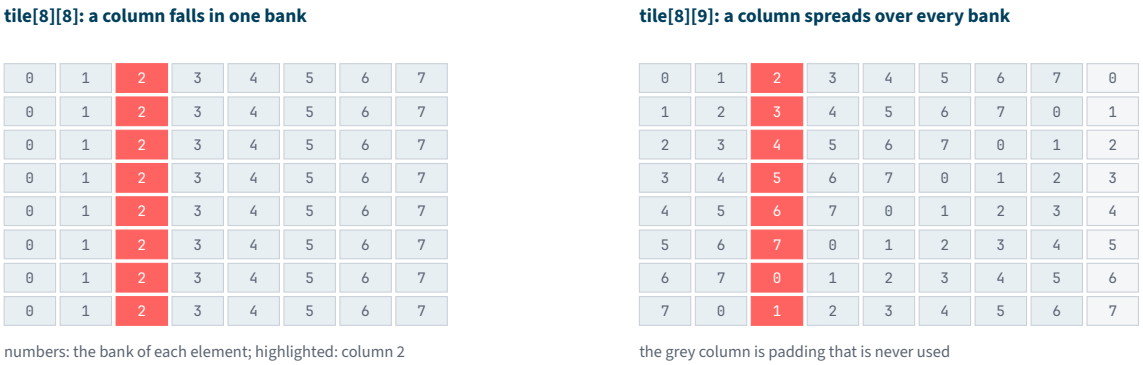


Figure 2.2.3. Bank conflicts, drawn with eight banks for legibility. Without padding, a column of the tile lies in one bank; with one word of padding per row, it spreads across every bank.

The padded kernel is the fastest of the three transposes with the small matrix. With the large one it gains nothing, because memory bandwidth limits it. Apple does not document how its GPUs organise shared memory, which is one more reason to measure instead of reasoning from another vendor's design.

`barrier()` and memory barriers

The tiled transpose would be wrong without its `barrier()`: an invocation would read elements of the tile that other invocations had not yet written. `barrier()` does two things. It is an *execution barrier*, which no invocation of the workgroup passes until all have reached it, and it is a *memory barrier* for shared memory, which makes every write to shared memory before it visible to every read after it. Together these let the invocations of a workgroup hand data to each other through shared memory.

The memory barrier functions `memoryBarrierShared()`, `memoryBarrierBuffer()`, `memoryBarrierImage()` and `groupMemoryBarrier()` order an invocation's own memory accesses as other invocations observe them, without waiting for other invocations. They matter for communication through buffers and images, combined with `barrier()` within a workgroup or with `coherent` memory across workgroups, which the course needs only rarely.

> **PITFALL**
> Every invocation of a workgroup must reach the same `barrier()`. Calling it inside a branch that some invocations skip, or after an early `return` taken by some, is undefined behaviour: on some GPUs the workgroup waits forever and the device is lost. Keep barriers in code that the whole workgroup executes, and turn bounds checks into conditions around the work instead of early returns.

Atomic operations: the histogram

An *atomic operation* reads, modifies and writes one memory location as a single indivisible step, so that concurrent updates from many invocations are all applied. GLSL provides `atomicAdd`, `atomicMin`, `atomicMax`, `atomicAnd`, `atomicOr`, `atomicXor`, `atomicExchange` and `atomicCompSwap` on 32-bit integers in buffers and in shared memory; each returns the value the location held before. 64-bit and floating-point atomics are optional features that a program must check for.

A histogram counts how many input values fall in each bin, and is a natural use of atomics. The straightforward kernel adds every byte of its input to one of 256 bins in a storage buffer:

A histogram with an atomic add to global memory for every byte — code/u2/histogram/global.comp

```
void main() {
    uint stride = gl_NumWorkGroups.x * gl_WorkGroupSize.x;
    for (uint i = gl_GlobalInvocationID.x; i < push.words; i += stride) {
        uint word = data[i];
        for (int byte = 0; byte < 4; ++byte) {
            atomicAdd(bins[bitfieldExtract(word, 8 * byte, 8)], 1u);
        }
    }
}
```

On most GPUs, atomics on global memory are performed in the L2 cache, and atomics on the same address are serialised there. The usual remedy is *privatisation*: each workgroup counts into its own bins in shared memory, where atomics are cheap, and adds its bins to the global ones once at the end.

Counting in shared memory first: one global atomic per bin per workgroup — code/u2/histogram/shared.comp

```
shared uint counts[BINS];

void main() {
    for (uint b = gl_LocalInvocationIndex; b < BINS; b += gl_WorkGroupSize.x) {
        counts[b] = 0;
    }
    barrier();

    uint stride = gl_NumWorkGroups.x * gl_WorkGroupSize.x;
    for (uint i = gl_GlobalInvocationID.x; i < push.words; i += stride) {
        uint word = data[i];
        for (int byte = 0; byte < 4; ++byte) {
            atomicAdd(counts[bitfieldExtract(word, 8 * byte, 8)], 1u);
        }
    }
    barrier();

    for (uint b = gl_LocalInvocationIndex; b < BINS; b += gl_WorkGroupSize.x) {
        if (counts[b] != 0) atomicAdd(bins[b], counts[b]);
    }
}
```

The program checks both against a histogram computed on the CPU, for two inputs: random bytes, which spread evenly over the bins, and an input in which every byte is the same, so that every atomic lands on one bin.

The CPU's histogram, the reference for every result — code/u2/histogram/main.cpp

```
std::array<uint32_t, binCount> histogram(std::span<const uint32_t> words) {
    std::array<uint32_t, binCount> counts{};
    for (uint32_t word : words) {
        for (int byte = 0; byte < 4; ++byte) ++counts[(word >> (8 * byte)) & 0xFF];
    }
    return counts;
}
```

The histograms on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_histogram
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
64 MiB of bytes, 256 bins, 4096 workgroups; median of 10 runs

input          kernel                       ms     GB/s   bins
uniform        global atomics            5.012     13.4   exact
uniform        shared, then global       0.374    179.6   exact
all identical  global atomics           57.835      1.2   exact
all identical  shared, then global       1.588     42.3   exact
PASS histogram: 4 of 4 histograms exact
```

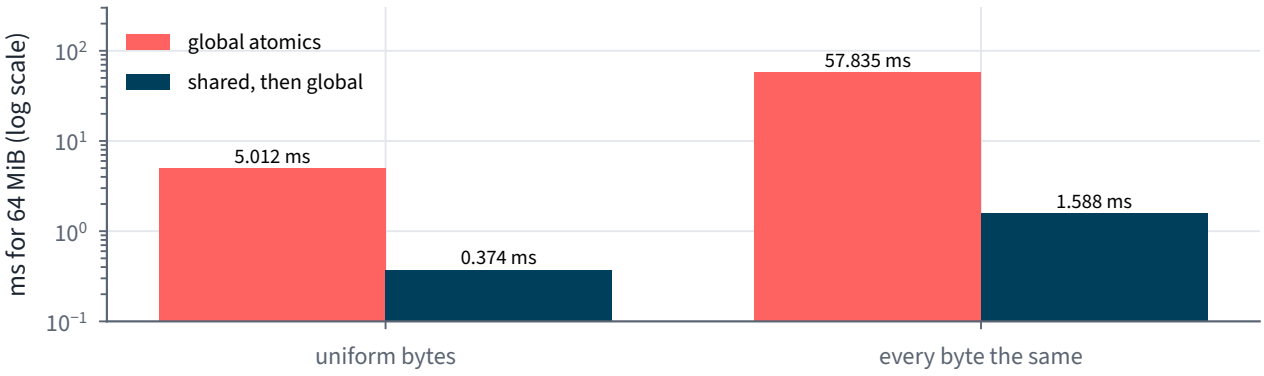


Figure 2.2.4. Time to build the histogram of 64 MiB. Privatisation is about thirteen times faster on random input, and more than thirty times faster when every update hits one bin.

Privatisation turns 64 million global atomics into about a million: 4096 workgroups adding 256 bins each. On random input it runs at the speed of memory; with every byte the same, the shared-memory atomics contend too, but cost far less than the global ones.

Data races

A *data race* occurs when two invocations access the same location without synchronisation and at least one of them writes. Its result is undefined, which in practice means that it depends on timing. The third program commits two races on purpose. In the first, a million invocations each add one to a counter with an ordinary read, add and write; the fix, chosen by a Boolean specialisation constant, is `atomicAdd`.

A counter updated without and with an atomic operation — code/u2/race/counter.comp

```
layout(constant_id = 0) const bool ATOMIC = false;

layout(std430, set = 0, binding = 0) buffer Counter {
    uint counter;
};

void main() {
    if (ATOMIC) {
        atomicAdd(counter, 1u);
    } else {
        counter += 1u;
    }
}
```

A Boolean specialisation constant passed as a `VkBool32` — code/u2/race/main.cpp

```
auto setLayout = u2::storageBufferLayout(ctx, 1);
auto layout = vkf::createPipelineLayout(ctx, {setLayout.get()});
vkf::Specialization spec;
spec.set(0, VkBool32(fix ? VK_TRUE : VK_FALSE));
auto pipeline = u2::loadPipeline(ctx, layout, "counter.comp", spec);
```

In the second, each workgroup sums 256 values in shared memory with a tree: half the invocations add the upper half of the array to the lower, then a quarter, and so on. Without the barriers, an invocation can read a partial sum before another invocation has written it.

A tree sum in shared memory, with its barriers made optional — code/u2/race/tree_sum.comp

```
shared uint partial[gl_WorkGroupSize.x];

void main() {
    uint t = gl_LocalInvocationIndex;
    partial[t] = values[gl_GlobalInvocationID.x];
    if (USE_BARRIER) barrier();
    for (uint stride = gl_WorkGroupSize.x / 2; stride > 0; stride /= 2) {
        if (t < stride) partial[t] += partial[t + stride];
        if (USE_BARRIER) barrier();
    }
    if (t == 0) sums[gl_WorkGroupID.x] = partial[0];
}
```

Both races on the M2 Pro

```
$ ./build/bin/u2_race
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
race 1: 1048576 invocations add 1 to one counter with a plain read-modify-write
  expected 1048576, observed 1 (100.0% of the additions lost)
race 2: 4096 workgroups each sum 256 values in shared memory without barrier()
  wrong sums in 86 of 4096 workgroups (2.1%)
A race depends on timing: how often it shows varies between GPUs, drivers and runs.
PASS race: 2 of 2 races showed; --fix synchronises the kernels
```

The same kernels, synchronised

```
$ ./build/bin/u2_race --fix
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
race 1: 1048576 invocations add 1 to one counter with atomicAdd
  expected 1048576, observed 1048576 (0.0% of the additions lost)
race 2: 4096 workgroups each sum 256 values in shared memory with barrier()
  wrong sums in 0 of 4096 workgroups (0.0%)
PASS race --fix: the synchronised kernels give exact results
```

The counter's result is the most extreme possible. A million invocations each added one, and the counter ended at one. At least the last invocation to write read the counter's initial zero, so every earlier increment was lost. The missing barriers corrupted about one sum in fifty in this run, a rate that changes from run to run. On another GPU, or with another load, either number could be different. This variability makes races dangerous, because a race that happens rarely passes tests and fails in production. Synchronisation validation checks the barriers between commands and does not track the accesses inside a dispatch, so it cannot detect these races; only careful design and testing can.

> **KEY IDEA**
> Within a workgroup, `barrier()` orders execution and shared memory. Within a dispatch, between workgroups, only atomic operations are safe on shared locations. Between dispatches, pipeline barriers order everything.

The Vulkan memory model

Vulkan 1.2 adopted a formal *memory model*, the `vulkanMemoryModel` feature, that defines precisely when a write by one invocation becomes visible to another. It describes writes becoming *available* and then *visible*, at a *scope*: the subgroup, the workgroup, the queue family or the device. GLSL exposes it through the `GL_KHR_memory_scope_semantics` extension, which adds atomic loads and stores with explicit scopes and acquire and release semantics. Most compute code never needs it explicitly, because `barrier()`, atomics and pipeline barriers already provide the guarantees. Unit 3 uses the same ideas of availability and visibility for the barriers between commands.

> **COMING FROM OPENGL**
> The shader side of this chapter is identical in OpenGL 4.3 and later: shared variables, `barrier()`, the memory barrier functions, atomics and the `coherent`, `restrict`, `readonly` and `writeonly` qualifiers all come from GLSL. Only the host side differs: OpenGL binds storage buffers to numbered binding points and orders dispatches with `glMemoryBarrier`, where Vulkan uses descriptor sets and pipeline barriers.

Reading

- The GLSL 4.60 specification, sections on shared variables, "Shader Invocation Control Functions" (`barrier`), "Shader Memory Control Functions" and "Atomic Memory Functions".
- The Vulkan Guide, "Shader Memory Layout" and "Atomics".
- Hwu, Kirk and El Hajj, *Programming Massively Parallel Processors*, chapters 5 (memory and tiling), 6 (performance, including coalescing) and 9 (parallel histogram).

Labs

LAB 2.2.1 Transpose at four sizes — 1.5 hours

Goal. Find where tiling matters on your GPU.

Code. `code/u2/transpose`

1. Run `VKF_VALIDATION=0 build/bin/u2_transpose` with `--n` set to 512, 1024, 2048 and 4096, using `--runs 50` for the two smallest.
2. Plot the bandwidth of each kernel against the matrix size.
3. Find your GPU's L2 cache size, and mark it on the plot at the matrix size whose source and destination together fill it.

DONE WHEN

- [ ] You have the plot from three runs at each size.
- [ ] You can explain where the naive kernel falls behind the tiled ones on your GPU, in terms of its caches.

EVIDENCE FOR THE NOTEBOOK

- The plot and your explanation.

LAB 2.2.2 Read your bank conflicts — 1 hour

Goal. Measure the cost of bank conflicts directly.

Code. `code/u2/transpose`

1. Add a variant of `tiled.comp` that writes `tile[id.x][row]` in the first phase and reads `tile[row][id.x]` in the second: the same transpose with the conflicts moved to the writes.
2. Measure all the tiled variants with padding of 0, 1 and 2 at `--n 1024 --runs 50`.

DONE WHEN

- [ ] Every variant still transposes exactly.
- [ ] You have the timings, and can say whether your GPU shows bank conflicts and how large they are.

EVIDENCE FOR THE NOTEBOOK

- The variant's code and the table of timings.

LAB 2.2.3 Histograms under contention — 1.5 hours

Goal. Measure how contention and privatisation interact.

Code. `code/u2/histogram`

1. Add an input in which every byte is one of four values, and run all three inputs with both kernels.
2. Change `wordsPerInvocation`, which sets how many workgroups the dispatch uses, to 4 and to 64, and measure again.
3. Explain the effect of the number of workgroups on each kernel.

DONE WHEN

- [ ] Every histogram is exact.
- [ ] You have a table of the six measurements at three numbers of workgroups, and your explanation refers to where the atomics are performed.

EVIDENCE FOR THE NOTEBOOK

- The table and the explanation.

LAB 2.2.4 Races, observed and fixed — 1 hour

Goal. See how a race's symptoms vary.

Code. `code/u2/race`

1. Run `build/bin/u2_race` ten times and record both failure rates each time.
2. In `tree_sum.comp`, keep the barrier before the loop and remove only the one inside it, then the reverse, and record the failure rate of each.
3. Run `--fix` and confirm that both kernels are exact.

DONE WHEN

- [ ] You have the failure rates for all three versions of the tree, over ten runs each.
- [ ] You can explain which data each missing barrier lets an invocation read too early.

EVIDENCE FOR THE NOTEBOOK

- The rates and the explanation.

Problem set

1. How many bytes does a transpose of an N × N matrix of floats move, and what is its arithmetic intensity? Why can no transpose run faster than a copy of the same matrix?
2. Thirty-two invocations read `tile[i][5]` from `shared float tile[32][32]` for i = 0 to 31. Which banks do they touch, assuming 32 banks of four bytes? Which with `tile[32][33]`?
3. A histogram has 65,536 bins of 32-bit counts. Why can privatisation in shared memory not work as written here, and what would you do instead?
4. The racing counter ended at exactly one on the M2 Pro. Describe an execution in which it ends at 256 instead.
5. Give an example of a kernel that needs the `coherent` qualifier on a buffer, and explain what could go wrong without it.

COMPLETION CHECKLIST

- [ ] I can choose the right kind of memory for each piece of shader data and use the buffer qualifiers.
- [ ] I can explain coalescing and use a tile in shared memory, with its barriers, to make accesses coalesced.
- [ ] I can recognise bank conflicts and remove them with padding.
- [ ] I can use atomics, privatise contended updates, and recognise and fix data races.
- [ ] Lab 2.2.1–2.2.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.3

Parallel patterns

TIME	7 hours
YOU BUILD	A reduction of 16 million floats in five variants, measured against memory bandwidth; three prefix sums, from a single workgroup to a multi-level scan of four million values; stream compaction built on the scan; and a stable radix sort of four million key-value pairs, checked against `std::stable_sort`.
NEEDS	Chapters 2.1 and 2.2.

WHY IT MATTERS

A small number of patterns account for most of what GPUs compute beyond simple element-wise work: reduction, which combines many values into one; scan, which gives every element the combination of those before it; compaction, which keeps the elements that pass a test; and sorting. Each looks sequential at first sight, and each has a parallel form whose structure, a tree of partial results computed level by level, recurs throughout GPU programming. This chapter builds all four, verifies each against the CPU, and measures them against the bandwidth limits of Chapter 2.1.

SKIP TEST

1. Why is a reduction in shared memory with `if (t < stride)` faster than one with `if (t % (2 * stride) == 0)`?
2. What are the work and the span of the Hillis–Steele scan and of the Blelloch scan for n values?
3. How does a scan of four million values that each workgroup can only scan 2048 of at a time produce a correct result?
4. How does an exclusive scan turn a list of keep-or-drop flags into the output positions of stream compaction?
5. Why must each pass of an LSD radix sort be stable?

If all five are easy, read the radix sort section and do Labs 2.3.1 and 2.3.4.

Core ideas

Work and span

Two numbers describe a parallel algorithm. Its *work* is the total number of operations it performs, and its *span*, or depth, is the length of its longest chain of operations that must happen one after another. Adding n numbers in a loop has work n and span n. Adding them in a tree, pairs first, then pairs of pairs, has the same work but a span of only $\log_2 n$, which is what lets thousands of invocations share it. An algorithm is *work-efficient* if its work is no larger, asymptotically, than the best sequential algorithm's; a parallel algorithm that does more work can still win with enough hardware, but it spends energy and bandwidth to do so.

Reduction

A reduction combines n values with an associative operation, here addition. Each workgroup loads its share of the input into shared memory and sums it in a tree, halving the number of active invocations at each step, with a barrier between steps; invocation 0 then writes the workgroup's sum. Which invocations stay active matters (Figure 2.3.1).

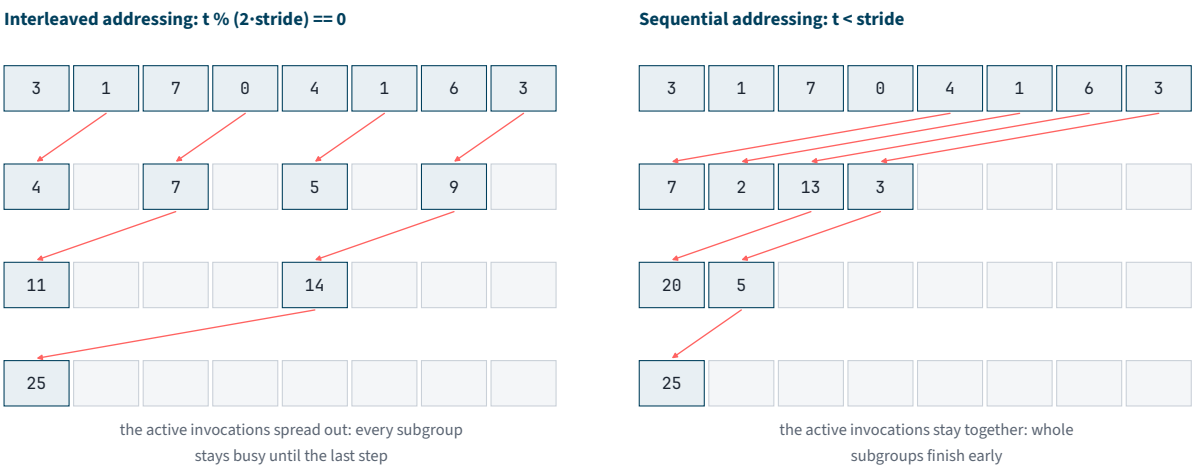


Figure 2.3.1. Two ways to address a tree reduction, shown for eight values. Both perform the same additions; they differ in which invocations do them.

With *interleaved addressing*, the invocations whose index is a multiple of twice the stride add their neighbours. The active invocations become sparser at every step but remain spread across every subgroup, so every subgroup keeps executing until the last step, mostly with idle lanes:

A tree with interleaved addressing — code/u2/reduce/interleaved.comp

```
void main() {
    uint t = gl_LocalInvocationIndex;
    uint i = gl_GlobalInvocationID.x;
    partial[t] = i < push.count ? src[i] : 0.0;
    barrier();
    for (uint stride = 1; stride < gl_WorkGroupSize.x; stride *= 2) {
        if (t % (2 * stride) == 0) partial[t] += partial[t + stride];
        barrier();
    }
    if (t == 0) dst[gl_WorkGroupID.x] = partial[0];
}
```

With *sequential addressing*, the first half adds the second half, then the first quarter adds the second quarter, and so on. The active invocations stay contiguous, so whole subgroups fall idle and stop at once:

A tree with sequential addressing — code/u2/reduce/sequential.comp

```
void main() {
    uint t = gl_LocalInvocationIndex;
    uint i = gl_GlobalInvocationID.x;
    partial[t] = i < push.count ? src[i] : 0.0;
    barrier();
    for (uint stride = gl_WorkGroupSize.x / 2; stride > 0; stride /= 2) {
        if (t < stride) partial[t] += partial[t + stride];
        barrier();
    }
    if (t == 0) dst[gl_WorkGroupID.x] = partial[0];
}
```

Each workgroup produces one partial sum, so the program reduces the partial sums again in a second pass, and a third if needed, alternating between two buffers until one value remains:

Planning the passes down to a single value — code/u2/shared/reduction.hpp

```
struct ReducePass {
    VkDescriptorSet set;
    uint32_t count;
    uint32_t groups;
};

inline std::vector<ReducePass> planReduction(uint32_t count, uint32_t perGroup,
                                             VkDescriptorSet inputToA, VkDescriptorSet aToB,
                                             VkDescriptorSet bToA) {
    const VkDescriptorSet evenOdd[] = {bToA, aToB};
    std::vector<ReducePass> passes;
    do {
        const VkDescriptorSet set = passes.empty() ? inputToA : evenOdd[passes.size() % 2];
        const uint32_t groups = vkf::groupCount(count, perGroup);
        passes.push_back({set, count, groups});
        count = groups;
    } while (count > 1);
    return passes;
}
```

Subgroups offer a shortcut. `subgroupAdd`, from `GL_KHR_shader_subgroup_arithmetic`, sums a value across a subgroup without shared memory or barriers, often in a few hardware instructions. A workgroup then needs only one tree step per subgroup: each subgroup's sum goes to shared memory, written by the single invocation that `subgroupElect` chooses, and the first subgroup sums those.

A workgroup sum built from two subgroup reductions — code/u2/reduce/subgroup.comp

```
sum = subgroupAdd(sum);
if (subgroupElect()) subgroupSums[gl_SubgroupID] = sum;
barrier();
if (gl_SubgroupID == 0) {
    sum = 0.0;
    for (uint s = gl_SubgroupInvocationID; s < gl_NumSubgroups; s += gl_SubgroupSize) {
        sum += subgroupSums[s];
    }
    sum = subgroupAdd(sum);
    if (subgroupElect()) dst[gl_WorkGroupID.x] = sum;
}
```

In the last variant, each invocation first adds several elements on its own, eight here, before the workgroup's tree starts. This *coarsening* divides the number of workgroups, the tree steps and the barriers by eight, and gives each invocation several independent loads to have in flight at once:

Each invocation sums ITEMS elements before the tree — code/u2/reduce/coarse.comp

```
uint first = gl_WorkGroupID.x * gl_WorkGroupSize.x * ITEMS + t;
float sum = 0.0;
for (uint k = 0; k < ITEMS; ++k) {
    uint i = first + k * gl_WorkGroupSize.x;
    if (i < push.count) sum += src[i];
}
partial[t] = sum;
```

The program sums 16,777,216 random floats in each variant, and compares each result with a sum computed in double precision on the CPU:

The CPU's reference sum — code/u2/reduce/main.cpp

```
std::vector<float> input(n);
std::mt19937 rng(5);
std::uniform_real_distribution<float> uniform(0.0f, 1.0f);
for (float& v : input) v = uniform(rng);
const double expected = std::accumulate(input.begin(), input.end(), 0.0);
```

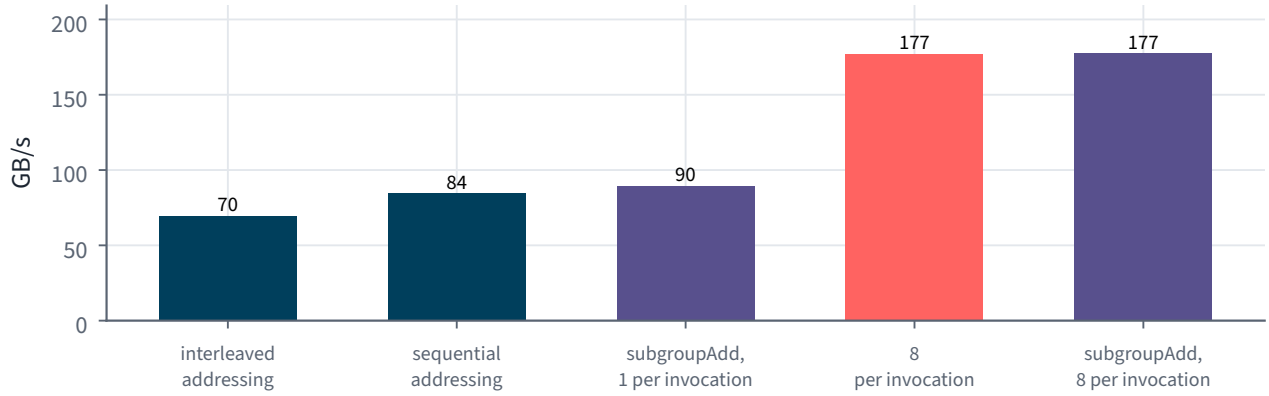


Figure 2.3.2. The reductions' bandwidth on the M2 Pro. Coarsening doubles the speed of either kind of tree; the choice of tree matters little once it does.

Five reductions of 64 MiB on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_reduce
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
sum of 16777216 floats (64 MiB), local size 512; median of 10 runs
double-precision CPU sum 8389282.968

variant                          passes       ms     GB/s   relative error
interleaved addressing                3    0.964     69.6   1.2e-07 ok
sequential addressing                 3    0.795     84.4   3.8e-09 ok
subgroupAdd, 1 per invocation         3    0.750     89.5   1.2e-07 ok
8 elements per invocation             2    0.379    177.1   3.8e-09 ok
subgroupAdd, 8 per invocation         2    0.379    177.3   3.8e-09 ok
PASS reduce: 5 of 5 sums within 1e-05 of the CPU sum
```

The three variants that load one element per invocation reach 70 to 90 GB/s. Interleaved addressing is the slowest, by a margin that changes from run to run on this GPU, and the other two are within a few per cent of each other. Coarsening doubles their speed, to about 177 GB/s, close to the SAXPY bandwidth of Chapter 2.1. The work per element left in the tree is then too small to matter, and memory limits the kernel, as it should for a reduction. On GPUs whose shared memory is slower relative to their memory bandwidth, the addressing and the subgroup operations matter more; Lab 2.3.1 measures how much on yours.

Adding floats in a different order gives a different result, so a GPU sum never matches a sequential CPU sum exactly. A tree is the more accurate order, because its rounding error grows with the logarithm of n, where a sequential loop's grows with n. The program accepts a relative error of 10^{-5} against the double-precision sum, which every variant meets with a wide margin.

Scan

A *scan*, or prefix sum, replaces each element with the sum of the elements before it. An *inclusive* scan includes the element itself; an *exclusive* scan does not and starts with zero, so that the exclusive scan of [3, 1, 7, 0, 4, 1, 6, 3] is [0, 3, 4, 11, 11, 15, 16, 22]. Scans allocate space in parallel: if each element needs some number of output slots, the exclusive scan of those numbers gives each element the position of its first slot. Compaction and the radix sort below are both built on that.

The simplest parallel scan, due to Hillis and Steele, adds to each element the element `offset` places before it, for offsets of 1, 2, 4 and so on. After $\log_2 n$ steps every element holds its inclusive prefix sum. It reads and writes the whole array at every step, so it double-buffers in shared memory:

The Hillis–Steele scan within one workgroup — code/u2/scan/hillis_steele.comp

```
shared uint buffers[2][gl_WorkGroupSize.x];

void main() {
    uint t = gl_LocalInvocationIndex;
    uint current = 0;
    buffers[current][t] = t < push.count ? inputs[t] : 0;
    barrier();
    for (uint offset = 1; offset < gl_WorkGroupSize.x; offset *= 2) {
        uint value = buffers[current][t];
        if (t ⩾ offset) value += buffers[current][t - offset];
        buffers[1 - current][t] = value;
        current = 1 - current;
        barrier();
    }
    if (t < push.count) outputs[t] = t > 0 ? buffers[current][t - 1] : 0;
}
```

Its span is $\log_2 n$, but its work is $n \log_2 n$, which makes it a poor choice for large arrays. Blelloch's scan is work-efficient. Its *up-sweep* is a reduction tree that leaves partial sums at the tree's nodes; its *down-sweep* clears the root and walks back down the tree, giving each left child its parent's value and each right child the parent's value plus the old left child (Figure 2.3.3). It performs about $2n$ additions in $2 \log_2 n$ steps.

The up-sweep: a reduction that keeps its partial sums — code/u2/scan/blelloch.comp

```
uint spacing = 1;
for (uint pairs = N / 2; pairs > 0; pairs ⊨ 2) {
    barrier();
    if (t < pairs) {
        uint left = spacing * (2 * t + 1) - 1;
        uint right = spacing * (2 * t + 2) - 1;
        tree[right] += tree[left];
    }
    spacing *= 2;
}
```

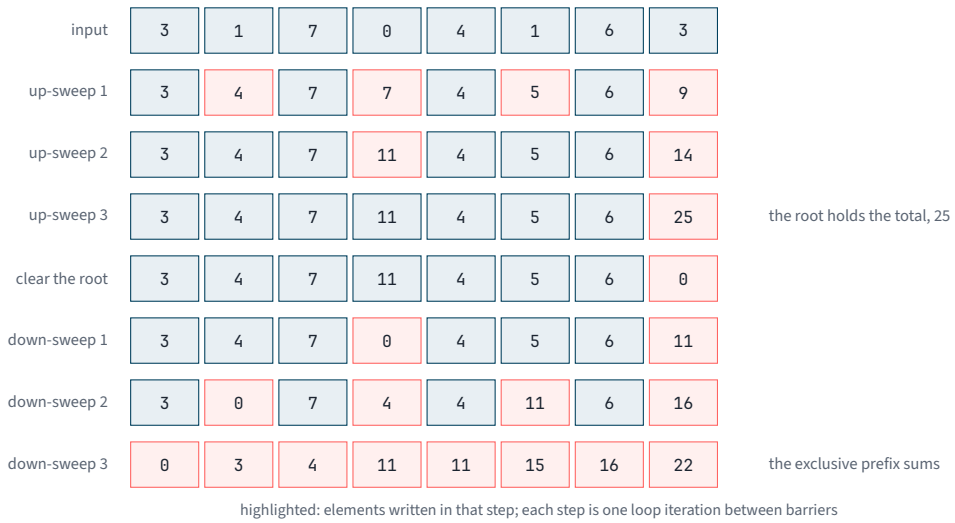


Figure 2.3.3. The Blelloch scan of eight values, in place in shared memory. Each row is one step, separated from the next by a barrier.

The down-sweep: pushing prefixes back down the tree — code/u2/scan/blelloch.comp

```
for (uint pairs = 1; pairs < N; pairs *= 2) {
    spacing /= 2;
    barrier();
    if (t < pairs) {
        uint left = spacing * (2 * t + 1) - 1;
        uint right = spacing * (2 * t + 2) - 1;
        uint leftValue = tree[left];
        tree[left] = tree[right];
        tree[right] += leftValue;
    }
}
```

The fastest block scan uses subgroups. `subgroupInclusiveAdd` scans a value across a subgroup in hardware. The last invocation of each subgroup writes the subgroup's total to shared memory, the first subgroup scans those totals, and each invocation adds its subgroup's prefix to its own result.

An exclusive scan across a workgroup, built from subgroup scans — code/u2/shared/scan.glsl

```
uint workgroupExclusiveAdd(uint value, out uint total) {
    uint inclusive = subgroupInclusiveAdd(value);
    if (gl_SubgroupInvocationID == gl_SubgroupSize - 1) {
        subgroupPrefixes[gl_SubgroupID] = inclusive;
    }
    barrier();
    if (gl_SubgroupID == 0) {
        uint carry = 0;
        for (uint first = 0; first < gl_NumSubgroups; first += gl_SubgroupSize) {
            uint s = first + gl_SubgroupInvocationID;
            uint subgroupTotal = s < gl_NumSubgroups ? subgroupPrefixes[s] : 0;
            uint prefix = carry + subgroupExclusiveAdd(subgroupTotal);
            if (s < gl_NumSubgroups) subgroupPrefixes[s] = prefix;
            carry += subgroupAdd(subgroupTotal);
        }
        if (subgroupElect()) workgroupTotal = carry;
    }
    barrier();
    total = workgroupTotal;
    uint result = subgroupPrefixes[gl_SubgroupID] + inclusive - value;
    barrier();  // the next call overwrites subgroupPrefixes
    return result;
}
```

This scan numbers invocations by subgroup, through `gl_SubgroupID` and `gl_SubgroupInvocationID`, so it needs each subgroup to be full and numbered in order. The program asks for full subgroups with `VK_PIPELINE_SHADER_STAGE_CREATE_REQUIRE_FULL_SUBGROUPS_BIT` when the device supports it, and relies on the linear numbering that Chapter 2.1 checks on the M2 Pro. Each invocation also scans four consecutive values of its own sequentially, which coarsens the scan as the reduction was coarsened:

Each invocation scans four values, and the workgroup scans the invocations' sums — code/u2/shared/scan_blocks.comp

```
void main() {
    uint first = (gl_WorkGroupID.x * gl_WorkGroupSize.x + scanPosition()) * ITEMS;
    uint values[ITEMS];
    uint sum = 0;
    [[unroll]] for (uint k = 0; k < ITEMS; ++k) {
        values[k] = first + k < push.count ? inputs[first + k] : 0;
        sum += values[k];
    }
    uint total;
    uint prefix = workgroupExclusiveAdd(sum, total);
    [[unroll]] for (uint k = 0; k < ITEMS; ++k) {
        if (first + k < push.count) outputs[first + k] = prefix;
        prefix += values[k];
    }
    if (gl_LocalInvocationIndex == 0) sums[gl_WorkGroupID.x] = total;
}
```

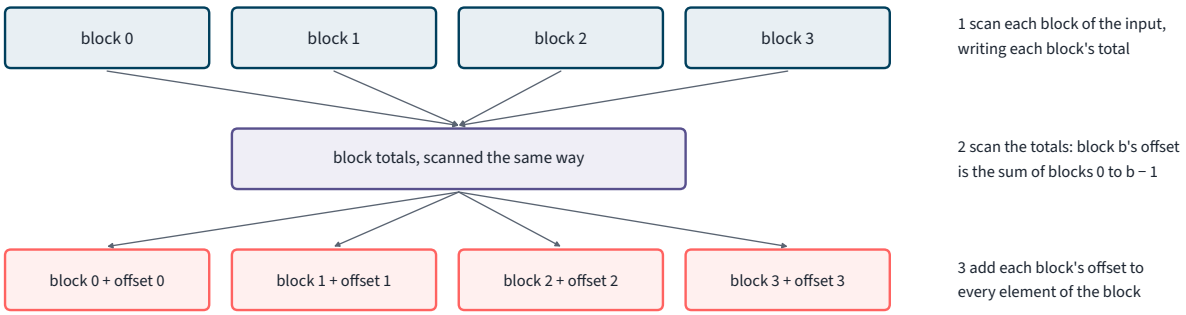


Figure 2.3.4. A multi-level scan. Blocks are scanned independently; the scan of their totals gives each block its offset; adding the offsets completes the scan.

Scanning more than one block

A workgroup scans one block of the input, 512 or 1024 values here. A larger input needs a *multi-level scan* (Figure 2.3.4). The first dispatch scans every block independently and writes each block's total. A second scans the totals, which gives each block the sum of all the blocks before it. A third adds that offset to every element of its block. When the totals themselves span several blocks, the second step is the same procedure one level up, recursively.

Building the levels: each level's totals are the next level's input — code/u2/shared/scan.hpp

```
inline MultiLevelScan::MultiLevelScan(const vkf::Context& ctx, const ScanKernels& kernels,
                                      vkf::DescriptorPool& pool, VkBuffer input,
                                      VkBuffer output, uint32_t count)
    : kernels_(&kernels) {
    const VkBufferUsageFlags usage =
        VK_BUFFER_USAGE_STORAGE_BUFFER_BIT | VK_BUFFER_USAGE_TRANSFER_SRC_BIT;
    levels_.push_back({.count = count, .input = input, .output = output});
    for (size_t i = 0;; ++i) {
        Level& level = levels_[i];
        level.blocks = vkf::groupCount(level.count, kernels.blockSize);
        level.sums = vkf::createBuffer(ctx, level.blocks * sizeof(uint32_t), usage,
                                        vkf::MemoryUse::DeviceLocal, "block sums");
        level.scanSet = pool.allocate(kernels.setLayout, "scan blocks");
        writeStorageBuffers(ctx, level.scanSet, {level.input, level.output, level.sums});
        if (level.blocks == 1) break;

        Level next{.count = level.blocks, .input = level.sums};
        next.ownedOutput = vkf::createBuffer(ctx, level.blocks * sizeof(uint32_t), usage,
                                             vkf::MemoryUse::DeviceLocal, "scanned sums");
        next.output = next.ownedOutput;
        level.addSet = pool.allocate(kernels.setLayout, "add offsets");
        writeStorageBuffers(ctx, level.addSet, {next.output, level.output});
        levels_.push_back(std::move(next));
    }
    if (levels_.front().blocks > ctx.properties().core.limits.maxComputeWorkGroupCount[0]) {
        throw std::runtime_error("the scan needs more workgroups than this device allows");
    }
}
```

Recording the scans up the levels and the additions back down, with a barrier after each dispatch — code/u2/shared/scan.hpp

```
inline void MultiLevelScan::record(VkCommandBuffer cmd) const {
    auto dispatch = [&](const Level& level, VkDescriptorSet set) {
        const ScanPush push{level.count};
        vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, kernels_->layout, 0, 1,
                                &set, 0, nullptr);
        vkCmdPushConstants(cmd, kernels_->layout, VK_SHADER_STAGE_COMPUTE_BIT, 0, sizeof(push),
                           &push);
        vkCmdDispatch(cmd, level.blocks, 1, 1);
        computeBarrier(cmd);
    };
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, kernels_->scanBlocks);
    for (const Level& level : levels_) dispatch(level, level.scanSet);
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, kernels_->addOffsets);
    for (size_t i = levels_.size() - 1; i-- > 0;) dispatch(levels_[i], levels_[i].addSet);
}
```

Adding each block's offset — code/u2/shared/scan_add.comp

```
void main() {
    uint offset = offsets[gl_WorkGroupID.x];
    uint first = gl_WorkGroupID.x * gl_WorkGroupSize.x * ITEMS + gl_LocalInvocationIndex;
    for (uint k = 0; k < ITEMS; ++k) {
        uint i = first + k * gl_WorkGroupSize.x;
        if (i < push.count) data[i] += offset;
    }
}
```

Scans on the M2 Pro: the single-workgroup Hillis–Steele scan, and two multi-level scans of four million values

```
$ VKF_VALIDATION=0 ./build/bin/u2_scan
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
Hillis-Steele scan of 1024 values in one workgroup: exact

exclusive scan of 4194304 values; median of 10 runs
block scan                      levels       ms     GB/s   result
Blelloch, blocks of 512              3    0.341     98.4   exact
subgroups, blocks of 1024            3    0.272    123.4   exact
PASS scan: 3 of 3 scans exact
```

The table's bandwidth counts one read and one write of each element, the least that any scan must do. The multi-level scan reads and writes every element twice, once in the block scan and again when the offsets are added, so it moves about twice the bytes the table counts. Counted that way, the Blelloch version runs at about 200 GB/s, the limit of the M2 Pro's memory, and the subgroup version at nearly 250 GB/s, which is possible only because part of the data is still in the GPU's caches when the second pass reads it. The single-pass *decoupled look-back* scan of Merrill and Garland is faster, because it reads and writes each element once, by having each workgroup wait for the totals of the workgroups before it. Chapter 2.1 explained why Vulkan does not guarantee that such waiting finishes, which is why this course uses the multi-level form.

Stream compaction

Stream compaction keeps the elements that satisfy a predicate, in their original order. With a scan it takes three passes: write a flag of 1 or 0 for every element, scan the flags exclusively, and write each kept element to the position its scanned flag gives. The scan's total is the number kept.

The predicate and the flags — code/u2/scan/compact_flags.comp

```
bool keep(uint value) {
    return value % 3 == 0;
}

void main() {
    uint i = gl_GlobalInvocationID.x;
    if (i < push.count) flags[i] = keep(values[i]) ? 1 : 0;
}
```

Each kept element goes to the position the scan gave it — code/u2/scan/compact_scatter.comp

```
layout(std430, set = 0, binding = 0) restrict readonly buffer Values { uint values[]; };
layout(std430, set = 0, binding = 1) restrict readonly buffer Flags { uint flags[]; };
layout(std430, set = 0, binding = 2) restrict readonly buffer Positions { uint positions[]; };
layout(std430, set = 0, binding = 3) restrict writeonly buffer Kept { uint kept[]; };

void main() {
    uint i = gl_GlobalInvocationID.x;
    if (i < push.count && flags[i] != 0) kept[positions[i]] = values[i];
}
```

Flags, scan and scatter, with the count read from the scan's total — code/u2/scan/main.cpp

```
const vkf::Stats ms = u2::timeGpu(ctx, runs, [&](VkCommandBuffer cmd) {
    dispatch(cmd, flagsPipeline, flagsPipelineLayout, flagsSet, n, groups);
    u2::computeBarrier(cmd);
    scan.record(cmd);
    dispatch(cmd, scatterPipeline, scatterPipelineLayout, scatterSet, n, groups);
});
const uint32_t count = vkf::download<uint32_t>(ctx, scan.total(), 1)[0];
```

Compaction of four million values on the M2 Pro

```
$ VKF_VALIDATION=0 ./build/bin/u2_scan --compact
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
stream compaction of 4194304 values, keeping multiples of 3
flags, a 3-level scan and the scatter: 0.743 ms
kept 1398840 of 4194304 values (33.4%): exact
PASS scan --compact: the kept values match std::copy_if
```

Radix sort

A *least-significant-digit radix sort* sorts integer keys by one digit at a time, starting with the least significant, with a *stable* sort for each digit: one that keeps keys with equal digits in their existing order. The method depends on that stability. After the pass for digit k, keys are ordered by their lowest k + 1 digits, because ties in digit k were left in the order the earlier passes produced. Thirty-two-bit keys with four-bit digits take eight passes. The sort carries a value with each key, and the values test stability, because equal keys must keep their values in the input's order.

Each pass has three steps (Figure 2.3.5). The *count* step has each workgroup count how many keys of its block have each digit. The counts are stored digit by digit, every block's count of digit 0 first, so that an exclusive scan of the whole array, the multi-level scan above, gives each block the first output position for each of its digits. The *scatter* step then writes every key to its digit's position for its block, plus the key's rank among the block's keys with the same digit.

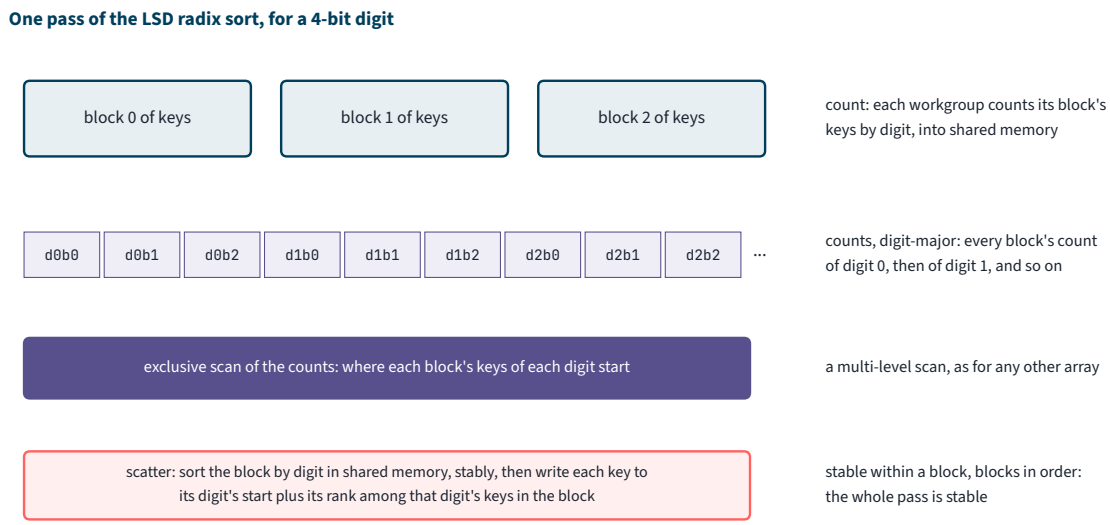


Figure 2.3.5. One pass of the radix sort. Counting and scanning give each block a starting position for each digit; the scatter writes each block's keys there in a stable order.

Counting each block's digits in shared memory — code/u2/sort/sort_count.comp

```
void main() {
    for (uint d = gl_LocalInvocationIndex; d < BUCKETS; d += gl_WorkGroupSize.x) {
        histogram[d] = 0;
    }
    barrier();
    uint first = gl_WorkGroupID.x * gl_WorkGroupSize.x * ITEMS + gl_LocalInvocationIndex;
    for (uint k = 0; k < ITEMS; ++k) {
        uint i = first + k * gl_WorkGroupSize.x;
        if (i < push.count) atomicAdd(histogram[digitOf(keys[i])], 1u);
    }
    barrier();
    for (uint d = gl_LocalInvocationIndex; d < BUCKETS; d += gl_WorkGroupSize.x) {
        counts[d * push.blocks + gl_WorkGroupID.x] = histogram[d];
    }
}
```

The scatter must rank keys stably within the block. It sorts the block in shared memory one bit of the digit at a time with a *split*: an exclusive scan of the bits tells each key how many ones precede it, so a key whose bit is 1 goes after all the zeros in that order, and a key whose bit is 0 goes to its position minus the ones before it. Each split is stable, so four of them sort the block stably by the four-bit digit.

Four stable splits sort the block by the current digit — code/u2/sort/sort_scatter.comp

```
for (uint bit = 0; bit < DIGIT_BITS; ++bit) {
    uint flag[ITEMS];
    uint ones = 0;
    [[unroll]] for (uint k = 0; k < ITEMS; ++k) {
        flag[k] = bitfieldExtract(key[k], int(push.shift + bit), 1);
        ones += flag[k];
    }
    uint totalOnes;
    uint onesBefore = workgroupExclusiveAdd(ones, totalOnes);
    uint zeros = BLOCK - totalOnes;
    [[unroll]] for (uint k = 0; k < ITEMS; ++k) {
        uint slot = position + k;
        uint target = flag[k] == 1 ? zeros + onesBefore : slot - onesBefore;
        onesBefore += flag[k];
        sortedKeys[target] = key[k];
        sortedValues[target] = value[k];
    }
    barrier();
    [[unroll]] for (uint k = 0; k < ITEMS; ++k) {
        key[k] = sortedKeys[position + k];
        value[k] = sortedValues[position + k];
    }
    barrier();
}
```

Writing each key to its global position — code/u2/sort/sort_scatter.comp

```
if (gl_LocalInvocationIndex == 0) {
    uint sum = 0;
    for (uint d = 0; d < BUCKETS; ++d) {
        digitStart[d] = sum;
        sum += counts[d * push.blocks + gl_WorkGroupID.x];
    }
}
barrier();
for (uint k = 0; k < ITEMS; ++k) {
    uint slot = k * gl_WorkGroupSize.x + gl_LocalInvocationIndex;
    if (blockStart + slot ⩾ push.count) continue;
    uint sortedKey = sortedKeys[slot];
    uint digit = digitOf(sortedKey);
    uint blockOffset = offsets[digit * push.blocks + gl_WorkGroupID.x];
    uint target = blockOffset + slot - digitStart[digit];
    keysOut[target] = sortedKey;
    valuesOut[target] = sortedValues[slot];
}
```

The passes alternate between two pairs of buffers, and each pass's three steps are separated by barriers:

Eight passes of count, scan and scatter — code/u2/sort/main.cpp

```
const uint32_t passes = 32 / digitBits;
auto sort = [&](VkCommandBuffer cmd) {
    for (uint32_t pass = 0; pass < passes; ++pass) {
        const int s = pass == 0 ? 0 : pass % 2 == 1 ? 1 : 2;
        const Push push{n, pass * digitBits, blocks};
        dispatch(cmd, countPipeline, countLayout, countSets[s], push, blocks);
        u2::computeBarrier(cmd);
        scan.record(cmd);
        dispatch(cmd, scatterPipeline, scatterLayout, scatterSets[s], push, blocks);
        u2::computeBarrier(cmd);
    }
};
```

The result must match `std::stable_sort` exactly, keys and values — code/u2/sort/main.cpp

```
std::vector<Pair> pairs(n);
for (uint32_t i = 0; i < n; ++i) pairs[i] = {keys[i], values[i]};
vkf::CpuTimer stableTimer;
std::stable_sort(pairs.begin(), pairs.end(),
                 [](const Pair& x, const Pair& y) { return x.key < y.key; });
const double stableMs = stableTimer.elapsedMs();
bool exact = true;
for (uint32_t i = 0; i < n; ++i) {
    exact = exact && gpuKeys[i] == pairs[i].key && gpuValues[i] == pairs[i].value;
}
```

Sorting four million pairs on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_sort
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
LSD radix sort of 4194304 key-value pairs: 4-bit digits, 8 passes
2380586 keys repeat an earlier key, so their values test stability

GPU radix sort (median)              3.802 ms   1103.3 M pairs/s
std::sort, keys only                83.805 ms     50.0 M keys/s
std::stable_sort, pairs            206.325 ms     20.3 M pairs/s
PASS sort: the GPU pairs match std::stable_sort exactly
```

The GPU sorts more than a billion pairs per second, about fifty times faster than `std::stable_sort` on one CPU core. A parallel sort on all the CPU's cores would narrow the gap without closing it, because each pass of the radix sort reads and writes every pair a few times, at memory bandwidth, and does almost nothing else.

> **KEY IDEA**
> Reduction, scan and the scatter that follows a scan are the building blocks of most parallel algorithms on the GPU. When a problem seems to need a sequential loop, ask whether a scan can compute what each element needs to know about the elements before it.

> **COMING FROM OPENGL**
> These algorithms are the same in OpenGL 4.3 compute shaders, with `glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT)` between passes where Vulkan records a pipeline barrier. Subgroup operations require the `GL_KHR_shader_subgroup` extension, which most desktop drivers offer and macOS does not. In CUDA, the CUB and Thrust libraries provide all four, and the CUDA literature in the reading list explains them with the same diagrams.

Reading

- Mark Harris, "Optimizing Parallel Reduction in CUDA", NVIDIA, 2007: seven versions of the reduction, from interleaved addressing to unrolled, coarsened kernels.
- Mark Harris, Shubhabrata Sengupta and John D. Owens, "Parallel Prefix Sum (Scan) with CUDA", *GPU Gems 3*, chapter 39.
- Duane Merrill and Michael Garland, "Single-pass Parallel Prefix Scan with Decoupled Look-back", NVIDIA, 2016, with this chapter's caveat about forward progress in mind.
- Hwu, Kirk and El Hajj, *Programming Massively Parallel Processors*, chapters 10 (reduction), 11 (scan) and 13 (sorting).
- The Vulkan Guide, "Subgroups".

Labs

LAB 2.3.1 Reduce at the bandwidth limit — 2 hours

Goal. Find the reduction's best configuration on your GPU.

Code. `code/u2/reduce`

1. Run `VKF_VALIDATION=0 build/bin/u2_reduce` and record the table.
2. Add variants with 2, 4, 16 and 32 elements per invocation, and run each with local sizes of 128, 256 and 512.
3. Compare the best result with your GPU's SAXPY and copy bandwidths from Chapters 2.1 and 2.2.

DONE WHEN

- [ ] Every variant's sum is within the tolerance.
- [ ] Your best variant reaches at least 90% of your GPU's copy bandwidth, or you can explain what prevents it.

EVIDENCE FOR THE NOTEBOOK

- The table of variants and the comparison.

LAB 2.3.2 A scan by hand, then in code — 1.5 hours

Goal. Understand the Blelloch scan's index arithmetic exactly.

Code. `code/u2/scan`

1. Run the Blelloch scan on paper for the sixteen values 1 to 16, writing the tree after every step.
2. Add an inclusive variant of the block scan, as a specialisation constant, and check it against `std::inclusive_scan`.
3. Run the scans with `--n 1` and `--n 4194305` and confirm they are still exact.

DONE WHEN

- [ ] Your paper scan matches the program's output for those sixteen values.
- [ ] The inclusive variant is exact at every size you tried.

EVIDENCE FOR THE NOTEBOOK

- The paper scan and the new variant.

LAB 2.3.3 Compaction that returns indices — 1.5 hours

Goal. Adapt the compaction to a new output.

Code. `code/u2/scan`

1. Change the compaction to keep the indices of elements whose highest bit is set, instead of the values that are multiples of three.
2. Check the result against the CPU, and time it at four million and sixteen million elements.

DONE WHEN

- [ ] The kept indices match a CPU implementation exactly at both sizes.
- [ ] You can say what fraction of the compaction's time each of its three steps takes, from timestamps around each.

EVIDENCE FOR THE NOTEBOOK

- The changes, the check and the breakdown of time.

LAB 2.3.4 Sort with eight-bit digits — 2 hours

Goal. Trade passes for work per pass.

Code. `code/u2/sort`

1. Change the sort to eight-bit digits: 256 buckets in the count step and eight splits in the scatter step, in four passes.
2. Check the result against `std::stable_sort`, and compare its time with the four-bit version at four million and sixteen million pairs.

DONE WHEN

- [ ] The eight-bit sort matches `std::stable_sort` exactly.
- [ ] You can explain which version is faster on your GPU, in terms of memory traffic and shared-memory work per pass.

EVIDENCE FOR THE NOTEBOOK

- The changes and the timings, with your explanation.

Problem set

1. Give the work and span of the Hillis–Steele and Blelloch scans for $n = 2^{20}$. For what n do the Blelloch scan's extra steps outweigh Hillis–Steele's extra work on a GPU with 100,000 lanes?
2. In a workgroup of 512 invocations with subgroups of 32, how many subgroups execute each step of the interleaved tree, and of the sequential tree?
3. How many passes does the coarsened reduction need for 2^{24} values with 512 invocations of 8 elements each? Check your answer against the program's output.
4. Explain, with a small example, why the radix sort's output would be wrong if the split within a block were not stable.
5. Why does compaction use an exclusive scan of the flags? What would go wrong with an inclusive scan, and how would you correct it?

COMPLETION CHECKLIST

- [ ] I can explain work and span, and why work efficiency matters.
- [ ] I can write a reduction with shared memory, subgroup operations and coarsening, and measure it against bandwidth.
- [ ] I can write the Hillis–Steele and Blelloch scans and a multi-level scan.
- [ ] I can build compaction and a stable radix sort from a scan.
- [ ] Lab 2.3.1–2.3.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.4

Images and image processing

TIME	6 hours
YOU BUILD	A separable Gaussian blur on storage images, tiled in shared memory with its aprons and compared with a naive two-dimensional kernel; a bilinear downscale through a sampler; and a Mandelbrot renderer that measures how much of the hardware its divergent loops waste. All write PNG files you can look at.
NEEDS	Chapters 2.1 to 2.3, and Chapter 1.2's section on images.

WHY IT MATTERS

Images differ from buffers in three ways. The GPU stores them in layouts suited to two-dimensional access, converts their formats as it reads and writes them, and can filter them in hardware through samplers. For these reasons much of graphics, and of image processing in compute, works on images. This chapter uses images from compute shaders in both ways: as storage images, read and written texel by texel, and as sampled images, filtered by the texture units. It puts their layouts, the subject of Unit 3, into practice for the first time, and it uses an image to make the cost of divergence, introduced in the Handbook, visible and measurable.

SKIP TEST

1. What is the difference between a storage image and a sampled image, as a shader sees them and as the descriptor set describes them?
2. Why is a two-pass separable blur of radius 8 so much faster than a direct two-dimensional one?
3. A workgroup loads a 16 × 16 tile of an image into shared memory to blur it horizontally with radius 8. How many texels must it load, and why?
4. Pixels in one subgroup need 10, 20 and 1000 iterations of the same loop. What does the subgroup cost, and how would you measure the waste?

If all four are easy, read the section on divergence and do Labs 2.4.1 and 2.4.4.

Core ideas

Storage images

A *storage image* is an image that a shader reads and writes texel by texel, much as it would a buffer. GLSL declares it as `image2D` with a *format qualifier* that names the image's format, such as `rgba32f`, `rgba8` or `r32ui`, and accesses it with `imageLoad`, `imageStore` and `imageSize`, using integer coordinates. The descriptor type is `VK_DESCRIPTOR_TYPE_STORAGE_IMAGE`, the image must have been created with `VK_IMAGE_USAGE_STORAGE_BIT`, and while a shader accesses it as storage it is in the `VK_IMAGE_LAYOUT_GENERAL` layout.

Two storage images and a buffer of weights — code/u2/blur/blur_rows.comp

```
layout(local_size_x = 16, local_size_y = 16) in;
layout(constant_id = 0) const int RADIUS = 8;
const int TILE = 16;

layout(set = 0, binding = 0, rgba32f) uniform restrict readonly image2D source;
layout(set = 0, binding = 1, rgba32f) uniform restrict writeonly image2D target;
layout(std430, set = 0, binding = 2) restrict readonly buffer Weights { float weights[]; };
```

Not every format supports storage on every device, as Chapter 1.2's survey showed. The blur stores its intermediate images as `VK_FORMAT_R32G32B32A32_SFLOAT`, which every Vulkan device must support as a storage image, and checks the formats it uses before creating anything:

Refusing to run on a device that lacks a format feature — code/u2/blur/main.cpp

```
void requireFormat(const vkf::Context& ctx, VkFormat format, VkFormatFeatureFlags features) {
    VkFormatProperties properties;
    vkGetPhysicalDeviceFormatProperties(ctx.physicalDevice(), format, &properties);
    if ((properties.optimalTilingFeatures & features) != features) {
        throw std::runtime_error("this device lacks a format feature the blur needs");
    }
}
```

An image created in device memory starts in `VK_IMAGE_LAYOUT_UNDEFINED`, and a barrier must move it to `GENERAL` before a shader writes it, as in Chapter 3.1's catalogue. The program records one such transition for each image it will write:

Creating the images and moving them to GENERAL — code/u2/blur/main.cpp

```
const vkf::ImageDesc desc{
    .format = VK_FORMAT_R32G32B32A32_SFLOAT,
    .width = size,
    .height = size,
    .usage = VK_IMAGE_USAGE_STORAGE_BIT | VK_IMAGE_USAGE_TRANSFER_SRC_BIT |
             VK_IMAGE_USAGE_TRANSFER_DST_BIT,
};
vkf::Image source = vkf::createImage(ctx, desc, "source");
vkf::Image middle = vkf::createImage(ctx, desc, "rows blurred");
vkf::Image blurred = vkf::createImage(ctx, desc, "blurred");
vkf::Image naive = vkf::createImage(ctx, desc, "naive 2D");
vkf::uploadImage(ctx, source, pattern.data(), pattern.size() * sizeof(float),
                 VK_IMAGE_LAYOUT_GENERAL);
vkf::submitNow(ctx, [&](VkCommandBuffer cmd) {
    for (VkImage image : {middle.image, blurred.image, naive.image}) {
        vkf::imageBarrier(cmd, image, VK_IMAGE_LAYOUT_UNDEFINED,
                          VK_IMAGE_LAYOUT_GENERAL, VK_PIPELINE_STAGE_2_NONE,
                          VK_ACCESS_2_NONE, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                          VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT);
    }
});
```

A separable Gaussian blur

A Gaussian blur replaces each pixel with a weighted average of its neighbours, the weights falling off with distance as a Gaussian. Computed directly, a blur of radius r reads $(2r + 1)^2$ pixels for each output pixel: 289 for r = 8. The two-dimensional Gaussian, however, is the product of a horizontal and a vertical one, so the same result comes from two passes of 2r + 1 taps each: blur every row, then blur every column of the result. That is 34 reads per pixel instead of 289.

The test input is a pattern of rings, checks and thin lines, generated on the CPU so that the reference blur can use exactly the same values:

A test pattern with edges in every direction — code/u2/blur/main.cpp

```
std::vector<float> testPattern(uint32_t width, uint32_t height) {
    std::vector<float> rgba(size_t(width) * height * 4);
    for (uint32_t y = 0; y < height; ++y) {
        for (uint32_t x = 0; x < width; ++x) {
            const float u = (x + 0.5f) / width, v = (y + 0.5f) / height;
            const float r = std::hypot(u - 0.5f, v - 0.5f);
            const bool ring = int(r * 20) % 2 == 0;
            const bool square = (x / 32 + y / 32) % 2 == 0;
            const bool line = x % 128 == 64 || y % 128 == 64;
            float* p = &rgba[(size_t(y) * width + x) * 4];
            p[0] = line ? 1.0f : ring ? 0.9f * u : 0.1f;
            p[1] = line ? 1.0f : square ? 0.8f * v : 0.2f;
            p[2] = line ? 1.0f : 0.3f + 0.6f * r;
            p[3] = 1.0f;
        }
    }
    return rgba;
}
```

Each pass is tiled in shared memory. A workgroup of 16 × 16 invocations produces a 16 × 16 tile of output, but to blur the tile's rows it needs eight more pixels on each side: the *apron* (Figure 2.4.1). The workgroup loads the tile with its aprons into shared memory, cooperatively, waits at a barrier, and then each invocation reads its 17 taps from shared memory instead of from the image.

The rows pass: load the tile and its aprons, then blur from shared memory — code/u2/blur/blur_rows.comp

```
shared vec4 row[TILE][TILE + 2 * RADIUS];

void main() {
    ivec2 size = imageSize(source);
    ivec2 origin = ivec2(gl_WorkGroupID.xy) * TILE;
    ivec2 id = ivec2(gl_LocalInvocationID.xy);
    for (int i = id.x; i < TILE + 2 * RADIUS; i += TILE) {
        ivec2 p = ivec2(origin.x - RADIUS + i, origin.y + id.y);
        row[id.y][i] = imageLoad(source, clamp(p, ivec2(0), size - 1));
    }
    barrier();
    vec4 sum = vec4(0.0);
    for (int k = -RADIUS; k <= RADIUS; ++k) {
        sum += weights[abs(k)] * row[id.y][id.x + RADIUS + k];
    }
    ivec2 p = origin + id;
    if (all(lessThan(p, size))) imageStore(target, p, sum);
}
```

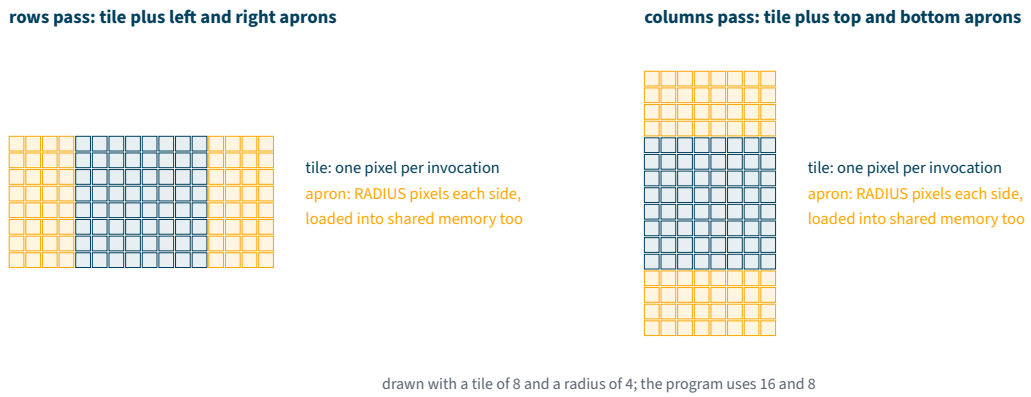


Figure 2.4.1. The data each pass loads into shared memory: the tile it writes, plus aprons wide enough for the blur's radius in the direction of the pass.

The columns pass, the same along the other axis — code/u2/blur/blur_columns.comp

```
shared vec4 column[TILE + 2 * RADIUS][TILE];

void main() {
    ivec2 size = imageSize(source);
    ivec2 origin = ivec2(gl_WorkGroupID.xy) * TILE;
    ivec2 id = ivec2(gl_LocalInvocationID.xy);
    for (int i = id.y; i < TILE + 2 * RADIUS; i += TILE) {
        ivec2 p = ivec2(origin.x + id.x, origin.y - RADIUS + i);
        column[i][id.x] = imageLoad(source, clamp(p, ivec2(0), size - 1));
    }
    barrier();
    vec4 sum = vec4(0.0);
    for (int k = -RADIUS; k ⩽ RADIUS; ++k) {
        sum += weights[abs(k)] * column[id.y + RADIUS + k][id.x];
    }
    ivec2 p = origin + id;
    if (all(lessThan(p, size))) imageStore(target, p, sum);
}
```

Pixels beyond the image's edges are clamped to the edge, in the shader and in the CPU's reference alike. Between the passes, the intermediate image is written by one dispatch and read by the next, so a barrier separates them. It is an image barrier whose old and new layouts are both `GENERAL`: it changes no layout, and serves only as a memory dependency on that image.

The two passes and the barrier between them — code/u2/blur/main.cpp

```
const vkf::Stats separableMs = u2::timeGpu(ctx, runs, [&](VkCommandBuffer cmd) {
    dispatch(cmd, rows, layout, rowsSet, size, size);
    vkf::imageBarrier(
        cmd, middle, VK_IMAGE_LAYOUT_GENERAL, VK_IMAGE_LAYOUT_GENERAL,
        VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
        VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_ACCESS_2_SHADER_STORAGE_READ_BIT);
    dispatch(cmd, columns, layout, columnsSet, size, size);
});
```

For comparison, the naive kernel computes the full two-dimensional sum directly from the image:

A direct two-dimensional blur: 289 taps per pixel — code/u2/blur/blur_2d.comp

```
void main() {
    ivec2 size = imageSize(source);
    ivec2 p = ivec2(gl_GlobalInvocationID.xy);
    if (any(greaterThanEqual(p, size))) return;
    vec4 sum = vec4(0.0);
    for (int dy = -RADIUS; dy ⩽ RADIUS; ++dy) {
        for (int dx = -RADIUS; dx ⩽ RADIUS; ++dx) {
            ivec2 q = clamp(p + ivec2(dx, dy), ivec2(0), size - 1);
            sum += weights[abs(dx)] * weights[abs(dy)] * imageLoad(source, q);
        }
    }
    imageStore(target, p, sum);
}
```

Sampled images

The texture units of a GPU read images through *samplers*, which add what storage images cannot: filtering between texels, normalised coordinates from 0 to 1, rules for coordinates outside the image, and mipmaps. A sampler is an object of its own, created with `vkCreateSampler`, and a shader reaches an image and a sampler together through a `VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER`, declared in GLSL as `sampler2D`. The image must have `VK_IMAGE_USAGE_SAMPLED_BIT` and is read in `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`, and linear filtering requires the format to support it, which the program checks.

A sampler with bilinear filtering that clamps at the edges — code/u2/blur/main.cpp

```
vkf::Unique<VkSampler> createLinearSampler(const vkf::Context& ctx) {
    const VkSamplerCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO,
        .magFilter = VK_FILTER_LINEAR,
        .minFilter = VK_FILTER_LINEAR,
        .mipmapMode = VK_SAMPLER_MIPMAP_MODE_NEAREST,
        .addressModeU = VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE,
        .addressModeV = VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE,
        .addressModeW = VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE,
        .maxLod = 0.0f,
    };
    VkSampler sampler = VK_NULL_HANDLE;
    VKF_CHECK(vkCreateSampler(ctx.device(), &info, nullptr, &sampler));
    return {ctx.device(), sampler};
}
```

The third kernel uses a sampler to shrink the input to three-eighths of its size. Each output pixel samples the input at the matching normalised position, and the texture unit blends the four nearest texels:

A bilinear downscale through `textureLod` — code/u2/blur/downscale.comp

```
layout(set = 0, binding = 0) uniform sampler2D source;
layout(set = 0, binding = 1, rgba8) uniform restrict writeonly image2D target;

void main() {
    ivec2 size = imageSize(target);
    ivec2 p = ivec2(gl_GlobalInvocationID.xy);
    if (any(greaterThanEqual(p, size))) return;
    vec2 uv = (vec2(p) + 0.5) / vec2(size);
    imageStore(target, p, textureLod(source, uv, 0.0));
}
```

The blurs and the downscale on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_blur
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
1024 x 1024 rgba32f image, Gaussian radius 8 (17 taps); median of 10 runs

blur                                    ms   Mpixel/s   max error
separable, two tiled passes          0.483     2173.0     3.0e-07
  the rows pass alone                0.202     5200.1
  the columns pass alone             0.183     5718.8
naive 2D kernel, 289 taps            5.342      196.3     9.5e-07
bilinear downscale to 384 x 384 through a sampler: max error 0.0041 (tolerance 0.0117)
wrote blur_input.png, blur_separable.png and blur_downscaled.png
PASS blur: both blurs and the downscale match the CPU references
```

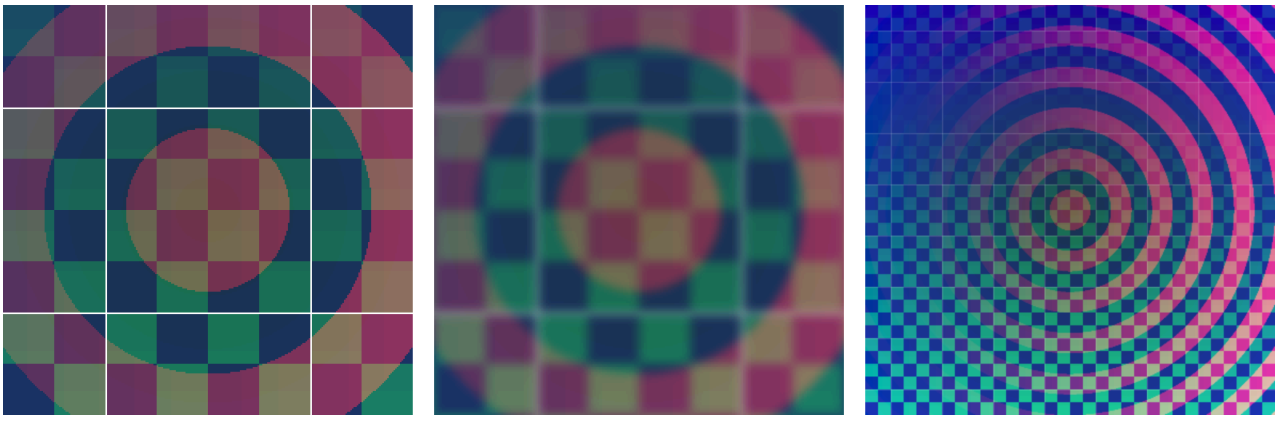


Figure 2.4.2. A 256 × 256 region of the input, the same region blurred, and the whole downscaled image, as the program wrote them.

The separable blur takes 0.48 ms for a 1024 × 1024 image of four floats per pixel, eleven times faster than the naive kernel, which performs eight and a half times as many reads. Each pass alone runs at more than 5 gigapixels per second. Both blurs agree with the CPU to better than 10^{-6}. The downscale's error, 0.004, is about one step of an 8-bit value. The texture unit resolves positions between texels with limited precision and rounds the 8-bit result, both of which the Vulkan specification allows, and the program's tolerance accounts for them.

> **PITFALL**
> Results filtered by a sampler are approximate. An implementation may resolve positions between texels with as few as four bits of precision, the limit `subTexelPrecisionBits` reports, and may round 8-bit formats differently, so results differ slightly between GPUs and from a CPU reference. Never compare sampled results for equality; derive a tolerance from the format and the precision the specification guarantees.

Divergence, made visible

The Mandelbrot set is the set of points c of the complex plane for which the iteration $z \leftarrow z^2 + c$, starting from $z = 0$, stays bounded. A renderer runs the iteration for each pixel until z escapes or a limit is reached, and colours the pixel by how quickly it escaped:

One invocation per pixel, iterating until escape or the limit — code/u2/mandelbrot/mandelbrot.comp

```
void main() {
    ivec2 size = imageSize(picture);
    ivec2 p = ivec2(gl_GlobalInvocationID.xy);
    if (any(greaterThanEqual(p, size))) return;
    vec2 c = push.centre + vec2(p - size / 2) * push.spacing;
    vec2 z = vec2(0.0);
    uint n = 0;
    while (n < push.maxIterations && dot(z, z) ⩽ 256.0) {
        z = vec2(z.x * z.x - z.y * z.y, 2.0 * z.x * z.y) + c;
        ++n;
    }
    imageStore(picture, p, colour(n, z));
    uint index = p.y * size.x + p.x;
    counts[index] = n;
    subgroups[index] = gl_SubgroupID;
}
```

Smooth colouring from the escape count and the final magnitude — code/u2/mandelbrot/mandelbrot.comp

```
vec3 palette(float t) {
    return 0.5 + 0.5 * cos(6.2831853 * (t + vec3(0.0, 0.15, 0.3)));
}

vec4 colour(uint n, vec2 z) {
    if (n == push.maxIterations) return vec4(0.0, 0.0, 0.0, 1.0);
    float mu = float(n) + 1.0 - log2(log2(length(z)));
    return vec4(palette(0.03 * mu), 1.0);
}
```

The program checks itself in two ways. Points known to be inside the set, such as 0 and −1, must reach the iteration limit, and points known to be outside must escape after the number of iterations the CPU computes. Beyond the known points, the CPU's iteration counts must equal the GPU's for at least 99% of a 256 × 256 grid. They cannot be required to match everywhere. The GPU may fuse a multiplication and an addition into one operation with a single rounding where the CPU rounds twice, and near the boundary of the set the iteration is chaotic, so a difference in the last bit of z can change when it escapes.

The CPU's iteration, the reference for every pixel — code/u2/mandelbrot/main.cpp

```
uint32_t iterations(float cx, float cy, uint32_t maxIterations) {
    float zx = 0, zy = 0;
    uint32_t n = 0;
    while (n < maxIterations && zx * zx + zy * zy ⩽ 256.0f) {
        const float x = zx * zx - zy * zy + cx;
        zy = 2.0f * zx * zy + cy;
        zx = x;
        ++n;
    }
    return n;
}
```

Every invocation of a subgroup runs the loop until the slowest of them finishes, so a subgroup whose pixels need widely different numbers of iterations wastes most of its lanes. The program records each pixel's iteration count and subgroup, and computes the fraction of executed lane-iterations that did useful work: the sum of every pixel's iterations, divided by the iterations each subgroup's slowest pixel forced on all its lanes.

Lane use: useful iterations over iterations executed — code/u2/mandelbrot/main.cpp

```
double laneEfficiency(const Render& r, uint32_t size) {
    struct Subgroup {
        uint32_t slowest = 0;
        uint32_t lanes = 0;
    };
    std::unordered_map<uint64_t, Subgroup> subgroups;
    const uint64_t groupsPerRow = vkf::groupCount(size, tile);
    for (uint32_t y = 0; y < size; ++y) {
        for (uint32_t x = 0; x < size; ++x) {
            const uint64_t workgroup = (y / tile) * groupsPerRow + x / tile;
            const size_t i = size_t(y) * size + x;
            Subgroup& s = subgroups[workgroup * tile * tile + r.subgroups[i]];
            s.slowest = std::max(s.slowest, r.counts[i]);
            ++s.lanes;
        }
    }
    double executed = 0;
    for (const auto& [key, s] : subgroups) executed += double(s.slowest) * s.lanes;
    return std::accumulate(r.counts.begin(), r.counts.end(), 0.0) / executed;
}
```

The Mandelbrot checks and three views on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_mandelbrot
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
256 x 256 check grid, at most 1024 iterations
  c = 0, -1 and -0.125+0.125i stay bounded for all 1024 iterations
  c = 0.5, 1 and -2+0.5i escape after 7, 4 and 4 iterations
  the known points: ok
  iteration counts equal the CPU's for 65428 of 65536 pixels (99.84%)

1024 x 1024 views, at most 1024 iterations; median of 10 runs
view                                ms  G iter/s   iterations  lane use
inside the main cardioid         5.069     211.8   1073741824    100.0%
seahorse valley (the boundary)   2.867     162.8    466859070     79.7%
the whole set                    1.106     169.3    187209676     90.8%
wrote mandelbrot.png and mandelbrot_boundary.png
PASS mandelbrot: known points and iteration counts match
```

In the view inside the main cardioid every pixel runs to the limit, so the work is uniform, every lane is in use, and the GPU performs 212 billion iterations per second. The Seahorse Valley view mixes pixels that escape early with pixels that never escape. Its lane use falls to 80%, and its throughput falls by almost the same fraction, to 163 billion iterations per second. Lane use, which a program can measure, predicts most of the cost of divergence.

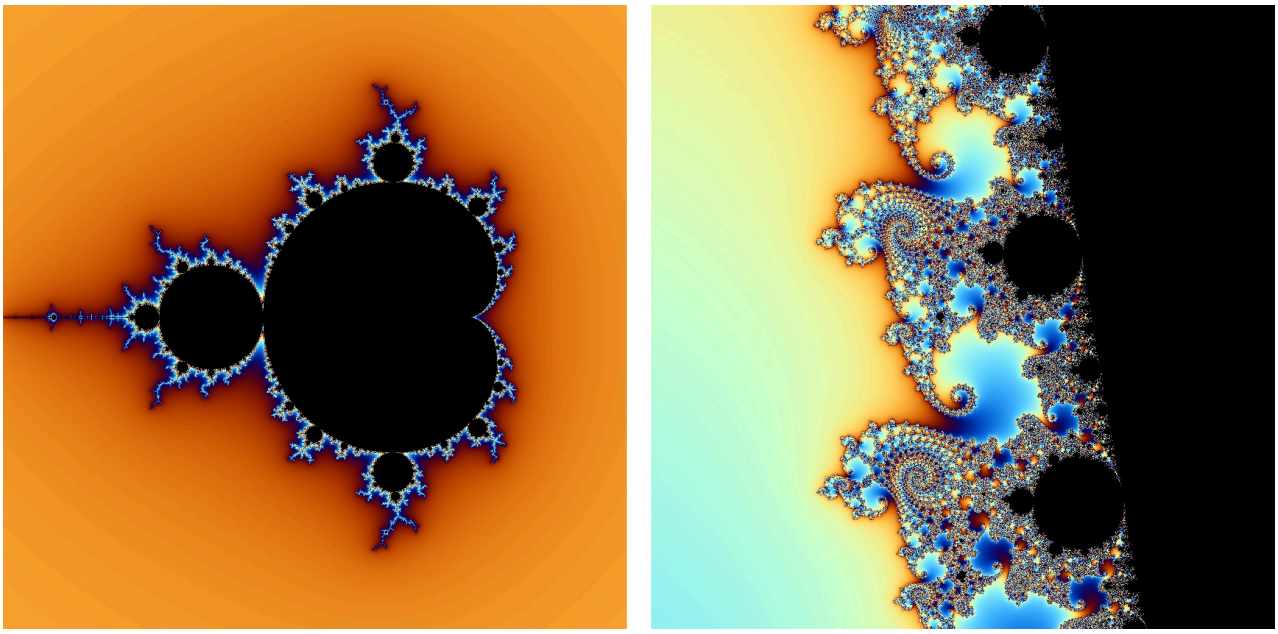


Figure 2.4.3. The whole set, and Seahorse Valley at the edge of the main cardioid, as the program rendered them.

KEY IDEA

A subgroup does not finish until its slowest lane finishes. When work per invocation varies, arrange it so that neighbouring invocations, which share subgroups, do similar amounts: by the order of work, by sorting it, or by splitting it into passes.

COMING FROM OPENGL

OpenGL 4.2 added image load and store: `image2D` variables bound with `glBindImageTexture` to numbered image units, which match Vulkan's storage images. Sampled textures in compute shaders work as in other OpenGL shaders, through texture units bound with `glBindTexture` and `glBindSampler`. OpenGL tracks the image layouts and inserts the transitions, where Vulkan makes each one part of a barrier you record.

Reading

- The Vulkan specification, chapters "Samplers" and "Image Operations", in particular "Texel Filtering", and the format tables in "Formats".
- The Vulkan Guide, "Storage Image and Texel Buffers".
- Hwu, Kirk and El Hajj, *Programming Massively Parallel Processors*, chapters 7 and 8 on convolution and stencils.

Labs

LAB 2.4.1 Blur at several radii — 1.5 hours

Goal. Find where separability and tiling pay off.

Code. `code/u2/blur`

1. Run `VKF_VALIDATION=0 build/bin/u2_blur` with `--radius` set to 1, 2, 4, 8 and 16.
2. Plot the time of the separable and naive blurs against the radius.
3. Work out the largest radius whose tile and aprons fit in your device's shared memory, and try it.

DONE WHEN

- [ ] Every run matches the CPU reference.
- [ ] Your plot shows the two curves, and you can explain their shapes from the number of reads per pixel.

EVIDENCE FOR THE NOTEBOOK

- The plot and the shared-memory calculation.

LAB 2.4.2 Two taps per fetch — 2 hours

Goal. Let the texture unit do half the blur's arithmetic.

Code. `code/u2/blur`

1. Write a rows pass that reads the input through a sampler with linear filtering. Sampling between two texels, at the position their weights determine, returns their weighted sum, so each fetch can replace two taps.
2. Work out the positions and weights for radius 8 on paper, then check the pass against the CPU reference with a tolerance justified by the filtering precision.
3. Time it against the tiled rows pass.

DONE WHEN

- [ ] The new pass matches the reference within your stated tolerance.
- [ ] You can say which rows pass is faster on your GPU, and why.

EVIDENCE FOR THE NOTEBOOK

- The derivation of the positions and weights, the tolerance, and the timings.

LAB 2.4.3 Blur into 8-bit storage — 1 hour

Goal. Trade precision for bandwidth.

Code. `code/u2/blur`

1. Change the output image of the columns pass to `VK_FORMAT_R8G8B8A8_UNORM`, with the `rgba8` format qualifier, and adapt the check.
2. Time both versions.

DONE WHEN

- [ ] The 8-bit output matches the reference within half a level of 8-bit quantisation.
- [ ] You have both timings and can explain the difference in terms of bytes written.

EVIDENCE FOR THE NOTEBOOK

- The changes, the tolerance and the timings.

LAB 2.4.4 Divergence and the shape of the workgroup — 1.5 hours

Goal. See how workgroup shape changes lane use.

Code. `code/u2/mandelbrot`

1. Change the Mandelbrot kernel's local size to 8×8, 32×8 and 64×4, adjusting `tile` and the dispatch, and run the boundary view with each.
2. Record the lane use and the time.
3. Explain why the shape of a subgroup's pixels on the screen affects its lane use.

DONE WHEN

- [ ] The checks pass for every shape.
- [ ] Your explanation predicts which shape uses its lanes best, and the measurements agree or you can say why not.

EVIDENCE FOR THE NOTEBOOK

- The table of shapes, lane use and times, and your explanation.

Problem set

1. How many texels does a 16 × 16 workgroup load into shared memory for the rows pass at radius 8? How many reads of shared memory does it perform? Compare with the naive kernel's reads of the image.
2. Why does the barrier between the blur's passes use an image barrier from `GENERAL` to `GENERAL`, and could it be a global memory barrier instead?
3. A sampler with linear filtering reads a 1024-texel-wide image at normalised coordinate 0.5. Which texels does it blend, and with what weights?
4. A subgroup of 32 pixels has 31 that escape after 10 iterations and one that never escapes, with a limit of 1024. What is its lane use?
5. Why can the Mandelbrot program not demand that every iteration count equals the CPU's? What could it demand instead, and would that be stronger or weaker than its check?

COMPLETION CHECKLIST

- [ ] I can use storage images and sampled images from compute shaders, with the right descriptors, formats and layouts.
- [ ] I can write a tiled, separable filter with aprons in shared memory and verify it.
- [ ] I can create and use samplers, and choose a tolerance for filtered results.
- [ ] I can measure divergence as lane use, and reduce it.
- [ ] Lab 2.4.1–2.4.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.5

Measuring and tuning

TIME	6 hours
YOU BUILD	GPU timing with timestamp queries; a benchmark of your device's memory bandwidth, with copies, vector loads and strided reads; a roofline measured point by point; and an autotuner that builds thirty variants of the reduction with specialisation constants and a pipeline cache, and picks the fastest.
NEEDS	Chapters 2.1 to 2.4.

WHY IT MATTERS

Every chapter of this unit has reported times, and every claim about speed has rested on them. This chapter explains how they are taken and how to interpret them: what the hardware can do at best, how close a kernel comes, and what limits it. It measures your GPU's bandwidth and arithmetic peak instead of relying on the vendor's figures, draws its roofline from measurements, and uses specialisation constants to find which configuration of a kernel runs fastest on it.

SKIP TEST

1. What does `vkCmdWriteTimestamp2` record, and how do you convert the values it returns to milliseconds?
2. Why does a benchmark need a warm-up, and why report the median rather than the mean?
3. A kernel performs 2 FLOP per byte and reaches 380 GFLOP/s on a GPU with 190 GB/s of bandwidth and 6 TFLOP/s of arithmetic. Is it memory-bound or compute-bound, and how far is it from its limit?
4. Why tune with specialisation constants rather than by editing the shader?

If all four are easy, read the section on autotuning and do Labs 2.5.1 and 2.5.3.

Core ideas

Timestamps

A *timestamp query* records the GPU's clock at a point in a command buffer. Queries live in a `VkQueryPool` of type `VK_QUERY_TYPE_TIMESTAMP`. A program resets the queries it will use, writes timestamps with `vkCmdWriteTimestamp2`, and after the work completes reads them with `vkGetQueryPoolResults`. The values are counts of ticks. `timestampPeriod`, a device limit, gives the nanoseconds per tick, and each queue family's `timestampValidBits` gives how many bits are meaningful; zero means that the family cannot write timestamps at all. `vkf::GpuTimer` wraps these steps:

A pool of timestamp queries, after checking that the queue supports them — code/common/src/timing.cpp

```
GpuTimer::GpuTimer(const Context& ctx, uint32_t capacity) : ctx_(&ctx), capacity_(capacity) {
    const uint32_t bits =
        ctx.properties().queueFamilies[ctx.mainQueue().family].timestampValidBits;
    if (bits == 0) {
        throw Error(VK_ERROR_FEATURE_NOT_PRESENT,
                    "the main queue does not support timestamps");
    }
    validMask_ = bits >= 64 ? ~0ull : ((1ull << bits) - 1);
    periodNs_ = ctx.properties().core.limits.timestampPeriod;

    VkQueryPoolCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_QUERY_POOL_CREATE_INFO,
        .queryType = VK_QUERY_TYPE_TIMESTAMP,
        .queryCount = capacity,
    };
    VkQueryPool pool = VK_NULL_HANDLE;
    VKF_CHECK(vkCreateQueryPool(ctx.device(), &info, nullptr, &pool));
    pool_ = Unique<VkQueryPool>(ctx.device(), pool);
}
```

Resetting the pool and writing timestamps in a command buffer — code/common/src/timing.cpp

```
void GpuTimer::reset(VkCommandBuffer cmd) {
    vkCmdResetQueryPool(cmd, pool_, 0, capacity_);
    used_ = 0;
    lastRead_.clear();
}

uint32_t GpuTimer::stamp(VkCommandBuffer cmd, VkPipelineStageFlags2 stage) {
    if (used_ == capacity_) {
        throw Error(VK_ERROR_OUT_OF_POOL_MEMORY,
                    "GpuTimer is full; create it with a larger capacity");
    }
    vkCmdWriteTimestamp2(cmd, stage, pool_, used_);
    return used_++;
}
```

Reading the ticks and converting them to milliseconds — code/common/src/timing.cpp

```
std::vector<double> GpuTimer::read() {
    std::vector<uint64_t> ticks(used_);
    if (used_ > 0) {
        VKF_CHECK(vkGetQueryPoolResults(
            ctx_->device(), pool_, 0, used_, ticks.size() * sizeof(uint64_t), ticks.data(),
            sizeof(uint64_t), VK_QUERY_RESULT_64_BIT | VK_QUERY_RESULT_WAIT_BIT));
    }
    lastRead_.assign(used_, 0.0);
    for (uint32_t i = 1; i < used_; ++i) {
        const uint64_t delta = (ticks[i] - ticks[0]) & validMask_;
        lastRead_[i] = static_cast<double>(delta) * periodNs_ / 1e6;
    }
    return lastRead_;
}
```

A timestamp is written when every earlier command in submission order has completed the stage it names. With `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`, the default here, that means when everything before it has finished, which makes the difference between two timestamps the time the work between them took on the device. Timestamps measure only the GPU; time that a submission spends waiting in the driver or the queue is not included, so they are more useful than wall-clock time for timing kernels.

Measuring well

A single timing is unreliable. GPUs raise their clocks over the first milliseconds of load and lower them when idle; the first run of a pipeline may pay for compilation or for page faults; other programs share the GPU. Unit 2's programs time every kernel with `u2::timeGpu`. It first runs the kernel for about 25 ms to

warm the GPU up. It then runs it repeatedly, with a timestamp before and after each run and a full barrier between runs so that runs cannot overlap, and returns the median:

Warm-up, repeated runs and the median — code/u2/shared/u2.hpp

```
inline vkf::Stats timeGpu(const vkf::Context& ctx, int runs,
                          const std::function<void(VkCommandBuffer)>& body) {
    vkf::GpuTimer timer(ctx, 2 * static_cast<uint32_t>(runs) + 2);
    auto repeat = [&](VkCommandBuffer cmd, int count, bool stampEach) {
        for (int run = 0; run < count; ++run) {
            vkf::memoryBarrier(cmd, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT,
                               VK_ACCESS_2_MEMORY_WRITE_BIT,
                               VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT,
                               VK_ACCESS_2_MEMORY_READ_BIT | VK_ACCESS_2_MEMORY_WRITE_BIT);
            if (stampEach) timer.stamp(cmd);
            body(cmd);
            if (stampEach) timer.stamp(cmd);
        }
    };
    vkf::submitNow(ctx, [&](VkCommandBuffer cmd) {
        timer.reset(cmd);
        timer.stamp(cmd);
        repeat(cmd, 3, false);
        timer.stamp(cmd);
    });
    // GPUs raise their clocks after some milliseconds of load, so warm up for about 25 ms.
    const double perRun = std::max(timer.elapsedMs(0, 1) / 3, 1e-3);
    const int warmup = std::clamp(static_cast<int>(25.0 / perRun), 1, 1000);
    vkf::submitNow(ctx, [&](VkCommandBuffer cmd) {
        timer.reset(cmd);
        repeat(cmd, warmup, false);
        repeat(cmd, runs, true);
    });
    std::vector<double> ms;
    for (uint32_t run = 0; run < static_cast<uint32_t>(runs); ++run) {
        ms.push_back(timer.elapsedMs(2 * run, 2 * run + 1));
    }
    return vkf::summarize(ms);
}
```

The median ignores the occasional run that something else interrupted, which would distort a mean. Lab 2.5.2 asks you to look at the whole distribution.

Bandwidth

The first question about any kernel that reads and writes memory is how fast memory can be read and written at all. `u2_bandwidth` measures three ways of copying 128 MiB: `vkCmdCopyBuffer`, which uses whatever path the driver chooses; a shader that copies one float per iteration; and a shader that copies a `vec4`, sixteen bytes, per iteration.

A copy with sixteen-byte loads and stores — code/u2/bandwidth/copy_vec4.comp

```
layout(std430, set = 0, binding = 0) restrict readonly buffer Src { vec4 src[]; };
layout(std430, set = 0, binding = 1) restrict writeonly buffer Dst { vec4 dst[]; };

void main() {
    uint stride = gl_NumWorkGroups.x * gl_WorkGroupSize.x;
    for (uint i = gl_GlobalInvocationID.x; i < push.count / 4; i += stride) {
        dst[i] = src[i];
    }
}
```

Timing the three copies and checking each result — code/u2/bandwidth/main.cpp

```
const CountPush all{count};
auto copyFloat = u2::loadPipeline(ctx, layout, "copy_float.comp");
auto copyVec4 = u2::loadPipeline(ctx, layout, "copy_vec4.comp");
const vkf::Stats copyBuffer = u2::timeGpu(ctx, runs, [&](VkCommandBuffer cmd) {
    const VkBufferCopy region{.srcOffset = 0, .dstOffset = 0, .size = bytes};
    vkCmdCopyBuffer(cmd, src, dst, 1, &region);
});
const bool copyBufferOk = output(count) == input;
const vkf::Stats floats = u2::timeGpu(ctx, runs, [&](VkCommandBuffer cmd) {
    record(cmd, copyFloat, &all, sizeof(all), count);
});
const bool floatsOk = output(count) == input;
const vkf::Stats vec4s = u2::timeGpu(ctx, runs, [&](VkCommandBuffer cmd) {
    record(cmd, copyVec4, &all, sizeof(all), count / 4);
});
const bool vec4sOk = output(count) == input;
```

The program then reads with a stride: neighbouring invocations read floats a fixed distance apart, so that the reads of a subgroup are scattered across memory, while the output is still written contiguously.

Reads a stride apart, writes contiguous — code/u2/bandwidth/strided.comp

```
void main() {
    uint rows = push.count / STRIDE;
    uint stride = gl_NumWorkGroups.x * gl_WorkGroupSize.x;
    for (uint i = gl_GlobalInvocationID.x; i < push.count; i += stride) {
        dst[i] = src[(i % rows) * STRIDE + i / rows];
    }
}
```

Copies, strided reads and the roofline on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_bandwidth
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
copies of 128 MiB; median of 10 runs
copy                              ms     GB/s   result
vkCmdCopyBuffer                1.408    190.7   exact
shader, float loads            1.424    188.5   exact
shader, vec4 loads             1.434    187.2   exact

strided reads of 64 MiB: neighbouring invocations read floats a stride apart
stride        ms     GB/s   result
     1     0.814    164.9   exact
     2     1.057    126.9   exact
     4     1.754     76.5   exact
     8     3.190     42.1   exact
    16     6.057     22.2   exact
    32     5.946     22.6   exact

roofline: 16777216 floats, each read, put through k FMAs and written
     k  FLOP/byte        ms   GFLOP/s     GB/s   result
     1       0.25     0.716      46.9    187.4   exact
     2       0.50     0.718      93.4    186.9   exact
     4       1.00     0.719     186.7    186.7   exact
     8       2.00     0.739     363.1    181.5   exact
    16       4.00     0.721     744.3    186.1   exact
    32       8.00     0.720    1492.3    186.5   exact
    64      16.00     0.743    2891.4    180.7   exact
   128      32.00     0.735    5840.7    182.5   exact
   256      64.00     1.397    6148.0     96.1   exact
   512     128.00     2.773    6196.0     48.4   exact
  1024     256.00     5.528    6215.5     24.3   exact

measured peaks: 190.7 GB/s (best copy), 6215.5 GFLOP/s (best roofline point)
ridge point: 32.6 FLOP/byte
PASS bandwidth: 20 of 20 results verified
```

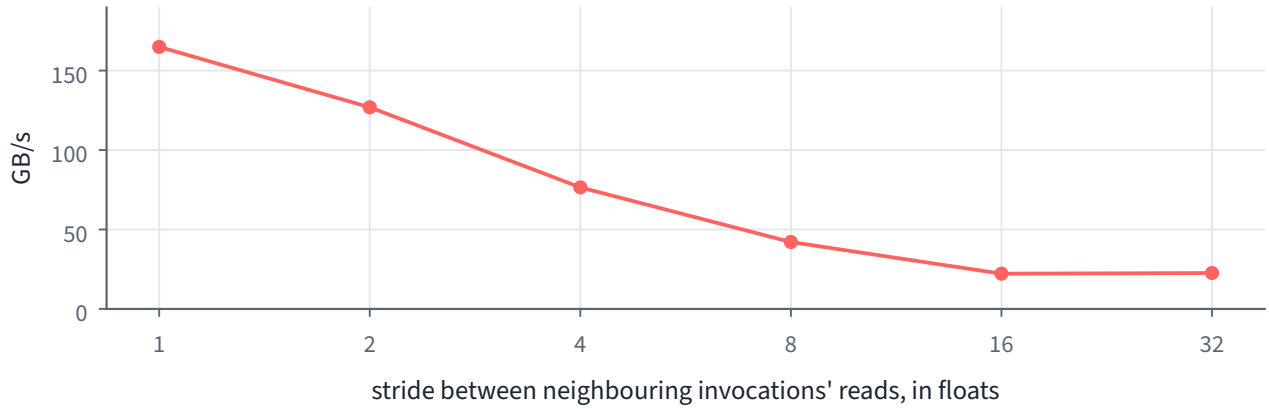


Figure 2.5.1. Bandwidth against the stride between neighbouring invocations' reads, on the M2 Pro.

The M2 Pro's memory reaches 191 GB/s through `vkCmdCopyBuffer`, 95% of the 200 GB/s Apple quotes, and a little less through shaders. The bandwidth of strided reads falls quickly as the stride grows (Figure 2.5.1). Memory is read in lines of many bytes, so once neighbouring invocations read floats sixteen or more apart, each four-byte read costs a whole line, and the bandwidth that reaches the kernel falls to about a seventh.

The roofline, measured

Part 3 of the Handbook drew a roofline from an illustrative GPU's specification. This program measures the roofline of your own GPU. Its kernel reads a `vec4`, applies `k` fused multiply-adds to it and writes the result, so that `k`, a specialisation constant, sets the arithmetic intensity: each element moves eight bytes and performs 2k operations.

A kernel whose arithmetic intensity is set by a specialisation constant — code/u2/bandwidth/roofline.comp

```
void main() {
    uint i = gl_GlobalInvocationID.x;
    if (i ⩾ push.count / 4) return;
    vec4 v = src[i];
    uint k = 0;
    // Loop overhead would dominate a rolled loop of single FMAs, so the bulk is unrolled.
    for (; k + 16 ⩽ FMAS; k += 16) {
        [[unroll]] for (uint j = 0; j < 16; ++j) {
            v = fma(v, vec4(push.a), vec4(push.b));
        }
    }
    for (; k < FMAS; ++k) {
        v = fma(v, vec4(push.a), vec4(push.b));
    }
    dst[i] = v;
}
```

One point of the roofline — code/u2/bandwidth/main.cpp

```
vkf::Specialization spec;
spec.set(0, fmas);
auto kernel = u2::loadPipeline(ctx, layout, "roofline.comp", spec);
const vkf::Stats ms = u2::timeGpu(ctx, runs, [&](VkCommandBuffer cmd) {
    record(cmd, kernel, &rooflinePush, sizeof(rooflinePush), part / 4);
});
const double flops = 2.0 * fmas * part;
const double moved = 2.0 * part * sizeof(float);
points.push_back({fmas, ms.median, flops / (ms.median * 1e6),
                  u2::gbPerSecond(moved, ms.median)});
```

The measured points follow the two bounds closely, and the ridge, where the arithmetic catches up with the memory, falls at about 33 operations per byte. Most kernels of this unit so far lie far to its left: SAXPY at 0.17, the reduction at 0.25, the separable blur at about 4. For such kernels the arithmetic is nearly free, and the only way to speed them up is to move fewer bytes, or to move them more efficiently, which is what tiling, coalescing and coarsening did. The Mandelbrot renderer, which reads nothing and writes a

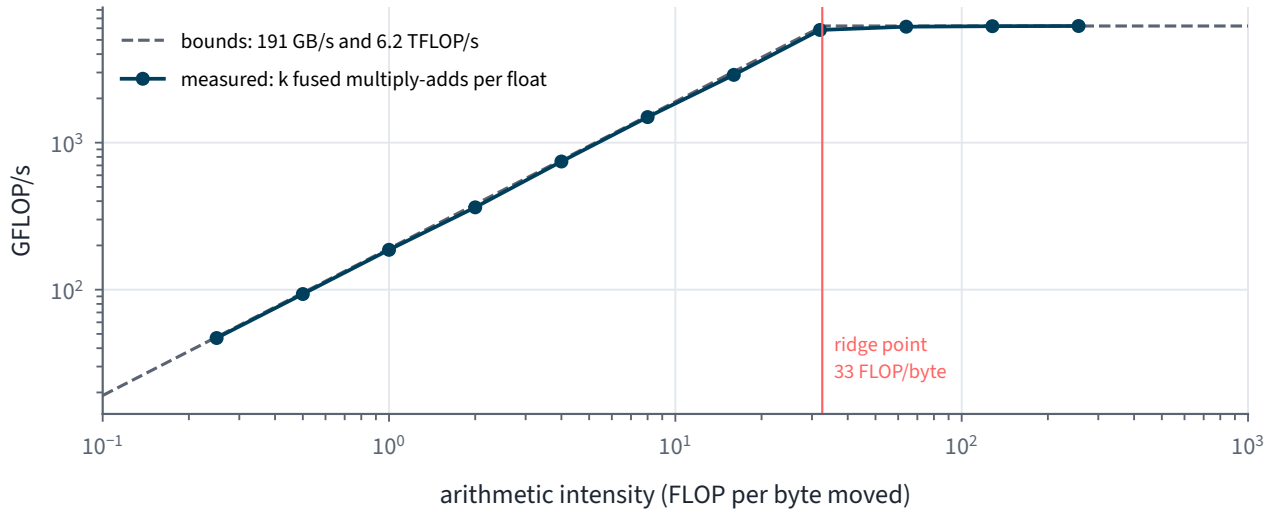


Figure 2.5.2. The M2 Pro's roofline, measured. Up to 32 operations per byte, the kernel runs at the memory's bandwidth; beyond, at about 6.2 trillion operations per second.

few bytes per pixel after hundreds of iterations, lies far to the right, as do Chapter 2.6's N-body simulation and Chapter 2.7's ray tracer.

> **KEY IDEA**
> Before optimising a kernel, place it on your GPU's roofline. To the left of the ridge, the bytes it moves limit its speed; to the right, the operations it performs. Once a kernel is close to its bound, further tuning gains little, whatever its absolute speed.

Occupancy, registers and subgroups

The execution model sets further limits below the bounds. A kernel can fall short of the bandwidth bound because too few subgroups are resident to hide memory latency (the *occupancy* of Part 3 of the Handbook, which registers and shared memory per workgroup limit), because its accesses are not coalesced, or because its workgroups synchronise too often. Vendor profilers show occupancy directly: NVIDIA Nsight Graphics, AMD's Radeon GPU Profiler and Xcode's Metal debugger all report the resident subgroups, the registers per invocation and the memory throughput of each dispatch. Vulkan does not expose them, so a portable program measures effects instead, as the rest of this chapter does.

The subgroup size is one more parameter. Vulkan 1.3's `subgroupSizeControl` lets a pipeline require a particular size, within `minSubgroupSize` and `maxSubgroupSize`, for the stages listed in `requiredSubgroupSizeStages`, through a `VkPipelineShaderStageRequiredSubgroupSizeCreateInfo`, which `vkf::ComputePipelineDesc::requiredSubgroupSize` passes. On AMD's RDNA and on Intel GPUs the choice can matter. On the M2 Pro, MoltenVK does not list the compute stage in `requiredSubgroupSizeStages`, so the size cannot be chosen, and the tuner below checks for that:

The subgroup sizes worth trying, if the device lets a compute pipeline choose — code/u2/tune/main.cpp

```
std::vector<uint32_t> requiredSubgroupSizes(const vkf::Context& ctx) {
    const VkPhysicalDeviceVulkan13Properties& p = ctx.properties().v13;
    if (!ctx.features().v13.subgroupSizeControl ||
        (p.requiredSubgroupSizeStages & VK_SHADER_STAGE_COMPUTE_BIT) == 0) {
        return {0};
    }
    std::vector<uint32_t> sizes;
    for (uint32_t size = p.minSubgroupSize; size <= p.maxSubgroupSize; size *= 2) {
        sizes.push_back(size);
    }
    return sizes;
}
```

Autotuning

The best local size and coarsening factor for a kernel depend on the GPU, and sometimes on the problem's size. Instead of guessing, a program can try them. `u2_tune` builds the subgroup reduction of Chapter 2.3 with every combination of five local sizes and six coarsening factors, both specialisation constants, times each, checks each sum, and reports the fastest.

The combinations to try — code/u2/tune/main.cpp

```
std::vector<Variant> variants;
for (uint32_t subgroupSize : subgroupSizes) {
    for (uint32_t localSize = 64; localSize ⩽ limits.maxComputeWorkGroupInvocations;
         localSize *= 2) {
        const bool fits = subgroupSize == 0 ||
                          localSize ⩽ subgroupSize * v13.maxComputeWorkgroupSubgroups;
        for (uint32_t items = 1; items ⩽ 32 && fits; items *= 2) {
            variants.push_back({localSize, items, subgroupSize});
        }
    }
}
```

One pipeline per combination, through the pipeline cache — code/u2/tune/main.cpp

```
vkf::Unique<VkPipeline> build(const vkf::Context& ctx, VkPipelineLayout layout,
                              VkShaderModule module, VkPipelineCache cache, const Variant& v) {
    vkf::Specialization spec;
    spec.set(0, v.localSize).set(1, v.items);
    return vkf::createComputePipeline(ctx, {.layout = layout,
                                            .module = module,
                                            .specialization = spec.info(),
                                            .requiredSubgroupSize = v.subgroupSize,
                                            .cache = cache});
}
```

Thirty pipelines take time to build, so the tuner uses a pipeline cache, as Chapter 1.4 described, and reports what the cache saves:

Building every variant with an empty cache, then from its contents — code/u2/tune/main.cpp

```
const auto module = vkf::loadShader(ctx, vkf::shaderPath("subgroup.comp"));
auto coldCache = createCache(ctx, {});
vkf::CpuTimer timer;
for (Variant& v : variants) v.pipeline = build(ctx, layout, module, coldCache, v);
const double coldMs = timer.elapsedMs();
const std::vector<char> data = cacheData(ctx, coldCache);
auto warmCache = createCache(ctx, data);
timer.restart();
for (Variant& v : variants) v.pipeline = build(ctx, layout, module, warmCache, v);
const double warmMs = timer.elapsedMs();
```

Timing and checking every variant — code/u2/tune/main.cpp

```
for (Variant& v : variants) {
    const std::vector<u2::ReducePass> passes =
        u2::planReduction(n, v.localSize * v.items, inputToA, aToB, bToA);
    if (passes.front().groups > limits.maxComputeWorkGroupCount[0]) continue;
    v.ms = u2::timeGpu(ctx, runs, [&](VkCommandBuffer cmd) {
               u2::recordReduction(cmd, v.pipeline, layout, passes);
           }).median;
    const float sum = vkf::download<float>(ctx, passes.size() % 2 ? a : b, 1)[0];
    v.correct = std::abs(sum - expected) / expected < 1e-5;
}
```

Thirty variants of the reduction on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_tune
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
subgroup size control: requiredSubgroupSizeStages lacks the compute stage,
so every variant runs at the driver's subgroup size
built 30 pipelines in 68.0 ms with an empty pipeline cache, and in 1.6 ms
from the 5816 bytes that cache then held

GB/s by local size (columns) and elements per invocation (rows)
           64     128     256     512    1024
    1    88.2    88.4    88.9    89.1   159.6
    2   168.0   166.4   162.5   162.8   180.1
    4   190.4   187.9   184.4   176.9   177.6
    8   185.9   184.9   174.1   175.3   181.8
   16   182.9   176.2   174.4   178.5   180.9
   32   176.3   166.4   174.6   178.6   181.7

fastest: local size 64, 4 elements per invocation: 0.352 ms (190.4 GB/s)
PASS tune: 30 of 30 variants verified
```

This run's first build was cold. Run the tuner again and the build with an empty cache takes only a few milliseconds as well, because Metal keeps the shaders it has compiled in a cache of its own, outside the program's control. Chapter 4.6 measures pipeline caches with that in mind.

The fastest variant reaches 190 GB/s, the bandwidth of a copy, with 64 invocations per workgroup and four elements each. Most combinations with two or more elements per invocation come within 10% of it, and every combination with one element is half as fast, except, on this GPU, with 1024 invocations per workgroup. A real program would run such a tuner once per device, store the winner beside its pipeline cache, and build only that variant thereafter.

PITFALL

Check the results of every benchmark. A kernel that skips work, reads the wrong buffer or exits early can look fast. Every timed variant in this course verifies its output, and the tuner rejects a variant whose sum is wrong, however fast it was.

COMING FROM OPENGL

OpenGL measures GPU time with timer queries, `glQueryCounter(query, GL_TIMESTAMP)` and `glBeginQuery(GL_TIME_ELAPSED, …)`, and the same caveats about warm-up and variation apply. Tuning by

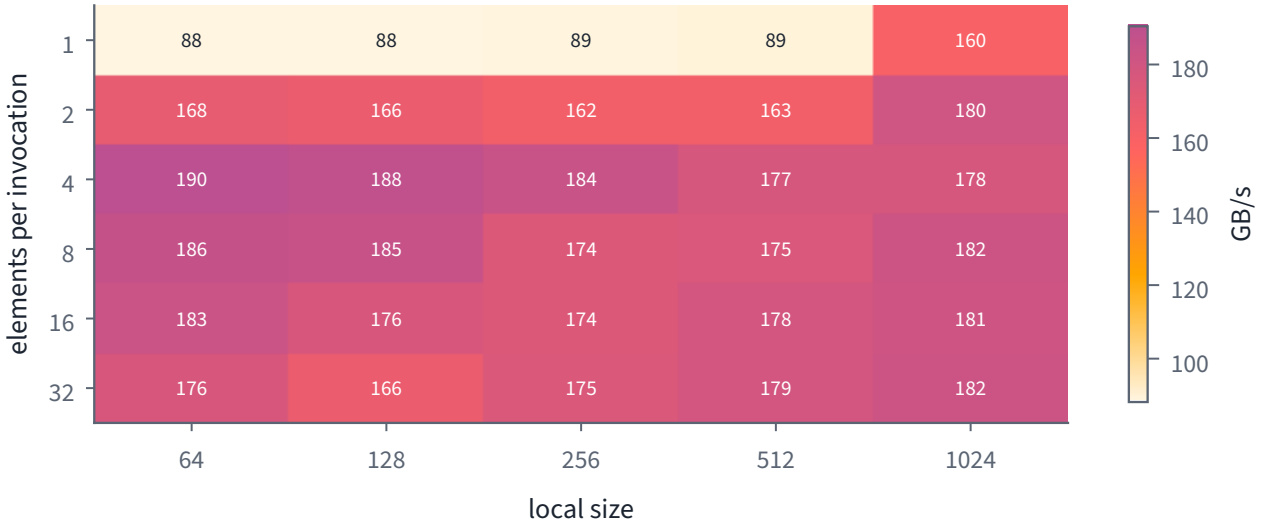


Figure 2.5.3. The reduction's bandwidth for each combination. Coarsening matters far more than the local size: one element per invocation is slow at every size but the largest.

specialisation has no direct equivalent: an OpenGL program would compile a separate program object for each variant, usually by editing the source text with `#define`s.

Reading

- The Vulkan specification, chapter "Queries", in particular "Timestamp Queries", and `VkPhysicalDeviceSubgroupSizeControlProperties`.
- Samuel Williams, Andrew Waterman and David Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures", *Communications of the ACM* 52(4), 2009.
- Your GPU vendor's optimisation guide from the Handbook's reference library: NVIDIA's "Vulkan Dos and Don'ts", AMD's *RDNA Performance Guide*, or Apple's documentation for Metal's GPU counters.

Labs

LAB 2.5.1 Your GPU's roofline — 1.5 hours

Goal. Measure your device's bounds and place this unit's kernels against them.

Code. `code/u2/bandwidth`

1. Run `VKF_VALIDATION=0 build/bin/u2_bandwidth` three times and plot the roofline from the median of each point.
2. Compare the measured bandwidth and arithmetic peaks with your GPU's published figures.
3. Place SAXPY, the coarsened reduction and the separable blur on the plot, from their measured speeds and arithmetic intensities.

DONE WHEN

- [ ] Your plot shows both bounds and at least twelve measured points.
- [ ] For each of the three kernels you can say which bound limits it, and by how much it misses it.

EVIDENCE FOR THE NOTEBOOK

- The plot and the three placements.

LAB 2.5.2 Timestamps by hand — 1 hour

Goal. Use timestamp queries without the helper, and see what a distribution looks like.

Code. `code/u2/localsize`

1. Write a timer of your own around one SAXPY dispatch: a query pool, a reset, two timestamps and a read, with `timestampPeriod` and `timestampValidBits` applied.
2. Time 200 runs, without warm-up, and plot the times in order and as a histogram.

DONE WHEN

- [ ] Your timer agrees with `vkf::GpuTimer` to within the run-to-run variation.
- [ ] Your plots show the warm-up and the spread, and you can say how many runs to discard and why the median is the right summary.

EVIDENCE FOR THE NOTEBOOK

- The code and both plots.

LAB 2.5.3 Tune on your GPU 1.5 hours

Goal. Find out how stable the tuner's choice is.

Code. `code/u2/tune`

1. Run `VKF_VALIDATION=0 build/bin/u2_tune` three times and record the winner each time.
2. Run it with `--n` set to 2^{20} and 2^{26}.
3. If your device allows the subgroup size to be chosen, record its effect.

DONE WHEN

- [ ] You have the three winners at the default size, and the winners at the two other sizes.
- [ ] You can say whether the winning configuration depends on the problem's size on your GPU, and how much choosing the second-best would cost.

EVIDENCE FOR THE NOTEBOOK

- The tables and your conclusion.

LAB 2.5.4 Look inside with a profiler 1.5 hours

Goal. Connect the measurements to the hardware's counters.

Code. `code/u2/reduce`

1. Capture the reduction with your vendor's profiler, or with Xcode on macOS through MoltenVK's capture settings from Chapter 1.6.
2. Find, for the slowest and the fastest variant, the occupancy or resident subgroups, the memory throughput, and the registers per invocation where the tool reports them.

DONE WHEN

- [ ] You have the counters for both variants.
- [ ] You can explain the difference in speed from the counters.

EVIDENCE FOR THE NOTEBOOK

- Screenshots of the counters, annotated.

Problem set

1. Timestamps are written with `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT`. What would a timestamp at `VK_PIPELINE_STAGE_2_TOP_OF_PIPE_BIT` measure instead, and why is it rarely what you want?
2. `timestampPeriod` is 1.0 and `timestampValidBits` is 36. After how long does the counter wrap, and how does `vkf::GpuTimer` cope?
3. The strided read at stride 32 runs at 22 GB/s. Assuming memory is read in lines of 64 bytes, what bandwidth would you predict, and why might it differ?
4. A kernel performs 40 operations per byte. On the M2 Pro's measured roofline, what is its best possible speed? On the illustrative GPU of the Handbook?
5. The tuner keeps the fastest variant. Name two situations in which a slightly slower variant would be the better choice.

COMPLETION CHECKLIST

- [ ] I can time GPU work with timestamp queries and report a sound measurement.
- [ ] I can measure my GPU's bandwidth and arithmetic peak and draw its roofline.
- [ ] I can place a kernel on the roofline and say what limits it.

- [ ] I can tune a kernel with specialisation constants and a pipeline cache, and know when not to trust the result.
- [ ] Lab 2.5.1–2.5.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.6

Dispatch at scale

TIME	6 hours
YOU BUILD	A gravitational N-body simulation of 16,384 particles, tiled in shared memory, that reaches its buffers through device addresses instead of descriptors, sizes its dispatches on the GPU with indirect dispatch, runs correctly with a dispatch-size limit forced to sixteen workgroups, and conserves energy to a few parts in ten million over two hundred steps.
NEEDS	Chapters 2.1 to 2.5.

WHY IT MATTERS

The kernels so far have been dispatched with sizes the CPU computed, through descriptors the CPU wrote, on problems that fit comfortably in one dispatch. Larger GPU programs cannot rely on any of the three: the amount of work is decided by earlier GPU work, data structures point at each other, and problems exceed what one dispatch can cover. This chapter handles each with the feature Vulkan provides for it: indirect dispatch, buffer device addresses, and grids that wrap a problem across dimensions. It does so in a simulation whose arithmetic outweighs its memory traffic, and whose physics provides a check on the results.

SKIP TEST

1. How does a dispatch take its workgroup counts from a buffer, and which barrier does the buffer need if a shader wrote it?
2. What does a buffer device address give a shader that a descriptor does not, and what must a buffer be created with to have one?
3. `maxComputeWorkGroupCount[0]` is 65,535 and a problem needs 100,000 workgroups of 256. How do you dispatch it?
4. Why is an N-body force calculation compute-bound when SAXPY is memory-bound?

If all four are easy, read the section on device addresses and do Labs 2.6.2 and 2.6.4.

Core ideas

The simulation

Each of n particles attracts every other with a force that falls with the square of the distance. Computing every pair is n^2 interactions per step, 268 million for 16,384 particles, and each interaction is about twenty floating-point operations on data that, with tiling, is read from memory only once per workgroup. That puts the kernel far to the right of the roofline's ridge: like the Mandelbrot renderer of Chapter 2.4, it is limited by arithmetic.

The particles start in a *Plummer sphere*, a standard model of a star cluster whose density and velocities are in equilibrium, generated on the CPU. The force is *softened*: a small constant is added to each squared distance, so that close encounters do not produce enormous accelerations. The integrator is *leapfrog*, also called velocity Verlet: a half-step kick of the velocities, a full-step drift of the positions, the forces at the new positions, and another half-step kick. Leapfrog is *symplectic*, so its energy error stays small over long runs. The program checks itself by computing the total energy at the start and the end and requiring them to agree.

The initial conditions: a Plummer sphere, centred on the origin — code/u2/nbody/main.cpp

```
// Aarseth, Henon and Wielen (1974): radii from the cumulative mass, speeds by rejection.
Particles plummer(uint32_t n) {
    std::mt19937 rng(29);
    std::uniform_real_distribution<double> uniform(0.0, 1.0);
    auto direction = [&] {
        const double z = 2 * uniform(rng) - 1, phi = 2 * std::numbers::pi * uniform(rng);
        const double s = std::sqrt(1 - z * z);
        return std::array<double, 3>{s * std::cos(phi), s * std::sin(phi), z};
    };
    std::vector<std::array<double, 6>> state(n);
    std::array<double, 6> mean{};
    for (auto& particle : state) {
        double r = 0, q = 0, y = 0;
        do {
            r = 1 / std::sqrt(std::pow(uniform(rng), -2.0 / 3.0) - 1);
        } while (r > 10);
        do {
            q = uniform(rng);
            y = 0.1 * uniform(rng);
        } while (y > q * q * std::pow(1 - q * q, 3.5));
        const double speed = q * std::sqrt(2.0) * std::pow(1 + r * r, -0.25);
        const auto position = direction(), velocity = direction();
        for (int c = 0; c < 3; ++c) {
            particle[c] = r * position[c];
            particle[3 + c] = speed * velocity[c];
        }
        for (int c = 0; c < 6; ++c) mean[c] += particle[c] / n;
    }
    Particles p{std::vector<float>(4 * size_t(n)), std::vector<float>(4 * size_t(n))};
    for (uint32_t i = 0; i < n; ++i) {
        for (int c = 0; c < 3; ++c) {
            p.positions[4 * i + c] = float(state[i][c] - mean[c]);
            p.velocities[4 * i + c] = float(state[i][3 + c] - mean[3 + c]);
        }
        p.positions[4 * i + 3] = 1.0f / n;
    }
    return p;
}
```

The energy check needs each particle's potential energy, another all-pairs sum, which a fourth kernel, `potential.comp`, computes in the same tiled way as the forces. The CPU adds the kinetic and potential energies in double precision:

Total energy, summed in double precision on the CPU — code/u2/nbody/main.cpp

```
double energy(std::span<const float> positions, std::span<const float> velocities,
              std::span<const float> potentials) {
    double kinetic = 0, potential = 0;
    for (size_t i = 0; i < potentials.size(); ++i) {
        const double mass = positions[4 * i + 3];
        const double vx = velocities[4 * i], vy = velocities[4 * i + 1],
                     vz = velocities[4 * i + 2];
        kinetic += 0.5 * mass * (vx * vx + vy * vy + vz * vz);
        potential += 0.5 * mass * potentials[i];
    }
    return kinetic + potential;
}
```

The GPU computes in single precision, because double precision is slow on most GPUs and unavailable through MoltenVK; only the sums for the energy check use doubles, on the CPU.

Tiling the force calculation

A naive kernel would have each invocation read all n positions from memory. The tiled kernel has the invocations of a workgroup load one tile of 256 positions into shared memory together, each loading one, and then accumulate the pull of all 256 on their own particles, before moving to the next tile (Figure 2.6.1).

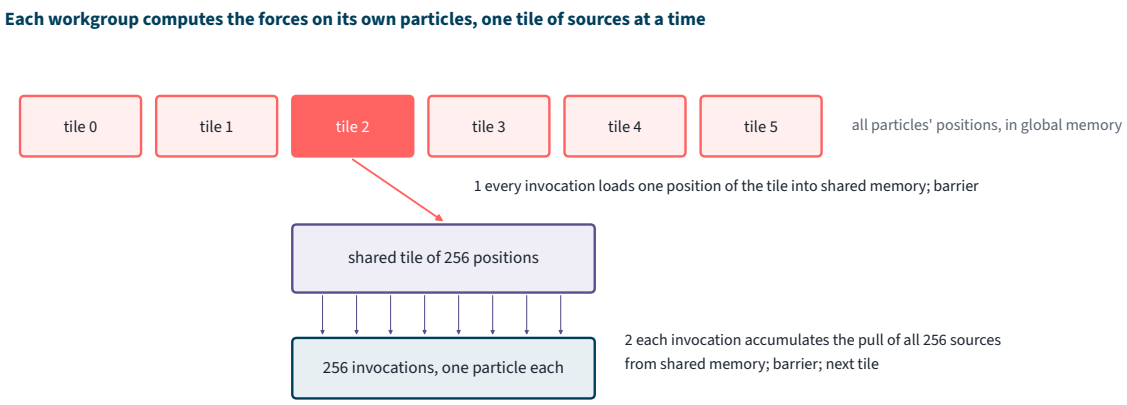


Figure 2.6.1. The tiled force calculation. Shared memory turns 256 reads of each position into one.

Each position is then read from memory once per workgroup instead of once per invocation, a saving of a factor of 256.

Accumulating the forces from shared memory, then the second half kick — code/u2/nbody/forces.comp

```
void main() {
    uint count = push.params.count;
    uint i = particleIndex();
    vec4 self = i < count ? push.positions.v[i] : vec4(0.0);
    vec3 acceleration = vec3(0.0);
    for (uint first = 0; first < count; first += gl_WorkGroupSize.x) {
        uint j = first + gl_LocalInvocationIndex;
        tile[gl_LocalInvocationIndex] = j < count ? push.positions.v[j] : vec4(0.0);
        barrier();
        for (uint k = 0; k < gl_WorkGroupSize.x; ++k) {
            vec3 d = tile[k].xyz - self.xyz;
            float inverse = inversesqrt(dot(d, d) + push.softening2);
            acceleration += (tile[k].w * inverse * inverse * inverse) * d;
        }
        barrier();
    }
    if (i < count) {
        push.accelerations.v[i] = vec4(acceleration, 0.0);
        push.velocities.v[i].xyz += acceleration * push.halfDt;
    }
}
```

The kernel needs no test to skip a particle's pull on itself: the separation is zero, so the softened term contributes nothing. Slots of the last tile beyond the particle count are filled with zero mass for the same reason. The first force dispatch of a run is made with a half step of zero, to compute the initial accelerations without changing the velocities.

The first half kick and the drift — code/u2/nbody/drift.comp

```
void main() {
    uint i = particleIndex();
    if (i ⩾ push.params.count) return;
    vec3 velocity = push.velocities.v[i].xyz + push.accelerations.v[i].xyz * push.halfDt;
    push.velocities.v[i].xyz = velocity;
    push.positions.v[i].xyz += velocity * push.dt;
}
```

Buffer device addresses

The shaders reach their five buffers without any descriptors. Since Vulkan 1.2, a buffer created with `VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT`, from memory allocated with the matching flag, has a 64-bit *device address* that `vkGetBufferDeviceAddress` returns. Shaders that enable `GL_EXT_buffer_reference` declare *buffer reference* types and use such addresses like pointers. The feature `bufferDeviceAddress` must be enabled. `vkf`

enables it whenever the device supports it, and `vkf::createBuffer` allocates the memory of a buffer with that usage with `VK_MEMORY_ALLOCATE_DEVICE_ADDRESS_BIT` and stores its address in `vkf::Buffer::address`.

Buffers with device addresses — code/u2/nbody/main.cpp

```
const VkBufferUsageFlags usage = VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT |
                                 VK_BUFFER_USAGE_TRANSFER_DST_BIT |
                                 VK_BUFFER_USAGE_TRANSFER_SRC_BIT;
auto buffer = [&](VkDeviceSize bytes, VkBufferUsageFlags extra, const char* name) {
    return vkf::createBuffer(ctx, bytes, usage | extra, vkf::MemoryUse::DeviceLocal, name);
};
const VkDeviceSize vec4s = VkDeviceSize(n) * 4 * sizeof(float);
positions_ = buffer(vec4s, 0, "positions");
velocities_ = buffer(vec4s, 0, "velocities");
accelerations_ = buffer(vec4s, 0, "accelerations");
potentials_ = buffer(VkDeviceSize(n) * sizeof(float), 0, "potentials");
params_ = buffer(sizeof(uint32_t), 0, "params");
dispatch_ = buffer(sizeof(VkDispatchIndirectCommand), VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT,
                   "dispatch");
```

The addresses travel in push constants, which hold 64-bit values as easily as 32-bit ones. On the GPU side, each buffer reference type describes the layout of what an address points at:

Buffer reference types, and push constants that carry five addresses — code/u2/nbody/nbody.glsl

```
layout(buffer_reference, std430, buffer_reference_align = 16) buffer Vec4s {
    vec4 v[];
};
layout(buffer_reference, std430, buffer_reference_align = 4) buffer Floats {
    float f[];
};
layout(buffer_reference, std430, buffer_reference_align = 4) buffer Params {
    uint count;
};

layout(push_constant) uniform Push {
    Vec4s positions;
    Vec4s velocities;
    Vec4s accelerations;
    Floats potentials;
    Params params;
    float dt;
    float halfDt;
    float softening2;
} push;
```

The same push constants on the host — code/u2/nbody/main.cpp

```
struct Push {
    VkDeviceAddress positions;
    VkDeviceAddress velocities;
    VkDeviceAddress accelerations;
    VkDeviceAddress potentials;
    VkDeviceAddress params;
    float dt;
    float halfDt;
    float softening2;
};

struct PreparePush {
    VkDeviceAddress params;
    VkDeviceAddress dispatch;
    uint32_t localSize;
    uint32_t maxGroupsX;
};
```

PITFALL

A buffer reference's `buffer_reference_align` declares the alignment of every address of that type; an address that is not so aligned is undefined behaviour. Device addresses of buffers are aligned to the buffer's memory requirements, but an address computed by adding an offset may not be. Declare the alignment your offsets actually guarantee.

Device addresses make GPU data structures straightforward to build: a buffer can hold addresses of other buffers, so that a tree, a list or a scene can link its parts as it would on the CPU, without a descriptor for each. They also remove the bookkeeping of descriptor sets for buffers. They are less safe, however. The validation layer cannot check what a shader does with an address, and a wrong address can corrupt any memory the device can reach. Synchronisation validation, which tracks accesses through descriptors, cannot see accesses through addresses at all, so the barriers between these kernels rest on reasoning alone.

PITFALL

Remove the barriers between the drift and force dispatches and, on the M2 Pro, the simulation still conserves energy and validation still says nothing. Neither proves the barriers unnecessary: the layer cannot see these accesses, and another GPU, or the same one under a different load, may overlap the dispatches.

Indirect dispatch

The number of workgroups a step needs depends on the number of particles, which here is fixed, but in many programs is decided by earlier GPU work: the survivors of a cull, the elements of a compaction, the cells a simulation refined. Reading the count back to the CPU to dispatch would stall the GPU. `vkCmdDispatchIndirect` instead reads the three workgroup counts from a `VkDispatchIndirectCommand` in a buffer, created with `VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT`, when the command executes.

A one-invocation kernel writes the command from the particle count stored on the GPU:

Computing the dispatch on the GPU — code/u2/nbody/prepare.comp

```
layout(buffer_reference, std430, buffer_reference_align = 4) readonly buffer Params {
    uint count;
};
layout(buffer_reference, std430, buffer_reference_align = 4) writeonly buffer Dispatch {
    uint x;
    uint y;
    uint z;
};

layout(push_constant) uniform Push {
    Params params;
    Dispatch dispatch;
    uint localSize;
    uint maxGroupsX;
} push;

void main() {
    uint groups = (push.params.count + push.localSize - 1) / push.localSize;
    uint x = min(groups, push.maxGroupsX);
    push.dispatch.x = x;
    push.dispatch.y = x == 0 ? 0 : (groups + x - 1) / x;
    push.dispatch.z = 1;
}
```

The indirect command is read at `VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT`, before any shader of the dispatch runs, so the barrier between the kernel that writes it and the dispatch that reads it must name that stage and `VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT`:

Preparing the dispatch, then dispatching from it — code/u2/nbody/main.cpp

```
vkf::submitNow(ctx_, [&](VkCommandBuffer cmd) {
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, prepare_);
    vkCmdPushConstants(cmd, prepareLayout_, VK_SHADER_STAGE_COMPUTE_BIT, 0,
                       sizeof(prepare), &prepare);
    vkCmdDispatch(cmd, 1, 1, 1);
    vkf::memoryBarrier(
        cmd, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
        VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT, VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT);
    dispatch(cmd, forces_, start);
});
```

Two hundred steps, each two indirect dispatches with barriers between them — code/u2/nbody/main.cpp

```
vkf::GpuTimer timer(ctx_, 2);
vkf::submitNow(ctx_, [&](VkCommandBuffer cmd) {
    timer.reset(cmd);
    timer.stamp(cmd);
    for (uint32_t s = 0; s < steps; ++s) {
        dispatch(cmd, drift_, step);
        u2::computeBarrier(cmd);
        dispatch(cmd, forces_, step);
        u2::computeBarrier(cmd);
    }
    timer.stamp(cmd);
});
```

Problems larger than a dispatch

`maxComputeWorkGroupCount` limits each dimension of a dispatch, to as little as 65,535. A one-dimensional problem that needs more workgroups can be dispatched as a two-dimensional grid, with each workgroup computing its linear index from both coordinates (Figure 2.6.2). Grid-stride loops, as in Chapter 2.1, are another answer: dispatch fewer workgroups and let each invocation handle several elements. A third is to split the problem across several dispatches, passing each the index of its first element in a push constant.

> **PITFALL**
>
> `vkCmdDispatchBase` dispatches a range of workgroups whose `gl_WorkGroupID` starts at a given base, which looks like a way past the limit. It does not get past it, because the base plus the count must still be within `maxComputeWorkGroupCount`.

Every kernel computes its particle's index from a two-dimensional grid — code/u2/nbody/nbody.glsl

```
uint particleIndex() {
    uint group = gl_WorkGroupID.y * gl_NumWorkGroups.x + gl_WorkGroupID.x;
    return group * gl_WorkGroupSize.x + gl_LocalInvocationIndex;
}
```

The program can pretend that the device's limit is smaller than it is, to test this path on hardware that would never need it. With `--max-groups 16`, the preparation kernel spreads the 64 workgroups over a 16 × 4 grid, and the simulation must give bit-identical positions:

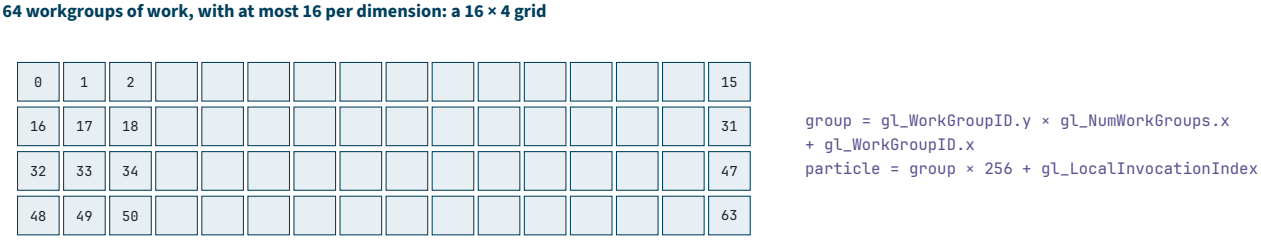


Figure 2.6.2. Sixty-four workgroups' worth of work on a device that allows only sixteen per dimension, as a grid of sixteen by four.

Running again with the limit forced down, and comparing every position — code/u2/nbody/main.cpp

```
const Result forced = simulation.run(steps, dt, forcedLimit);
identical = forced.positions == r.positions;
vkf::print(
    "maxComputeWorkGroupCount[0] forced to {}: {} x {} x {} workgroups, "
    "{} positions\n",
    forcedLimit, forced.groups[0], forced.groups[1], forced.groups[2],
    identical ? "bit-identical" : "DIFFERENT");
```

Two hundred steps on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_nbody
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
16384 particles in a Plummer sphere; 200 leapfrog steps of 0.01
indirect dispatch from a GPU-side count: 64 x 1 x 1 workgroups of 256
2.008 ms per step, 133.7 billion interactions per second
total energy -0.1544414 at the start, -0.1544415 at the end
relative energy drift 2.45e-07 (limit 1e-04)
PASS nbody: energy conserved to 2.4e-07
```

The same with the dispatch limit forced to sixteen, with validation on

```
$ ./build/bin/u2_nbody --max-groups 16
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
16384 particles in a Plummer sphere; 200 leapfrog steps of 0.01
indirect dispatch from a GPU-side count: 64 x 1 x 1 workgroups of 256
2.005 ms per step, 133.9 billion interactions per second
total energy -0.1544414 at the start, -0.1544415 at the end
relative energy drift 2.45e-07 (limit 1e-04)
maxComputeWorkGroupCount[0] forced to 16: 16 x 4 x 1 workgroups, bit-identical positions
PASS nbody: energy conserved to 2.4e-07
```

Each step takes 2 ms, 134 billion interactions per second. At the customary count of twenty operations per interaction, that is 2.7 trillion operations per second, 43% of the arithmetic peak that Chapter 2.5 measured. Part of the difference is accounting: the peak assumes that every instruction is a fused multiply-add, worth two operations, and few of the loop's instructions are. The rest goes mainly to the reciprocal square root, which is slower than a multiply-add, and to the reads from shared memory. The energy drifts by 2.5 parts in ten million over the run, well within the limit of one part in ten thousand.

> **KEY IDEA**
> With indirect dispatch the GPU decides how much work to do, and with device addresses it finds its data the way a CPU program would. Together with grids that wrap their work, they make the GPU-driven rendering of Unit 4 possible.

> **COMING FROM OPENGL**
> OpenGL 4.3 has indirect dispatch, `glDispatchComputeIndirect`, reading from the buffer bound to `GL_DISPATCH_INDIRECT_BUFFER` after a `glMemoryBarrier(GL_COMMAND_BARRIER_BIT)`. Pointers to buffers exist only as NVIDIA's `NV_shader_buffer_load` extension, which has no portable equivalent; Vulkan's buffer device addresses have been core since Vulkan 1.2 and are widely supported.

Reading

- Lars Nyland, Mark Harris and Jan Prins, "Fast N-Body Simulation with CUDA", *GPU Gems 3*, chapter 31: the tiled algorithm this chapter uses.
- The Vulkan Guide, "Buffer Device Address".
- The `GL_EXT_buffer_reference` extension specification, in the KhronosGroup/GLSL repository.

- S. J. Aarseth, M. Hénon and R. Wielen, “A comparison of numerical methods for the study of star cluster dynamics”, *Astronomy and Astrophysics* 37, 1974, for the Plummer sphere’s construction.

Labs

LAB 2.6.1 Run, scale and check — 1.5 hours

Goal. Measure how the simulation scales with n.

Code. `code/u2/nbody`

1. Run `VKF_VALIDATION=0 build/bin/u2_nbody` with `--n` set to 4096, 16,384 and 65,536.
2. Record the interactions per second and the energy drift at each size.
3. Place the force kernel on your GPU’s roofline from Chapter 2.5.

DONE WHEN

- [ ] Every run passes its energy check.
- [ ] You can explain how the interactions per second change with n, from the roofline and the number of workgroups.

EVIDENCE FOR THE NOTEBOOK

- The table and the placement on the roofline.

LAB 2.6.2 Without tiling — 1 hour

Goal. Measure what shared memory saves here.

Code. `code/u2/nbody`

1. Write a force kernel that reads every source position directly from the buffer, and time it against the tiled one.
2. Try tiles of 64 and 512 invocations in the tiled kernel.

DONE WHEN

- [ ] Every version conserves energy within the limit.
- [ ] You can explain the difference in speed in terms of memory traffic, using your GPU’s caches.

EVIDENCE FOR THE NOTEBOOK

- The timings and the explanation.

LAB 2.6.3 Change the particle count on the GPU — 2 hours

Goal. Drive the indirect dispatch from a count that the GPU computes.

Code. `code/u2/nbody`

1. Add a pass that removes particles that move beyond a radius of 50, with stream compaction from Chapter 2.3, and writes the new count to the parameter buffer.
2. Prepare each step’s dispatch from the new count with the existing kernel, so that the CPU never reads the count during the run.
3. Check at the end that the remaining particles are exactly those a CPU filter would keep, and that their energy is conserved.

DONE WHEN

- [ ] The CPU reads nothing during the run, and the final check passes.
- [ ] Validation is silent, including for the new barriers.

EVIDENCE FOR THE NOTEBOOK
- The new pass, its barriers, and the result.

LAB 2.6.4 Addresses in data — 1.5 hours

Goal. Build a structure that links buffers by address.

Code. `code/u2/nbody`

1. Put the five addresses in a small buffer of their own, and pass only that buffer's address in the push constants.
2. Read the addresses in the shaders through a buffer reference to that structure.

DONE WHEN
- [ ] The simulation's results are bit-identical to the original's.
- [ ] You can explain why the alignment declared for each buffer reference type is safe.

EVIDENCE FOR THE NOTEBOOK
- The changed shaders and the comparison.

Problem set

1. How many interactions does a step perform for n = 16,384, and how many bytes does the tiled kernel read from memory per step? Compute its arithmetic intensity.
2. Write the barrier between a compute shader that writes a `VkDispatchIndirectCommand` and the `vkCmdDispatchIndirect` that reads it, and explain why a destination of `COMPUTE_SHADER` would be wrong.
3. Why can synchronisation validation not check the barriers between these kernels? What could you do to gain some confidence that they are correct?
4. A device's limit is 65,535 workgroups per dimension. Give the grid that `prepare.comp` writes for 100 million particles in workgroups of 256, and the number of invocations that do nothing. Find a grid that wastes fewer.
5. Leapfrog conserves energy over long runs where a simple Euler integrator does not. Without the mathematics, what property of leapfrog makes the difference, and how would the energy check behave with Euler?

COMPLETION CHECKLIST
- [ ] I can tile an all-pairs computation in shared memory and know when it is compute-bound.
- [ ] I can use buffer device addresses from push constants and from data, with their alignment and their risks.
- [ ] I can size dispatches on the GPU with indirect dispatch and the right barrier.
- [ ] I can dispatch problems larger than the device's limits.
- [ ] Lab 2.6.1–2.6.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.7

A ray tracer in a compute shader

TIME	8 hours
YOU BUILD	A path tracer written entirely in compute shaders: a Cornell box with spheres, quads and a mesh of 5,120 triangles in a bounding volume hierarchy built on the CPU, rendered progressively over many passes and tone-mapped for display. It checks every primary ray against a double-precision CPU reference, the hierarchy against brute force, and its light transport against the exact answer of a white furnace, and it measures how coherence and workgroup size decide its speed.
NEEDS	Chapters 2.1 to 2.6.

WHY IT MATTERS

Ray tracing appears well suited to a GPU, because every pixel is independent, but the rays scatter through the scene in different directions, take different numbers of bounces and visit different parts of the acceleration structure. A path tracer therefore exercises the whole of this unit at once: divergence, memory access, data structures in buffers, work accumulated over many dispatches, and measurement. It also needs an unusual kind of testing, because its output is random by design. Vulkan can drive dedicated ray-tracing hardware, and building a tracer in plain compute first shows what that hardware accelerates.

SKIP TEST

1. How do you test a renderer whose every pixel is a random estimate?
2. What does a bounding volume hierarchy save, and what decides how good a hierarchy is?
3. Two dispatches read and write the same storage image. Which barrier goes between them, and does the image change layout?
4. Why can a path tracer run several times slower per ray than a tracer of primary rays alone, on the same scene?

If all four are easy, read the sections on testing and on performance, and do Labs 2.7.2 and 2.7.3.

Core ideas

Rays and paths

A *ray* is a starting point and a direction, and a ray tracer finds the closest surface each ray meets. A *path tracer* uses that to estimate how much light reaches the camera through each pixel. It follows a path from the camera into the scene; at each surface it adds the light the surface emits, then picks a new direction at random, as the material scatters light, and continues. The average over many random paths converges to the light the pixel receives: a Monte Carlo estimate of the integral that Kajiya's rendering equation describes.

The scene is a *Cornell box*, a standard test scene: five walls and a light, a glass sphere, a steel sphere, and a third sphere made of 5,120 triangles, which gives the acceleration structure something to do.

The Cornell box, and its white-furnace variant for testing — code/u2/raytrace/scene.hpp

```
inline Scene cornellBox(bool furnace, float albedo, float emission) {
    Scene scene;
    auto diffuse = [&](Vec3 colour, Vec3 light = {}) {
```

```
        return addMaterial(scene, {colour, Diffuse, light, 0});
    };
    const uint32_t uniform = diffuse({albedo, albedo, albedo}, {emission, emission, emission});
    auto pick = [&](uint32_t material) { return furnace ? uniform : material; };
    const uint32_t white = pick(diffuse({0.73f, 0.73f, 0.73f}));
    const uint32_t red = pick(diffuse({0.65f, 0.05f, 0.05f}));
    const uint32_t green = pick(diffuse({0.12f, 0.45f, 0.15f}));
    const uint32_t light = pick(diffuse({0, 0, 0}, {10, 10, 10}));
    const uint32_t glass = pick(addMaterial(scene, {{1, 1, 1}, Dielectric, {}, 1.5f}));
    const uint32_t steel = pick(addMaterial(scene, {{0.8f, 0.8f, 0.85f}, Metal, {}, 0.05f}));
    const uint32_t blue = pick(diffuse({0.25f, 0.45f, 0.85f}));

    addQuad(scene, {-1, 0, -1}, {2, 0, 0}, {0, 0, 2}, white);
    addQuad(scene, {-1, 2, -1}, {2, 0, 0}, {0, 0, 2}, white);
    addQuad(scene, {-1, 0, -1}, {2, 0, 0}, {0, 2, 0}, white);
    addQuad(scene, {-1, 0, -1}, {0, 2, 0}, {0, 0, 2}, red);
    addQuad(scene, {1, 0, -1}, {0, 2, 0}, {0, 0, 2}, green);
    addQuad(scene, {-0.4f, 1.999f, -0.4f}, {0.8f, 0, 0}, {0, 0, 0.8f}, light);
    scene.spheres.push_back(
        {.centre = {-0.55f, 0.35f, 0.2f}, .radius = 0.35f, .material = glass});
    scene.spheres.push_back(
        {.centre = {0.05f, 0.4f, -0.45f}, .radius = 0.4f, .material = steel});
    addIcosphere(scene, {0.55f, 0.38f, 0.3f}, 0.38f, 4, blue);
    if (furnace) {
        scene.spheres.push_back({.centre = {0, 1, 0}, .radius = 10, .material = uniform});
    }
    scene.eye = {0, 1, 3.6f};
    scene.target = {0, 0.95f, 0};
    scene.verticalFov = 38;
    return scene;
}
```

The scene in buffers

Each kind of primitive lives in a storage buffer of its own, with the materials in another and the camera in a uniform buffer. In a `std430` array, GLSL aligns a `vec3` like a `vec4`, so a structure that mixes `vec3` and scalars has holes that the C++ side must reproduce exactly. The program writes its padding out explicitly and lets the compiler check every offset:

The scene's structures on the CPU, padded as `std430` lays them out — code/u2/raytrace/scene.hpp

```
struct Material {
    Vec3 albedo;
    uint32_t type;
    Vec3 emission;
    float parameter;
};

struct Sphere {
    Vec3 centre;
    float radius;
    uint32_t material;
    uint32_t pad[3];
};

struct Quad {
    Vec3 corner;
    uint32_t material;
    Vec3 u;
    float pad0;
    Vec3 v;
    float pad1;
};

struct Triangle {
    Vec3 v0;
    uint32_t material;
    Vec3 edge1;
    float pad0;
```

```
    Vec3 edge2;
    float pad1;
};

struct Node {
    Vec3 boundsMin;
    uint32_t leftOrFirst;
    Vec3 boundsMax;
    uint32_t count;
};
```

Layout mistakes become build errors — code/u2/raytrace/scene.hpp

```
static_assert(sizeof(Material) == 32 && offsetof(Material, emission) == 16);
static_assert(sizeof(Sphere) == 32 && offsetof(Sphere, material) == 16);
static_assert(sizeof(Quad) == 48 && offsetof(Quad, u) == 16 && offsetof(Quad, v) == 32);
static_assert(sizeof(Triangle) == 48 && offsetof(Triangle, edge1) == 16 &&
              offsetof(Triangle, edge2) == 32);
static_assert(sizeof(Node) == 32 && offsetof(Node, boundsMax) == 16);
```

A layout mistake that compiles makes the renderer draw nonsense or crash the GPU, and a `static_assert` turns it into a build error. The shaders see ten bindings in one set, and a push-constant block that names the pass:

The tracer's interface — code/u2/raytrace/scene.glsl

```
layout(std430, set = 0, binding = 1) restrict readonly buffer Materials {
    Material materials[];
};
layout(std430, set = 0, binding = 2) restrict readonly buffer Spheres {
    Sphere spheres[];
};
layout(std430, set = 0, binding = 3) restrict readonly buffer Quads {
    Quad quads[];
};
layout(std430, set = 0, binding = 4) restrict readonly buffer Triangles {
    Triangle triangles[];
};
layout(std430, set = 0, binding = 5) restrict readonly buffer Nodes {
    Node nodes[];
};
layout(set = 0, binding = 6, rgba32f) uniform restrict image2D accumulation;
layout(std430, set = 0, binding = 7) restrict buffer RayCounts {
    uint rayCounts[];
};
layout(std430, set = 0, binding = 8) restrict writeonly buffer DebugHits {
    DebugHit hits[];
};
layout(set = 0, binding = 9, rgba8) uniform restrict writeonly image2D picture;

layout(push_constant) uniform Push {
    uint pass;
    uint maxDepth;
    uint useBvh;
} push;
```

Intersections

Each pixel's ray starts at the camera and passes through a point on the image plane, jittered within the pixel so that the passes together smooth the edges:

A camera ray through a point of the image — code/u2/raytrace/trace.glsl

```
Ray cameraRay(vec2 pixel, vec2 lens) {
    vec2 uv = pixel / vec2(scene.width, scene.height);
    vec3 offset = scene.lensRadius * (lens.x * scene.right + lens.y * scene.up);
    vec3 target = scene.corner + uv.x * scene.horizontal - uv.y * scene.vertical;
    return Ray(scene.origin + offset, normalize(target - scene.origin - offset));
}
```

Each kind of primitive has a test that returns the distance along the ray to the hit. A sphere needs a quadratic equation, and a triangle the Möller–Trumbore test, which finds the hit's barycentric coordinates without computing the triangle's plane first. The constant `T_MIN`, 10^{-4}, discards hits closer than that: a ray leaving a surface would otherwise hit the same surface again through rounding. A hit records its distance and an identifier whose top four bits give the kind of primitive and whose remaining bits give its index.

Ray and sphere — code/u2/raytrace/trace.glsl

```
bool hitSphere(Sphere s, Ray ray, float tMax, out float t) {
    vec3 oc = ray.origin - s.centre;
    float b = dot(oc, ray.direction);
    float c = dot(oc, oc) - s.radius * s.radius;
    float discriminant = b * b - c;
    if (discriminant < 0.0) return false;
    float root = sqrt(discriminant);
    t = -b - root;
    if (t < T_MIN) t = -b + root;
    return t ⩾ T_MIN && t < tMax;
}
```

Ray and triangle, after Möller and Trumbore — code/u2/raytrace/trace.glsl

```
bool hitTriangle(Triangle tri, Ray ray, float tMax, out float t) {
    vec3 p = cross(ray.direction, tri.edge2);
    float determinant = dot(tri.edge1, p);
    if (abs(determinant) < 1e-12) return false;
    vec3 s = ray.origin - tri.v0;
    float u = dot(s, p) / determinant;
    vec3 q = cross(s, tri.edge1);
    float v = dot(ray.direction, q) / determinant;
    if (u < 0.0 || v < 0.0 || u + v > 1.0) return false;
    t = dot(tri.edge2, q) / determinant;
    return t ⩾ T_MIN && t < tMax;
}
```

A bounding volume hierarchy

Testing every triangle for every ray costs 5,120 tests per ray. A *bounding volume hierarchy* (BVH) is a binary tree of axis-aligned boxes in which each node's box encloses everything below it, and each leaf holds a few triangles (Figure 2.7.1). A ray that misses a box skips everything inside it, and a ray that has already hit something skips every box that starts beyond the hit.

The CPU builds the tree once, with the *surface area heuristic* (SAH). The chance that a ray through a box also passes through a smaller box inside it is roughly the ratio of their surface areas, so the expected cost of a split is the area of each side times the number of triangles it holds. Instead of trying every possible split, the builder sorts the triangles' centres into 16 bins along each axis and evaluates the 15 boundaries between the bins:

Choosing a split by binned SAH — code/u2/raytrace/bvh.hpp

```
inline BvhBuilder::Split BvhBuilder::bestSplit(uint32_t first, uint32_t count,
                                               const Bounds& centres) const {
    Split best;
    for (int axis = 0; axis < 3; ++axis) {
        if (centres.hi[axis] ⩽ centres.lo[axis]) continue;
        Bounds binBounds[bins];
        uint32_t binCounts[bins] = {};
```

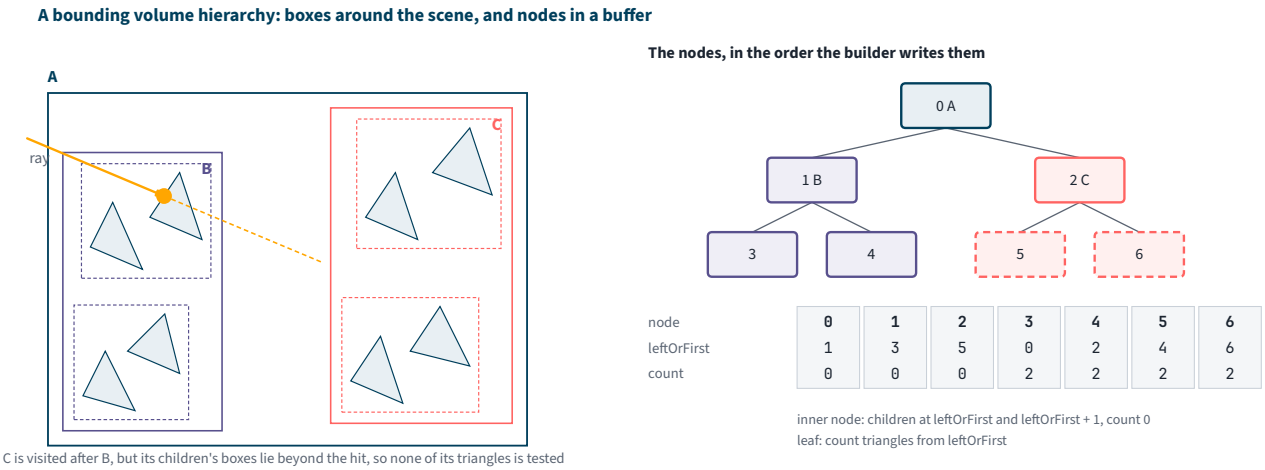


Figure 2.7.1. A hierarchy over eight triangles. Traversal visits the nearer of two children first, and the closest hit found there lets it skip the boxes of the farther one.

```
        for (uint32_t i = first; i < first + count; ++i) {
            const int bin = binOf(centroids_[order_[i]], axis, centres);
            binBounds[bin].grow(bounds_[order_[i]]);
            ++binCounts[bin];
        }
        float leftCost[bins] = {};
        Bounds left;
        uint32_t leftCount = 0;
        for (int b = 0; b < bins - 1; ++b) {
            left.grow(binBounds[b]);
            leftCount += binCounts[b];
            leftCost[b] = leftCount * left.area();
        }
        Bounds right;
        uint32_t rightCount = 0;
        for (int b = bins - 1; b > 0; --b) {
            right.grow(binBounds[b]);
            rightCount += binCounts[b];
            const float cost = leftCost[b - 1] + rightCount * right.area();
            if (cost < best.cost) best = {axis, b, cost};
        }
    }
    return best;
}
```

A node becomes a leaf when it holds two triangles or fewer, or when no split is cheaper than testing its triangles directly. Otherwise its triangles are partitioned at the chosen boundary, and its two children are written next to each other, so that one index finds both. The triangles are reordered to match, so that each leaf's triangles are contiguous:

Building the tree, depth first — code/u2/raytrace/bvh.hpp

```
inline void BvhBuilder::subdivide(uint32_t index, uint32_t first, uint32_t count,
                                  uint32_t level) {
    depth = std::max(depth, level);
    Bounds box, centres;
    for (uint32_t i = first; i < first + count; ++i) {
        box.grow(bounds_[order_[i]]);
        centres.grow(centroids_[order_[i]]);
    }
    nodes[index] = {
        .boundsMin = box.lo, .leftOrFirst = first, .boundsMax = box.hi, .count = count};
    const Split split = bestSplit(first, count, centres);
    if (count <= 2 || split.axis < 0 || split.cost >= count * box.area()) return;
    if (level + 1 >= maxBvhDepth) throw std::runtime_error("the BVH is too deep");

    const auto middle = std::partition(
```

```
        order_.begin() + first, order_.begin() + first + count,
        [&](uint32_t t) { return binOf(centroids_[t], split.axis, centres) < split.bin; });
    const auto leftCount = uint32_t(middle - (order_.begin() + first));
    if (leftCount == 0 || leftCount == count) return;
    const auto left = uint32_t(nodes.size());
    nodes.push_back({});
    nodes.push_back({});
    nodes[index].leftOrFirst = left;
    nodes[index].count = 0;
    subdivide(left, first, leftCount, level + 1);
    subdivide(left + 1, first + leftCount, count - leftCount, level + 1);
}
```

GLSL has no recursion, so the traversal keeps its own stack of node indices in a local array of 32 entries. Each level of the tree adds at most one entry, and the builder refuses to go deeper than the stack. At each inner node, the traversal measures the distance to both children's boxes, drops any box that the ray misses or that starts beyond the closest hit so far, and pushes the farther child first so that the nearer is visited next:

Traversing the hierarchy with an explicit stack — code/u2/raytrace/trace.glsl

```
void traverseBvh(Ray ray, inout Hit hit) {
    vec3 inverseDirection = 1.0 / ray.direction;
    uint stack[STACK_SIZE];
    uint top = 0;
    stack[top++] = 0;
    while (top > 0) {
        Node node = nodes[stack[--top]];
        if (node.count > 0) {
            for (uint i = node.leftOrFirst; i < node.leftOrFirst + node.count; ++i) {
                testTriangle(i, ray, hit);
            }
            continue;
        }
        uint nearChild = node.leftOrFirst, farChild = node.leftOrFirst + 1;
        float tNear = boxDistance(nodes[nearChild], ray, inverseDirection, hit.t);
        float tFar = boxDistance(nodes[farChild], ray, inverseDirection, hit.t);
        if (tFar < tNear) {
            uint child = nearChild;
            nearChild = farChild;
            farChild = child;
            float t = tNear;
            tNear = tFar;
            tFar = t;
        }
        if (tFar < FAR) stack[top++] = farChild;
        if (tNear < FAR) stack[top++] = nearChild;
    }
}
```

For this scene the builder makes 6,109 nodes, 14 levels deep. Without the hierarchy the tracer is 96 times slower.

Random numbers and materials

Each invocation needs its own stream of random numbers, different for every pixel and every pass. The tracer runs a PCG generator, whose state is a single 32-bit integer, seeded from a hash of the pixel's index and the pass number:

A random-number generator per invocation code/u2/raytrace/trace.glsl

```
uint rngState;

// The hash and the generator follow Jarzynski and Olano, "Hash Functions for GPU Rendering".
uint pcgHash(uint value) {
    uint state = value * 747796405u + 2891336453u;
    uint word = ((state >> ((state >> 28u) + 4u)) ^ state) * 277803737u;
    return (word >> 22u) ^ word;
}

void seedRandom(uint pixel, uint pass) {
    rngState = pcgHash(pixel ^ pcgHash(pass));
}

float random() {
    rngState = rngState * 747796405u + 2891336453u;
    uint word = ((rngState >> ((rngState >> 28u) + 4u)) ^ rngState) * 277803737u;
    return float(((word >> 22u) ^ word) >> 8) * (1.0 / 16777216.0);
}
```

The materials scatter light in three ways. A diffuse surface sends it in a random direction with the cosine-weighted distribution of an ideal matte surface, which adding a random unit vector to the normal produces exactly. Metal reflects about the normal, blurred by a random offset. Glass refracts by Snell's law, or reflects with the probability that Schlick's approximation gives, and always reflects beyond the critical angle.

Three materials code/u2/raytrace/trace.glsl

```
bool scatter(Material m, vec3 incoming, Surface s, out vec3 direction) {
    if (m.type == DIFFUSE) {
        direction = s.normal + randomUnitVector();
        direction = dot(direction, direction) > 1e-8 ? normalize(direction) : s.normal;
        return true;
    }
    if (m.type == METAL) {
        direction = normalize(reflect(incoming, s.normal) + m.parameter * randomUnitVector());
        return dot(direction, s.normal) > 0.0;
    }
    float eta = s.frontFace ? 1.0 / m.parameter : m.parameter;
    float cosine = min(dot(-incoming, s.normal), 1.0);
    bool mustReflect = eta * sqrt(1.0 - cosine * cosine) > 1.0;
    if (mustReflect || schlick(cosine, m.parameter) > random()) {
        direction = reflect(incoming, s.normal);
    } else {
        direction = refract(incoming, s.normal, eta);
    }
    return true;
}
```

The path loop adds each surface's emission, weighted by the product of the albedos of the surfaces before it, the path's *throughput*. A path ends when it leaves the scene, when a material absorbs it, or at the depth limit. After three bounces the loop plays *Russian roulette*: it continues with a probability equal to the throughput's largest component, at most 0.95, and divides the throughput by that probability when it does. Paths that could contribute little end early, and the division keeps the estimate's expected value unchanged.

Following a path code/u2/raytrace/trace.glsl

```
vec3 tracePath(Ray ray, inout uint rays) {
    vec3 radiance = vec3(0.0);
    vec3 throughput = vec3(1.0);
    for (uint depth = 0; depth < push.maxDepth; ++depth) {
        Hit hit = closestHit(ray);
        ++rays;
        if (hit.id == MISS) break;
        Surface s = surfaceAt(hit, ray);
        Material m = materials[s.material];
        radiance += throughput * m.emission;
        vec3 direction;
        if (!scatter(m, ray.direction, s, direction)) break;
        throughput *= m.albedo;
        if (depth >= 3) {
            float survival = min(max(throughput.r, max(throughput.g, throughput.b)), 0.95);
            if (random() >= survival) break;
            throughput /= survival;
        }
        ray = Ray(s.position, direction);
    }
    return radiance;
}
```

Accumulating over passes

Each pass traces one path per pixel and adds its radiance to an `rgba32f` storage image whose alpha channel counts the samples. A shared counter totals the rays its workgroup traced, and one atomic addition per workgroup adds them to a buffer, which gives the measurements below their ray counts.

One path per pixel per pass code/u2/raytrace/render.comp

```
void main() {
    uvec2 p = gl_GlobalInvocationID.xy;
    if (gl_LocalInvocationIndex == 0) groupRays = 0;
    barrier();
    uint rays = 0;
    if (p.x < scene.width && p.y < scene.height) {
        seedRandom(p.y * scene.width + p.x, push.pass);
        vec2 jitter = vec2(random(), random());
        vec3 radiance = tracePath(cameraRay(vec2(p) + jitter, randomInDisk()), rays);
        vec4 sum = push.pass == 0 ? vec4(0.0) : imageLoad(accumulation, ivec2(p));
        imageStore(accumulation, ivec2(p), sum + vec4(radiance, 1.0));
    }
    atomicAdd(groupRays, rays);
    barrier();
    if (gl_LocalInvocationIndex == 0) atomicAdd(rayCounts[push.pass], groupRays);
}
```

Each pass is a dispatch of its own. The next pass reads and writes the pixels this one wrote, so a barrier orders them, from storage writes to storage reads and writes in the compute stage. The image stays in `VK_IMAGE_LAYOUT_GENERAL`, the layout storage images use, and the barrier names it on both sides: an image barrier without a layout transition, used for its memory dependency alone.

Passes, with a barrier between each and the next — code/u2/raytrace/main.cpp

```
for (uint32_t pass = first; pass < last; ++pass) {
    if (pass > 0) {
        vkf::imageBarrier(cmd, gpu.accumulation, VK_IMAGE_LAYOUT_GENERAL,
                          VK_IMAGE_LAYOUT_GENERAL,
                          VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                          VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
                          VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                          VK_ACCESS_2_SHADER_STORAGE_READ_BIT |
                              VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT);
    }
    const Push push{pass, depth, bvh ? 1u : 0u};
    vkCmdPushConstants(cmd, layouts.pipeline, VK_SHADER_STAGE_COMPUTE_BIT, 0,
                       sizeof(push), &push);
    timer.stamp(cmd);
    vkCmdDispatch(cmd, vkf::groupCount(gpu.camera.width, shape.x),
                  vkf::groupCount(gpu.camera.height, shape.y), 1);
    timer.stamp(cmd);
}
```

Short passes, 1.5 ms each here, keep the GPU responsive. A dispatch that runs for seconds can trip the operating system's GPU timeout, which on Windows resets the device after two seconds. Short passes also let an interactive viewer show the image as it converges. A last pass divides each pixel's sum by its count, compresses the range with Narkowicz's fit to the ACES filmic curve, and applies the sRGB curve itself before writing an 8-bit image, since sRGB storage images are not portable (Chapter 1.2).

Resolving the accumulated sums for display — code/u2/raytrace/resolve.comp

```
// Narkowicz's fit to the ACES filmic tone curve.
vec3 toneMap(vec3 x) {
    return clamp(x * (2.51 * x + 0.03) / (x * (2.43 * x + 0.59) + 0.14), 0.0, 1.0);
}

vec3 encodeSrgb(vec3 c) {
    vec3 curve = 1.055 * pow(c, vec3(1.0 / 2.4)) - 0.055;
    return mix(12.92 * c, curve, greaterThan(c, vec3(0.0031308)));
}

void main() {
    uvec2 p = gl_GlobalInvocationID.xy;
    if (p.x >= scene.width || p.y >= scene.height) return;
    vec4 sum = imageLoad(accumulation, ivec2(p));
    imageStore(picture, ivec2(p), vec4(encodeSrgb(toneMap(sum.rgb / sum.a)), 1.0));
}
```

Testing a random renderer

A renderer can be wrong and still draw a plausible picture: a triangle slightly out of place, a material that loses energy, a hierarchy that drops a node. The program checks three things, from exact to statistical.

The first check is exact. A debugging kernel traces one ray through the centre of each pixel of a small image, with no randomness, and records what it hit and how far away:

Primary rays, recorded for checking — code/u2/raytrace/debug.comp

```
void main() {
    uvec2 p = gl_GlobalInvocationID.xy;
    if (p.x >= scene.width || p.y >= scene.height) return;
    Hit hit = closestHit(cameraRay(vec2(p) + 0.5, vec2(0.0)));
    hits[p.y * scene.width + p.x] = DebugHit(hit.t, hit.id);
}
```

The CPU traces the same rays in double precision through every primitive, and the two must agree. Rounding can legitimately change the answer for a ray that grazes an edge or meets two surfaces at the same distance, so where they disagree, the CPU traces the ray again with every primitive shrunk and grown

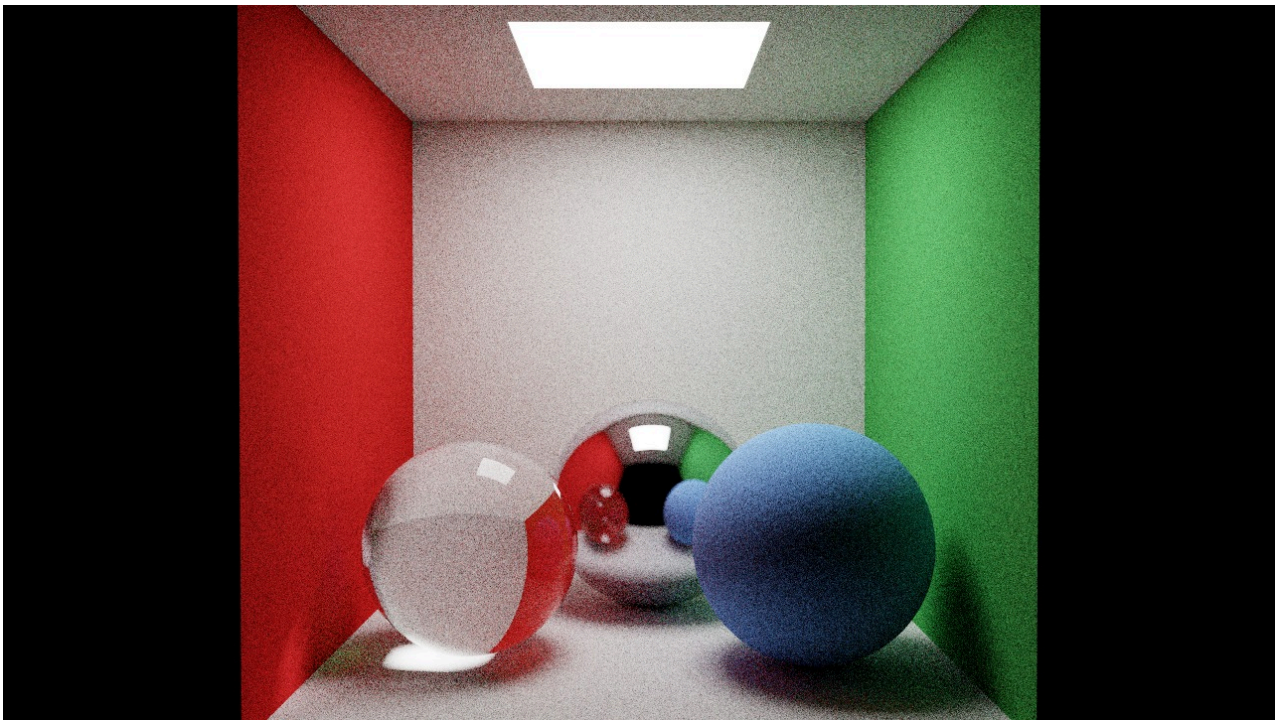


Figure 2.7.2. The Cornell box at 1280 × 720 with 512 paths per pixel, which took 3 seconds on the M2 Pro.

by 10^{-4}. It accepts the GPU's answer only if the GPU's primitive, grown, is hit at the GPU's distance, and nothing in the shrunk scene is hit noticeably closer.

Every primary ray against a double-precision reference — code/u2/raytrace/main.cpp

```
PrimaryCheck checkPrimaryRays(const rt::Scene& scene, const rt::Camera& camera,
                              std::span<const DebugHit> hits) {
    const double margin = 1e-4, distanceTolerance = 1e-3;
    PrimaryCheck check;
    for (uint32_t y = 0; y < camera.height; ++y) {
        for (uint32_t x = 0; x < camera.width; ++x) {
            const DebugHit gpu = hits[y * camera.width + x];
            const rt::ReferenceRay ray = rt::primaryRay(camera, x, y);
            const rt::ReferenceHit exact = rt::closestHit(scene, ray, 0.0);
            bool acceptable = false;
            if (gpu.id == exact.id) {
                const double difference =
                    gpu.id == rt::missId ? 0 : std::abs(gpu.t - exact.t) / exact.t;
                check.largestDifference = std::max(check.largestDifference, difference);
                acceptable = difference <= distanceTolerance;
                if (acceptable) ++check.agree;
            } else {
                const rt::ReferenceHit shrunk = rt::closestHit(scene, ray, -margin);
                acceptable = shrunk.id == rt::missId;
                if (gpu.id != rt::missId) {
                    const double grown = rt::hitPrimitive(scene, gpu.id, ray, margin);
                    acceptable = std::abs(grown - gpu.t) <= distanceTolerance * gpu.t &&
                                 shrunk.t >= gpu.t * (1 - distanceTolerance);
                }
                if (acceptable) ++check.edges;
            }
            if (!acceptable) ++check.wrong;
        }
    }
    return check;
}
```

The second check compares the hierarchy with brute force. The same rays, traced through the BVH and through every triangle, must give bit-identical distances, and the same primitive except at exact ties. Both use the same intersection code on the same triangles, so any other difference is a node the traversal lost.

The hierarchy against brute force — code/u2/raytrace/main.cpp

```
GpuScene view(ctx, layouts, pool, showcase, 128, 72, 1);
const auto withBvh = primaryHits(ctx, layouts, view, pipelines.debug, o.shape, true);
const auto bruteForce = primaryHits(ctx, layouts, view, pipelines.debug, o.shape, false);
const PrimaryCheck primary = checkPrimaryRays(showcase, view.camera, withBvh);
uint32_t same = 0, ties = 0;
for (size_t i = 0; i < withBvh.size(); ++i) {
    const bool sameT =
        std::bit_cast<uint32_t>(withBvh[i].t) == std::bit_cast<uint32_t>(bruteForce[i].t);
    if (sameT && withBvh[i].id == bruteForce[i].id) {
        ++same;
    } else if (sameT) {
        ++ties;
    }
}
```

The third check tests the light transport, which has no exact answer for any one pixel. In a *white furnace*, every surface has the same albedo a and emits the same radiance E, and the scene is closed, here by a large sphere around everything. Every path then meets a surface at every bounce, whatever the geometry, and a path of at most D bounces collects $E\,(1 + a + a^2 + \ldots + a^{D-1}) = E\,(1 - a^D) / (1 - a)$ on average. The mean over all pixels must lie within five standard errors of that value. This check covers what the first two cannot: the scattering, the random numbers and the roulette.

The furnace's statistics — code/u2/raytrace/main.cpp

```
Furnace furnaceStatistics(const vkf::Context& ctx, GpuScene& gpu, float albedo, float emission,
                          uint32_t depth) {
    const std::vector<uint8_t> bytes =
        vkf::downloadImage(ctx, gpu.accumulation, VK_IMAGE_LAYOUT_GENERAL);
    std::vector<float> sums(bytes.size() / sizeof(float));
    std::memcpy(sums.data(), bytes.data(), bytes.size());
    Furnace f;
    double total = 0, squares = 0;
    const size_t pixels = sums.size() / 4;
    for (size_t i = 0; i < pixels; ++i) {
        const double mean =
            (sums[4 * i] + sums[4 * i + 1] + sums[4 * i + 2]) / (3 * sums[4 * i + 3]);
        f.finite = f.finite && std::isfinite(mean);
        total += mean;
        squares += mean * mean;
    }
    f.mean = total / pixels;
    const double variance = (squares / pixels - f.mean * f.mean) * pixels / (pixels - 1);
    f.standardError = std::sqrt(std::max(variance, 0.0) / pixels);
    f.expected = emission * (1 - std::pow(albedo, depth)) / (1 - albedo);
    return f;
}
```

Each check was tested by breaking the code it guards. A traversal that skips a child fails the first two checks, a sign error in the triangle test fails the first, and roulette without the division by the survival probability fails the furnace.

The checks and a render on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_raytrace
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
scene: 2 spheres, 6 quads and 5120 triangles in a BVH of 6109 nodes, depth 14
check 1: primary rays at 128 x 72 against a double-precision CPU reference
  9216 agree (largest distance difference 1.2e-05), 0 at edges or ties, 0 wrong
check 2: BVH and brute force agree on 9216 of 9216 primary rays (0 exact ties)
check 3: white furnace, albedo 0.5 and emission 0.5, at most 16 bounces, 64 samples
  mean radiance 1.00083, expected 0.99998, standard error 3.9e-04: ok

render: 640 x 360, 64 samples, at most 8 bounces, BVH on, workgroups of 8 x 8
  1.532 ms per pass (median), 624205 rays per pass, 414.8 Mrays/s
wrote raytrace.png
PASS raytrace: primary hits, BVH and furnace checks hold
```

Coherence and workgroups

Five configurations of the tracer, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u2_raytrace --benchmark
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
scene: 2 spheres, 6 quads and 5120 triangles in a BVH of 6109 nodes, depth 14
check 1: primary rays at 128 x 72 against a double-precision CPU reference
  9216 agree (largest distance difference 1.2e-05), 0 at edges or ties, 0 wrong
check 2: BVH and brute force agree on 9216 of 9216 primary rays (0 exact ties)
check 3: white furnace, albedo 0.5 and emission 0.5, at most 16 bounces, 64 samples
  mean radiance 1.00083, expected 0.99998, standard error 3.9e-04: ok

640 x 360, 8 samples per configuration
configuration                        ms/pass     rays/pass    Mrays/s
BVH, 8 x 8, full paths                 1.540        624366      405.1
BVH, 16 x 16, full paths               2.032        624366      295.3
BVH, 64 x 1, full paths                1.648        624366      378.8
BVH, 8 x 8, primary rays only          0.185        230400     1235.1
brute force, 8 x 8, full paths       149.336        624366        4.2
PASS raytrace: primary hits, BVH and furnace checks hold
```

Primary rays alone run at 1.24 billion rays per second, and full paths at 405 million, three times fewer. The difference comes from coherence. Rays from neighbouring pixels start together, hit the same surfaces and visit the same nodes, so the invocations of a subgroup run the same instructions on nearby data. After a bounce, each ray leaves in a random direction of its own. The invocations of a subgroup then visit different nodes, test different triangles, meet different materials and end after different numbers of bounces, and a subgroup runs at the speed of its slowest invocation (Chapter 2.4).

The size of a workgroup matters more here than its shape. With 64 invocations, square tiles of 8 × 8 are 7% faster than strips of 64 × 1, because their subgroups start with rays from a compact patch of the image. Workgroups of 16 × 16 are 27% slower than 8 × 8. A sweep of shapes on the M2 Pro finds 32 × 8 as slow as 16 × 16, while every shape from 32 to 128 invocations runs within 7% of 8 × 8. Each invocation keeps a 32-entry stack as well as its path's state, and the more state each invocation keeps, the fewer invocations a core can hold. A core holds only whole workgroups, so the larger the workgroup, the more of that capacity is left unused. On the M2 Pro, a stack of 16 entries, enough for this tree's depth of 14, makes workgroups of 8 × 8 about 14% faster and workgroups of 16 × 16 about 35% faster.

Hardware ray tracing

Many GPUs now have hardware for the work this chapter does in software: traversing a hierarchy of boxes and intersecting triangles. Vulkan exposes it through extensions. `VK_KHR_acceleration_structure` builds hierarchies on the device, in a format only the driver knows; `VK_KHR_ray_query` lets any shader, a compute shader included, trace a ray through one; and `VK_KHR_ray_tracing_pipeline` adds a pipeline with shaders for ray generation, hits and misses. A tracer that uses ray queries keeps the camera, the materials, the paths, the accumulation and the tests of this chapter, and replaces only the hierarchy and the intersection code.

KEY IDEA

A path tracer is a Monte Carlo estimate computed in parallel, with every pixel independent and every path different. Keep the data structures in buffers whose layouts are checked at compile time, accumulate over short passes with a barrier between each, and test exactly what has an exact answer and the rest statistically.

COMING FROM OPENGL

OpenGL 4.3 can run this tracer almost line for line: the scene in shader storage buffers, the accumulation in an image bound with `glBindImageTexture`, and `glMemoryBarrier(GL_SHADER_IMAGE_ACCESS_BARRIER_BIT)` between passes. Its compute shaders use the same `std430` layouts, so the layout checks carry over unchanged.

Reading

- Peter Shirley, Trevor David Black and Steve Hollasch, *Ray Tracing in One Weekend* and its sequels: the camera and materials this tracer follows.
- Matt Pharr, Wenzel Jakob and Greg Humphreys, *Physically Based Rendering*, fourth edition, MIT Press, 2023: chapter 2 for Monte Carlo integration and chapter 7 for hierarchies built with the surface area heuristic.
- Jacco Bikker, "How to build a BVH", a series of articles from 2022, for binned construction and traversal on the CPU and the GPU.
- Mark Jarzynski and Marc Olano, "Hash Functions for GPU Rendering", *Journal of Computer Graphics Techniques* 9(3), 2020.
- The Khronos Group, "Ray Tracing In Vulkan", 2020, on the ray-tracing extensions.

Labs

LAB 2.7.1 Measure convergence — 1.5 hours

Goal. Measure how noise falls with the number of samples.

Code. `code/u2/raytrace`

1. Render with `--spp` set to 16, 64, 256 and 1024, renaming each image.
2. Taking the 1024-sample image as the reference, compute the root-mean-square difference of each other image from it over a region of the back wall.
3. Plot the differences against the number of samples on logarithmic axes.

DONE WHEN

- [ ] The slope of your plot is within 0.1 of −½.
- [ ] You can explain why four times as many samples halve the noise, and why the reference's own noise flattens the slope slightly.

EVIDENCE FOR THE NOTEBOOK

- The plot and the four images.

LAB 2.7.2 Break it on purpose — 1.5 hours

Goal. Find out which bugs each check detects.

Code. `code/u2/raytrace`

1. Make three changes, one at a time: push only the nearer child in the traversal; reverse the test `u < 0.0` in the triangle intersection; remove the division by the survival probability in the path loop.
2. Run the program after each change and record which checks fail.

DONE WHEN
- [ ] Every change fails at least one check.
- [ ] For each change, you can explain why the checks that passed could not see it.

EVIDENCE FOR THE NOTEBOOK
- The three outputs and the explanations.

LAB 2.7.3 Size the stack — 1.5 hours

Goal. Measure how workgroup size and stack size affect the tracer's speed.

Code. `code/u2/raytrace`

1. Run `VKF_VALIDATION=0 build/bin/u2_raytrace` with `--local` set to `8x4`, `8x8`, `64x1`, `16x8`, `16x16` and `32x8`, and record the rays per second.
2. Reduce `STACK_SIZE` in `trace.glsl` to 16, and `maxBvhDepth` in `bvh.hpp` to match, then repeat.

DONE WHEN
- [ ] You have both tables.
- [ ] You can explain from them why workgroup size matters more than shape here, and why the stack can be no smaller than the tree's depth.

EVIDENCE FOR THE NOTEBOOK
- The tables and the explanation.

LAB 2.7.4 Measure the divergence — 2 hours

Goal. Measure the lane use of full paths.

Code. `code/u2/raytrace`

1. Record, for each pixel, the number of bounces its path took and the subgroup that traced it.
2. On the CPU, compute the lane use of each subgroup as in Chapter 2.4: the bounces traced, over the subgroup's largest number of bounces times its number of lanes.
3. Compare the mean lane use of workgroups of 8×8 and 64×1.

DONE WHEN
- [ ] You have the mean lane use for both shapes.
- [ ] You can relate the lane use to the speeds you measured in Lab 2.7.3.

EVIDENCE FOR THE NOTEBOOK
- The changes and the numbers.

Problem set

1. The default render traces 624,205 rays per pass for 640×360 pixels. What is the mean number of rays per path? Would it rise or fall without Russian roulette, and why?
2. A hierarchy of 6,109 nodes holds 5,120 triangles. How many leaves does it have, and how many triangles does a leaf hold on average? Why do some leaves hold more than two?
3. Derive the furnace's expected radiance, and explain why it does not depend on the scene's geometry. What else must be true of the scene for the test to be valid?

4. The primary-ray check accepts some disagreements. What bug could hide behind that allowance, and how does the second check limit the damage?
5. What would be gained and lost by tracing all of a pixel's samples in one dispatch, with a loop in the shader?

COMPLETION CHECKLIST

- [ ] I can trace rays through spheres, quads and triangles in a compute shader, with the scene's layouts checked at compile time.
- [ ] I can build a bounding volume hierarchy with the surface area heuristic and traverse it with an explicit stack.
- [ ] I can accumulate a Monte Carlo estimate over many dispatches with the barrier it needs.
- [ ] I can test a random renderer exactly where answers are exact, and statistically elsewhere.
- [ ] I can explain a tracer's speed from coherence, workgroup size and the state each invocation keeps.
- [ ] Lab 2.7.1–2.7.4 done-when criteria all hold, with evidence filed.

UNIT 2

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and repository, and you can do each review task below without notes. Date each line as you finish it. If someone who writes GPU code for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
2.1	The compute execution model	☐	☐	
2.2	Memory in shaders	☐	☐	
2.3	Parallel patterns	☐	☐	
2.4	Images and image processing	☐	☐	
2.5	Measuring and tuning	☐	☐	
2.6	Dispatch at scale	☐	☐	
2.7	A ray tracer in a compute shader	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook and code.

- ☐ Explain, with a drawing of your own, how a dispatch becomes workgroups, subgroups and invocations, and where shared memory and `barrier()` act.
- ☐ Place each of your Unit 2 kernels on your GPU's roofline from your Lab 2.5 measurements, and say for each what limits it.
- ☐ Write a multi-level scan from memory, with its barriers, and explain how it extends to any size.
- ☐ Explain why the N-body simulation uses indirect dispatch and device addresses, and what the barrier between the preparation kernel and the dispatch must name.
- ☐ Explain what each of the ray tracer's three checks proves, and which bug each would miss.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____